Reinforcement Fine-Tuning LLMs With GRPO

https://www.deeplearning.ai/courses/reinforcement-fine-tuning-llms-grpo

Reinforcement Fine-Tuning LLMs With GRPO **1**
Introduction Reinforcement Learning **3**
Supervised Fine-Tuning (SFT) 3
Forward Pass, backward pass 3
SFT Dataset 3
SFT Data + Reasoning 5
Limitations of Supervised Fine-Tuning 6
Heavily relies on large (100s of examples) accurately labeled datasets 6
Model may learn pattern too specific to training domain (overfitting) 7
Reinforcement learning : intuition 7
Agent learn to complete a task by trying different action and receiving rewards, with goal of getting most reward over time 7
Action : Puppy performs trick 9
Reward: you give or withhold a treat 9
Observation: puppy notice the result and update its memory 9
Applying Reinforcement Fine-Tuning To LLMs 9
Algorithm for Reinforcement Learning 10
Reinforcement learning with human feedback (RLHF) 11
Benefits of reinforcement fine-tuning **18**
Doesn’t require labeled data, just a mean to ‘verify” correctness 19
Pytorch to Triton Translation Performance 20
When should you use Reinforcement Fine-Tuning 20
You have no labeled data 20
You have limited labeled data 21
Chain of thought (COT) reasoning improves performance 21
Task suited for Reinforcement Fine-tuning 22
Mathematical Problem solving 22
Code Generation and debugging 22
Logical and multi-step reasoning 22
When should you choose RFT ? 22
Can A Large language model master wordle? 23
Wordle-style 25
Load Qwen2.5 7B model 26
Helper class 27
render_user_prompt 29
Render_prmpt 30
Generate Stream 31
Generate Prompt 33
Sent prompt to LLM (1st chat completion) 34
Get Feedback function 38
Next turn function 40
Start game: “BRICK” 42
Second guess 44
For fine tuning 44
inital 44
Load fine tuned model 45
After first guess, analyze result 46
List of guess word 47

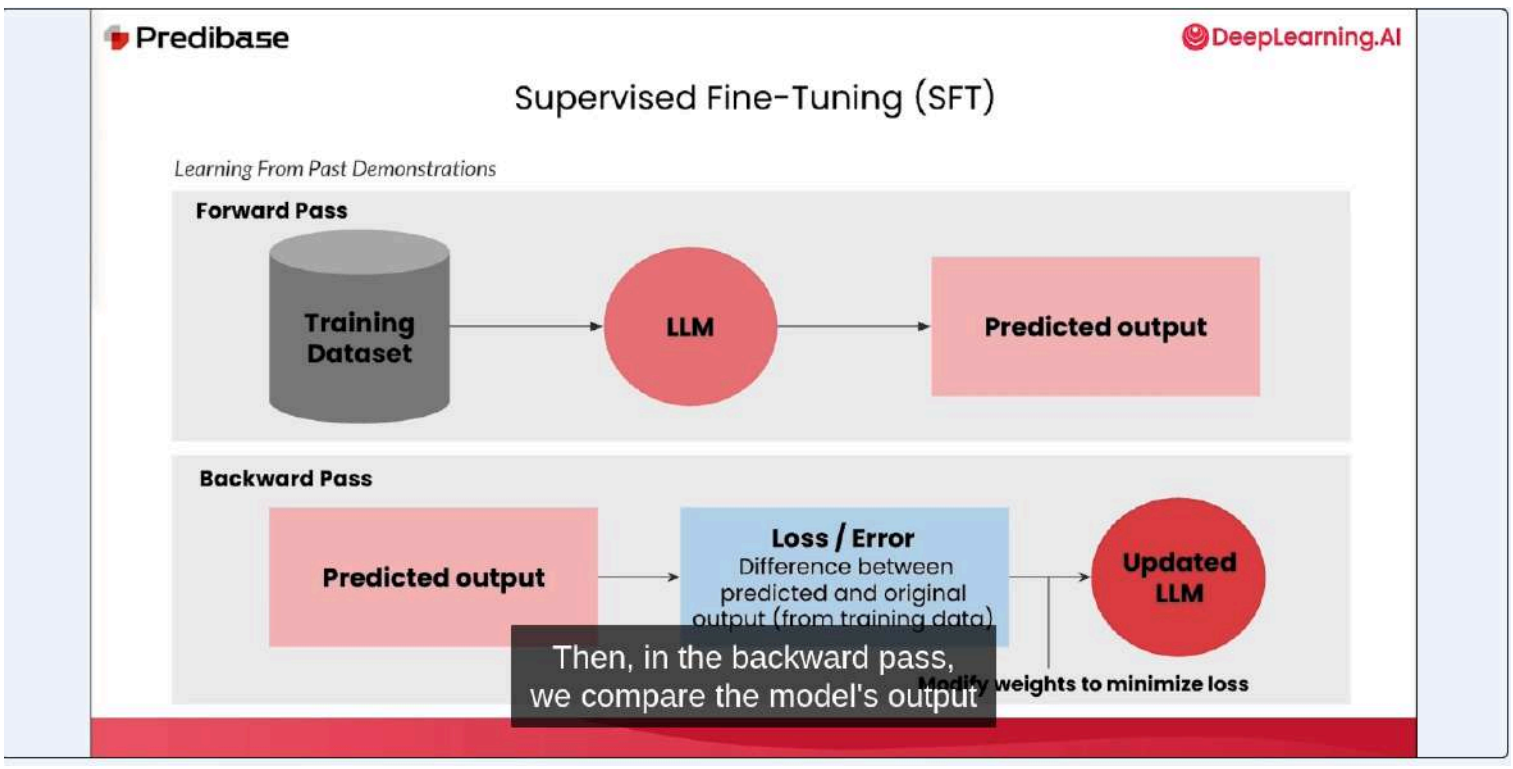
Second guess 49
List of guess word 50
After filter out invalid option 50
Finial answer 52
Compare base model, fine tuning model iterative thought though reasoning process 53
Reward functions 54
Load deployment for base model 55
Model : qwen2.5 7B instruct 55
World_reward 56
Run guesses word 57
Model guess words → reward =0 incorrect 58
How rewards are used to learn 59
Step 1: Generate response 59
Step 2: Assign score to response using reward function 59
Agent Goal : maximum reward received 60
Advantages 61
Calculating Advantages 62
Ai = ri - mean ({r1, r2, …, rG}) / std({r1, r2, … , rG}) , where i =--- 62
Compute_advantages 63
Calculate Rewards 64
render_guess_table 65
New wordle reward partial credit function 67
Calculate render_guess_table 68
temperature = 0 68
Temperature = 1.3 69
Temperature = 0.7 70
Reward functions with LLM as a judge 72
Initial LLM 74
Load dataset 75
Summarize prompt 76
Summarize function 76
Run summarize, 78
A lot of unwanted summary 78
Judge prompt : rating 78
Jude reward function 79
Run judge_reward score function 82
Generate 8 summarize output sample, calculate scores 83
Generate multiple voice 84
Pydantic schema 86
Str 88
Quiz 89
Reward Hacking 99
Load Dataset 99
Prompt transcrpt 100
Higher rewards from longer summaries 102
Length penalty reward 105
Advantages calculate 109
Calculating loss in GRPO 112
GRPO algorithm 113
Load model and tokenizer 115
Base model 116
Reference and policy model in GRPO 118
Create reference model 119
Apply LoRA 119

Print model 120
prepare_inputs 121
attention_mask 123
Compute_log_probs 124
Grop_loss funtion 125
Ratio of probabilities 126
Calculate grop_loss 130
Calculate ratio 131
Clipping 131
Grpo_loss with clipped 132
Pick the minimum ratio 133
Add polcy_loss 134
Rename to grop_loss_with_clip 135
Calculate Grop_loss_with_clip 136
ref_token_log_probs 138
KL Divergence 141
grpo_loss_with_kl 142
Beta * per_token_kl 143
Plot KL_divergence 145
Course correcting with KL divergence 146
Come back to reference model 149
Beta term in KL divergence 149
Beta increase → loss decreasing 151
Putting it all together: Training wordle 153
RFT for Wordle 154
Build prompt 154
Get 16 response 155
Get score using 3 rewarding function 155
** Reward function for wordle 156
1 Output format check 156
2 Use previous feedback 157
3 Guess value 158
Full code in reward_function.py 159
Use reward score → calculate advantage , calculate loss 160
Project inital 161
Load dataset 163
Create new repository in Predibase 164
Import reward function 165
Create fine tuning job 166
Fine tuning 167
Results 168
After GPRO result 169
**Combining RFT and SFT 169
Step1 SFT 170
Step 2 GRPO 171
Use SFT model → GRPO (guess faster convergence) 171
Combine SFT+ GRPO result 171

Introduction Reinforcement Learning

Supervised Fine-Tuning (SFT)

Forward Pass, backward pass



SFT Dataset

Predibase

DeepLearning.AI

SFT Dataset

Prompt	Completion
Natalia sold clips to 48 of her friends in April, and then she sold half as many clips in May. How many clips did Natalia sell altogether in April and May?	72
Weng earns $12 an hour for babysitting. Yesterday, she just did 50 minutes of babysitting. How much did she earn?	10
James writes a 3-page letter to 2 different friends twice a week. How many pages does he write a year?	624

and it will learn the patterns
to produce these outputs,

SFT Data + Reasoning

Predibase DeepLearning.AI

SFT Dataset + Reasoning

Prompt	Completion
Natalia sold clips to 48 of her friends in April, and then she sold half as many clips in May. How many clips did Natalia sell altogether in April and May?	<think>Natalia sold 48/2 = <<48/2=24>>24 clips in May. Natalia sold 48+24 = <<48+24=72>>72 clips altogether in April and May</think> <answer>72</answer>
Weng earns $12 an hour for babysitting. Yesterday, she just did 50 minutes of babysitting. How much did she earn?	<think>Weng earns 12/60 = $<<12/60=0.2>>0.2 per minute. Working 50 minutes, she earned 0.2 x 50 = $<<0.2*50=10>>10.</think> <answer>10</answer>
James writes a 3-page letter to 2 different friends twice a week. How many pages does he write a year?	<think>He writes each friend 3*2=<<3*2=6>>6 pages a week So 6*2=<<6*2=12>>12 pages every week, or 12*52=<<12*52=624>>624 pages a year</think>

this way, you can teach the model

learn.deeplearning.ai – To exit full screen, drag from the top and touch the back button

Predibase DeepLearning.AI

SFT Dataset + Reasoning

Prompt	Completion
Natalia sold clips to 48 of her friends in April, and then she sold half as many clips in May. How many clips did Natalia sell altogether in April and May?	<think>Natalia sold 48/2 = <<48/2=24>>24 clips in May. Natalia sold 48+24 = <<48+24=72>>72 clips altogether in April and May</think> <answer>72</answer>
Weng earns $12 an hour for babysitting. Yesterday, she just did 50 minutes of babysitting. How much did she earn?	<think>Weng earns 12/60 = $<<12/60=0.2>>0.2 per minute. Working 50 minutes, she earned 0.2 x 50 = $<<0.2*50=10>>10.</think> <answer>10</answer>
James writes a 3-page letter to 2 different friends twice a week. How many pages does he write a year?	<think>He writes each friend 3*2=<<3*2=6>>6 pages a week So 6*2=<<6*2=12>>12 pages every week, or 12*52=<<12*52=624>>624 pages a year</think>

that leads from the prompt to the desired solution.

Predibase DeepLearning.AI

SFT Dataset + Reasoning

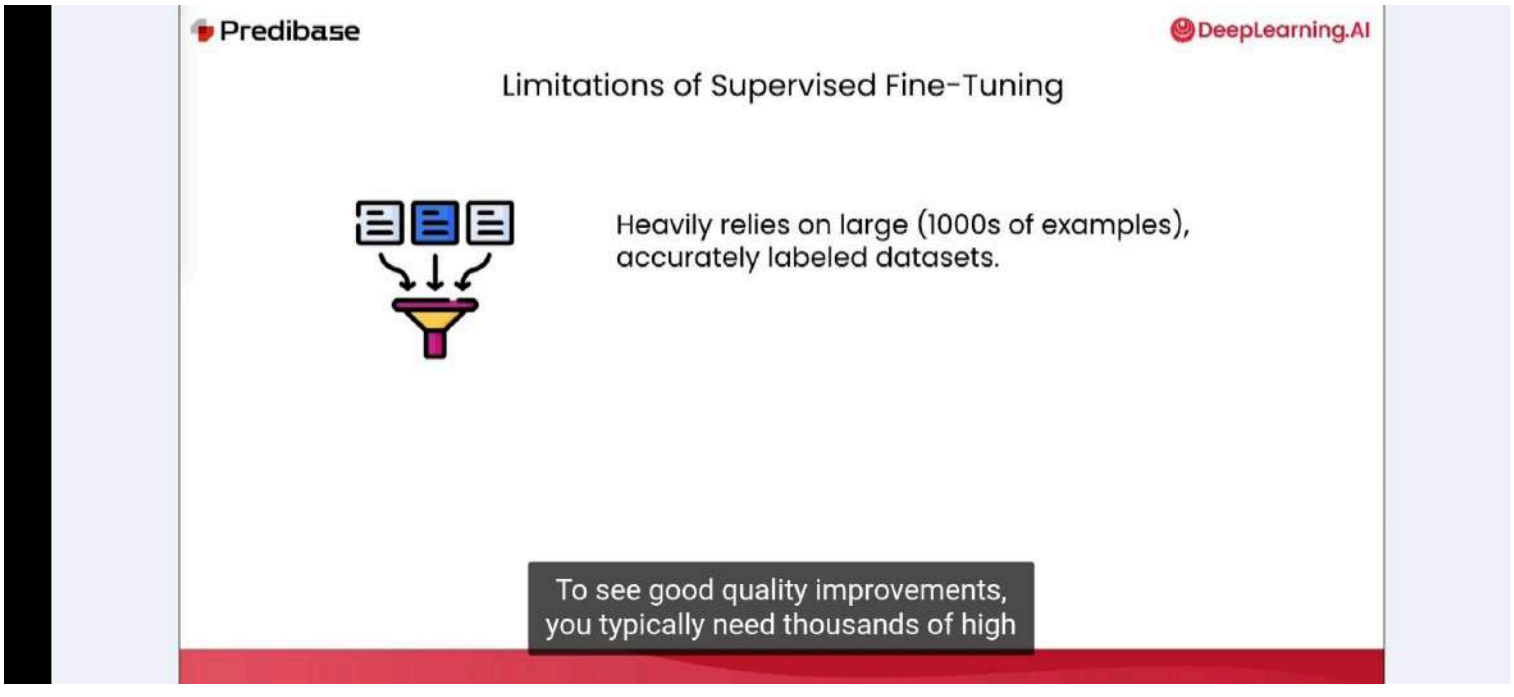
Prompt	Completion
Natalia sold clips to 48 of her friends in April, and then she sold half as many clips in May. How many clips did Natalia sell altogether in April and May?	<think>Natalia sold 48/2 = <<48/2=24>>24 clips in May. Natalia sold 48+24 = <<48+24=72>>72 clips altogether in April and May</think> <answer>72</answer>
Weng earns $12 an hour for babysitting. Yesterday, she just did 50 minutes of babysitting. How much did she earn?	<think>Weng earns 12/60 = $<<12/60=0.2>>0.2 per minute. Working 50 minutes, she earned 0.2 x 50 = $<<0.2*50=10>>10.</think> <answer>10</answer>
James writes a 3-page letter to 2 different friends twice a week. How many pages does he write a year?	<think>He writes each friend 3*2=<<3*2=6>>6 pages a week So 6*2=<<6*2=12>>12 pages every week, or 12*52=<<12*52=624>>624 pages a year</think>

that leads from the prompt

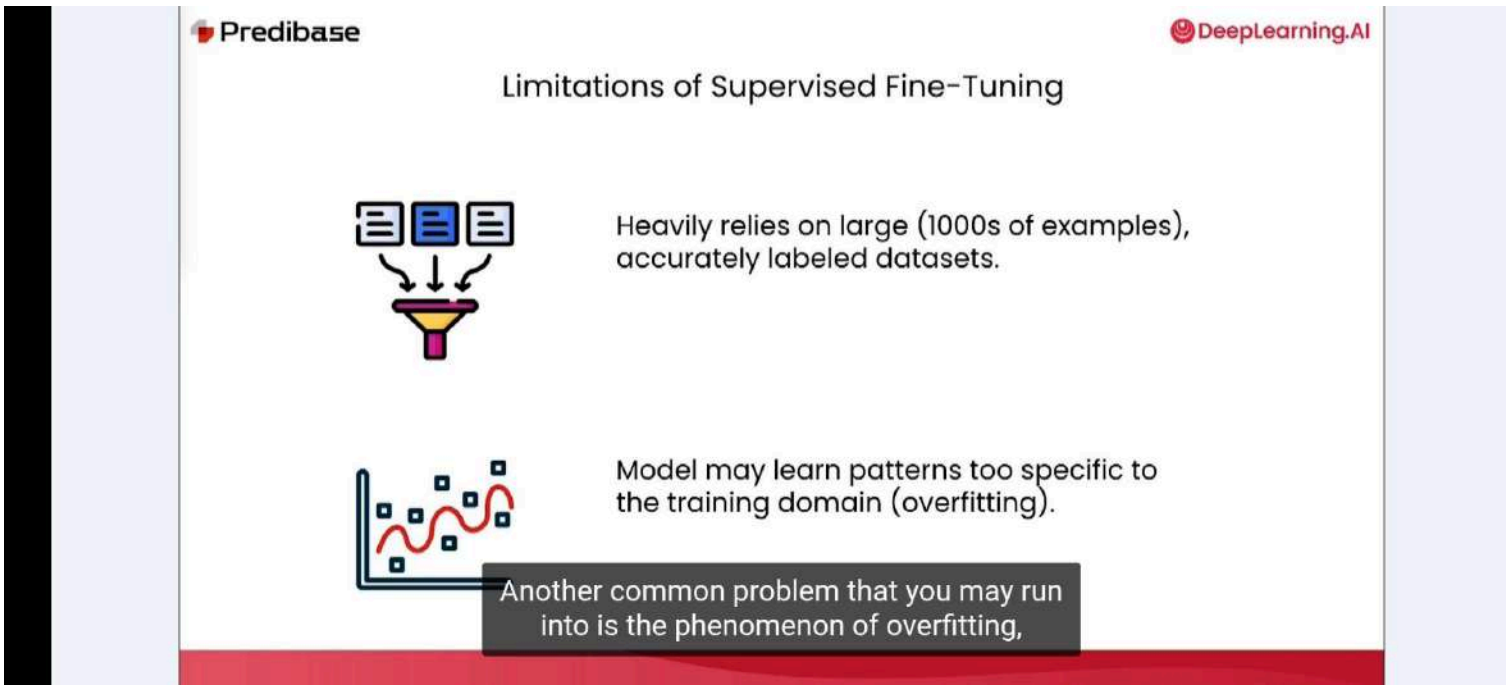
learn.deeplearning.ai – To exit full screen, drag from the top and touch the back button

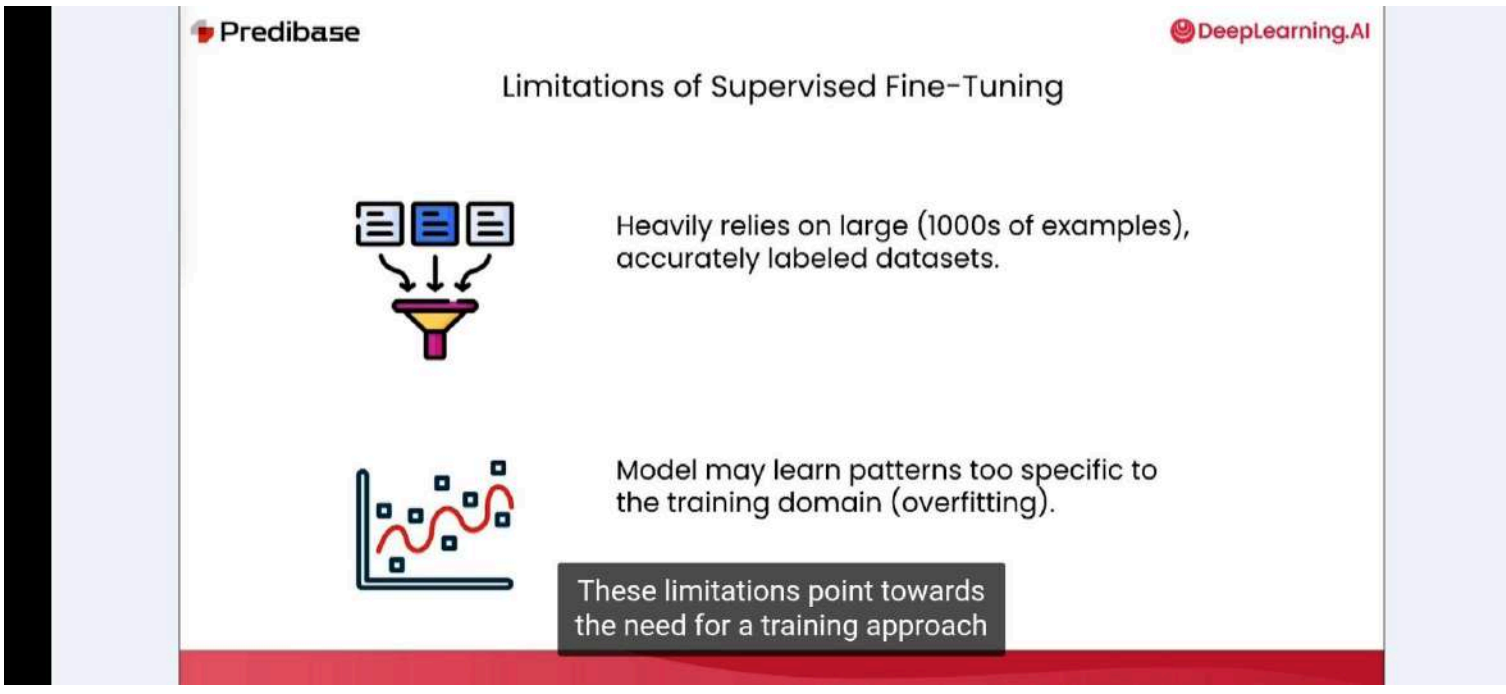
Limitations of Supervised Fine-Tuning

Heavily relies on large (100s of examples) accurately labeled datasets



Model may learn pattern too specific to training domain (overfitting)





Reinforcement learning : intuition

Agent learn to complete a task by trying different action and receiving rewards, with goal of getting most reward over time

Predibase

DeepLearning.AI

Reinforcement Learning: Intuition

Reinforcement Learning: An agent learns to complete a task by trying different actions and receiving rewards, with goal of getting the most reward over time.

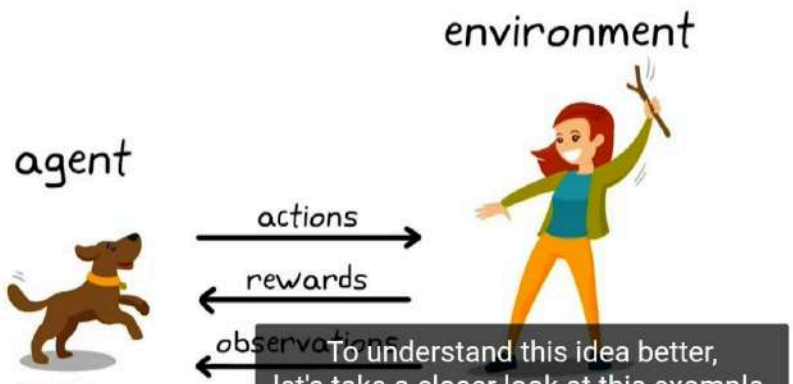


Image Source: Mathworks

To understand this idea better, let's take a closer look at this example.

Predibase

DeepLearning.AI

Reinforcement Learning: Intuition

Reinforcement Learning: An agent learns to complete a task by trying different actions and receiving rewards, with goal of getting the most reward over time.

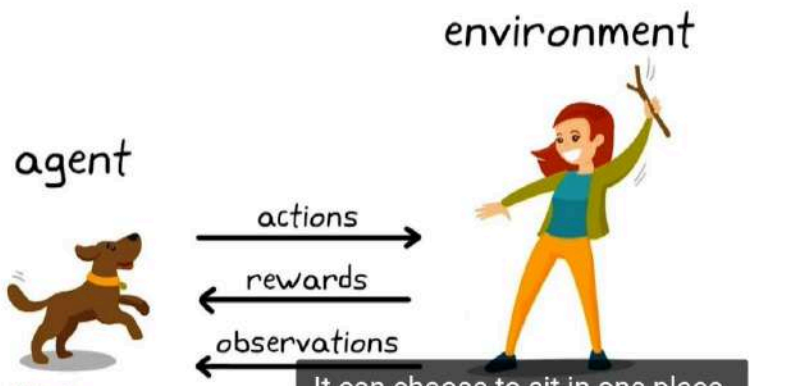


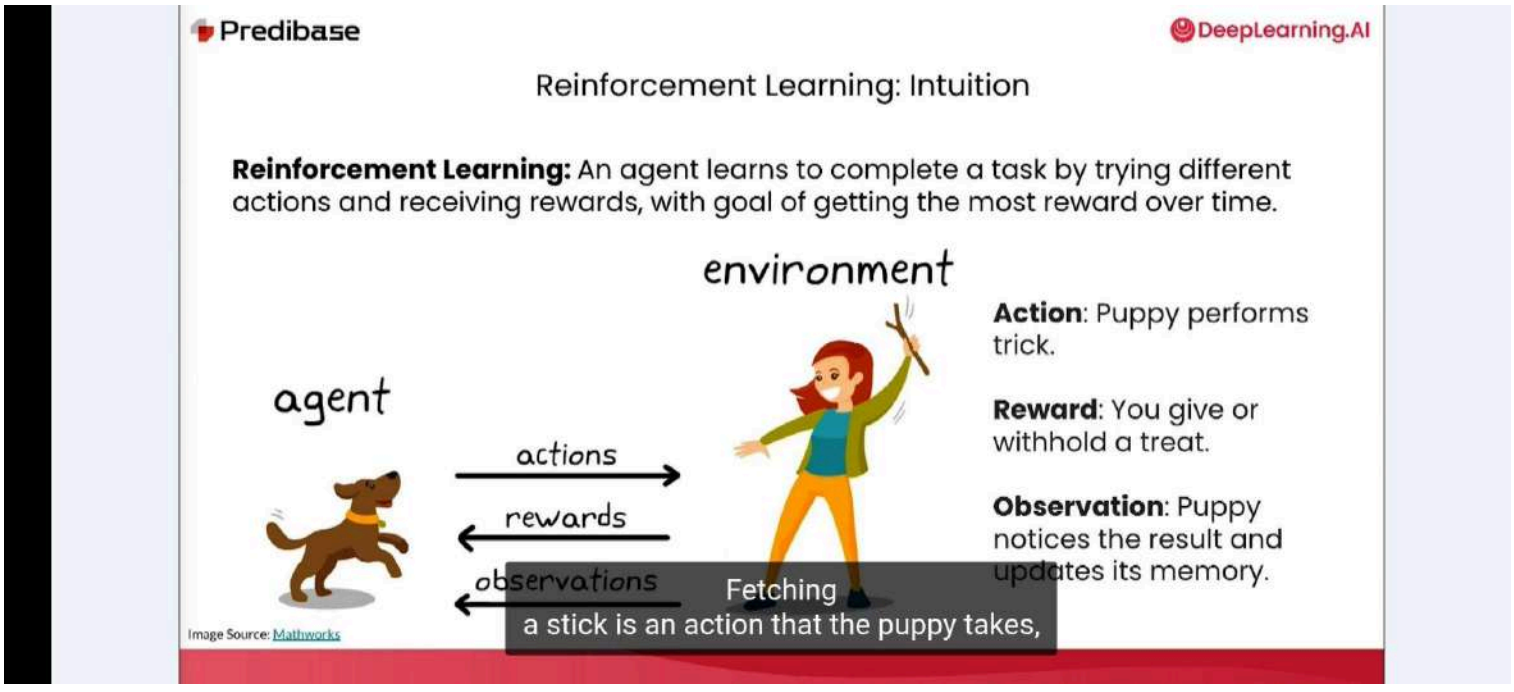
Image Source: Mathworks

It can choose to sit in one place.

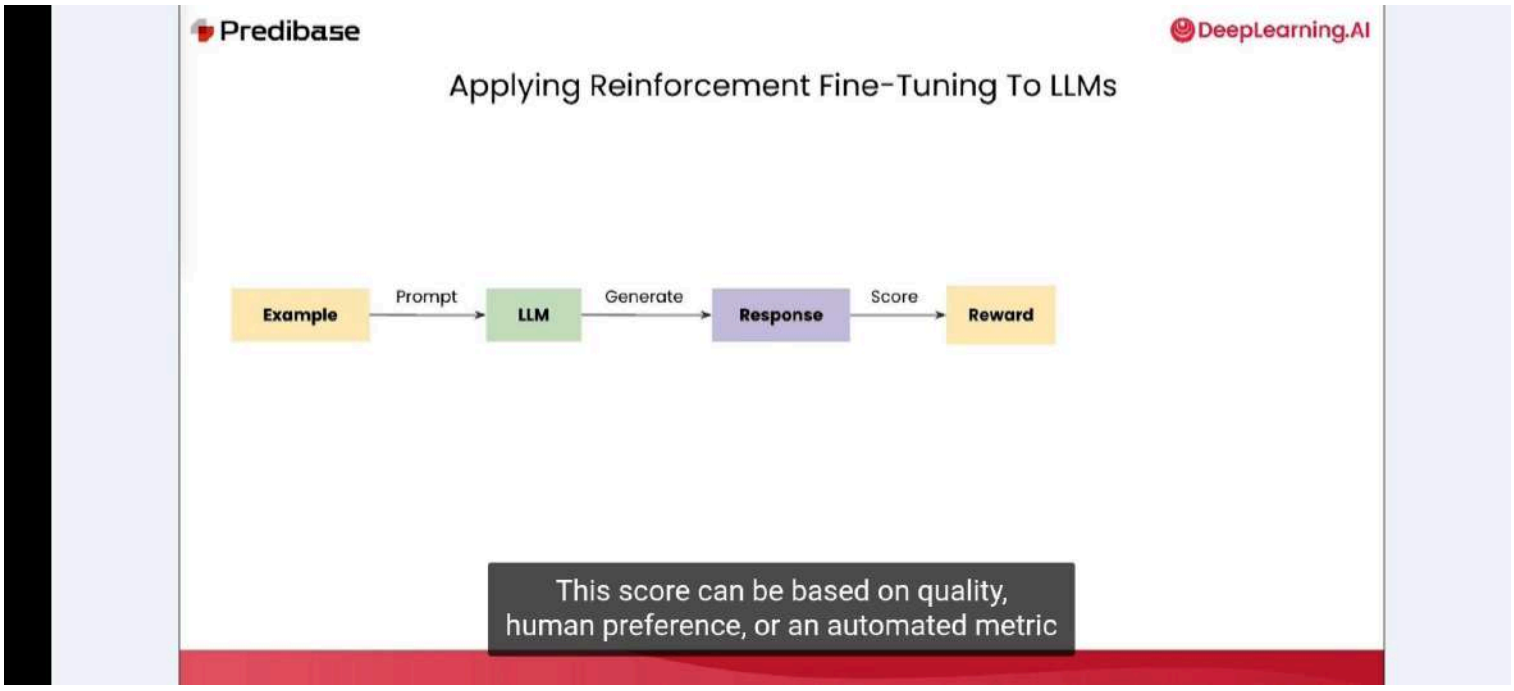
Action : Puppy performs trick

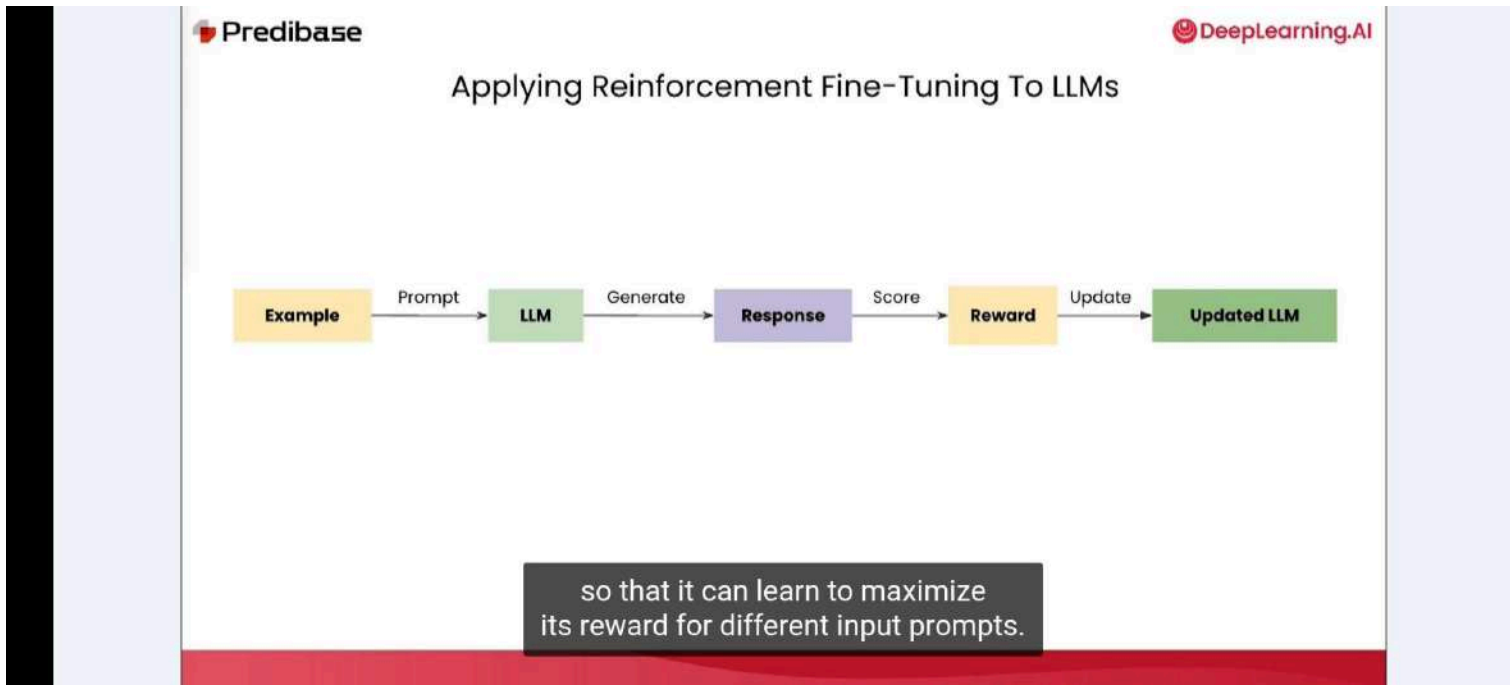
Reward: you give or withhold a treat

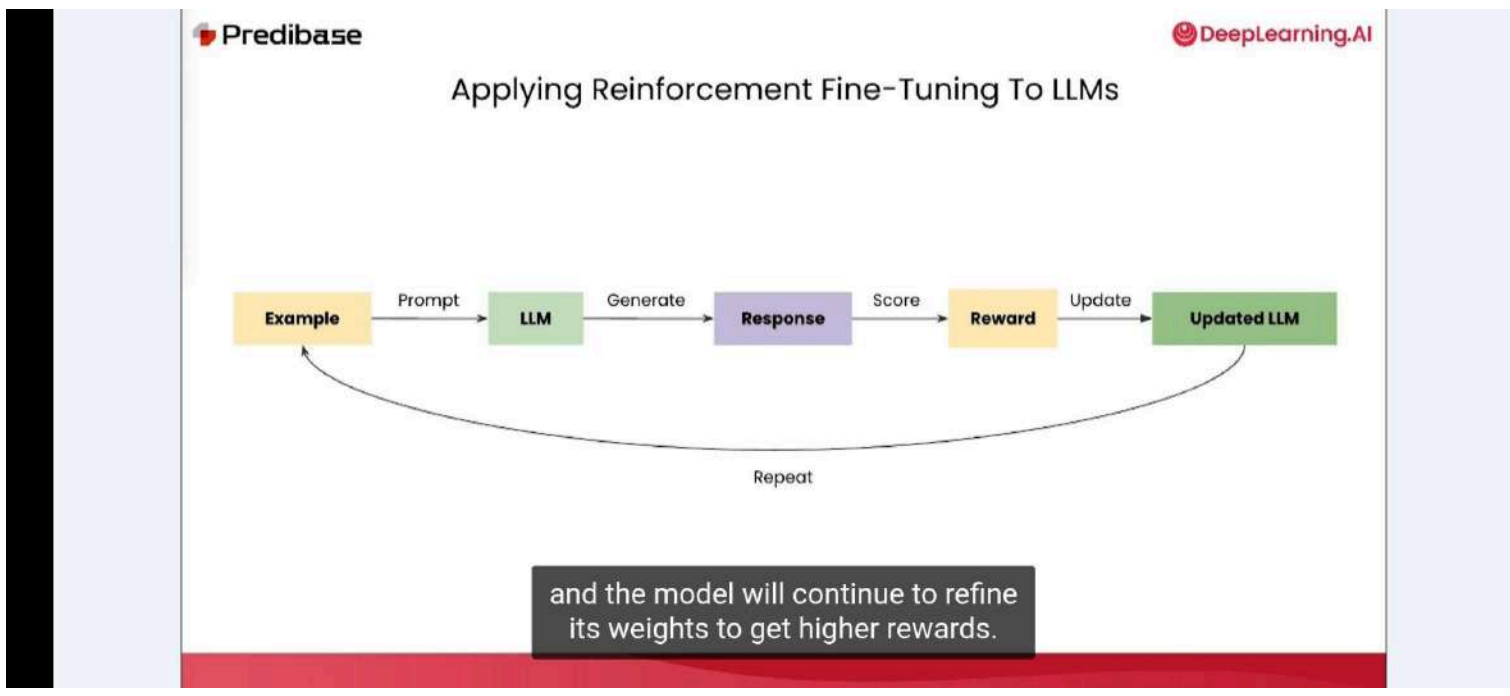
Observation: puppy notice the result and update its memory



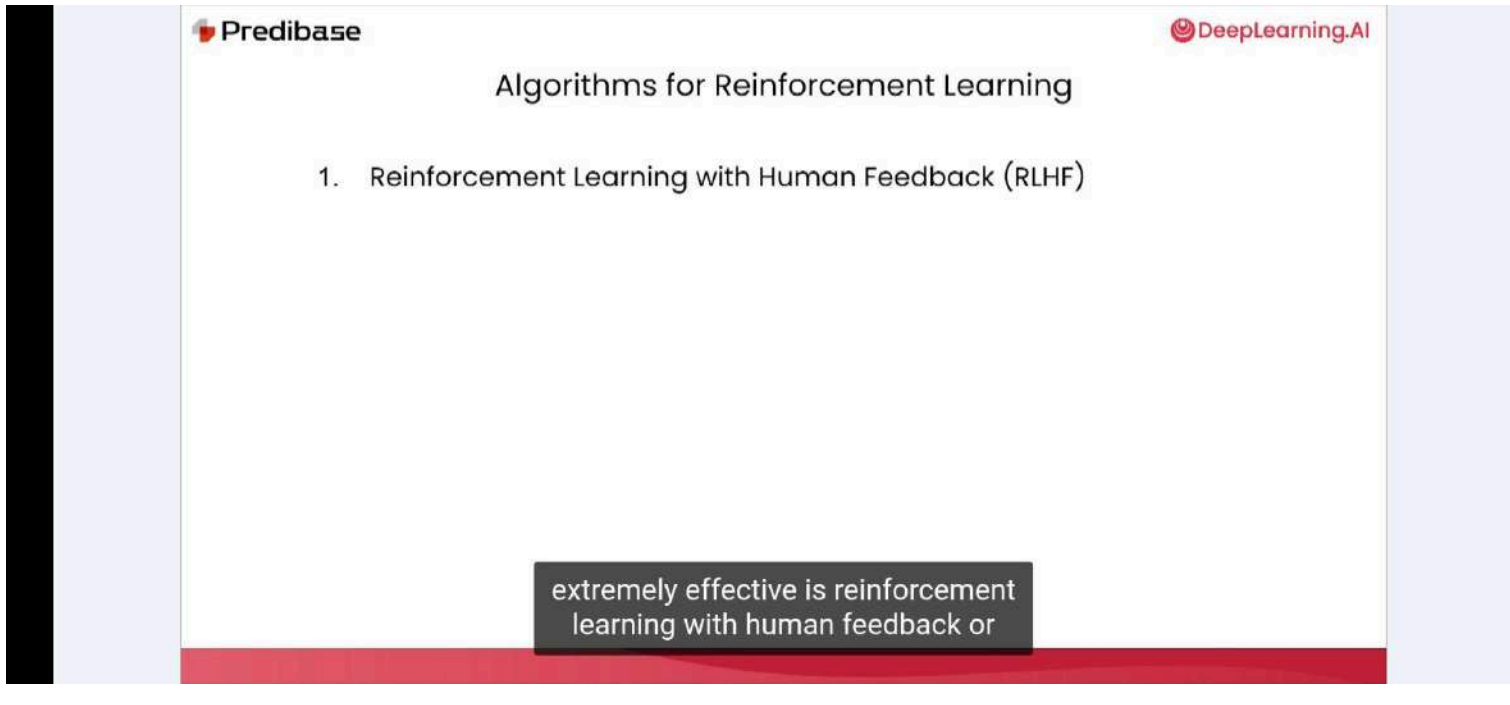
Applying Reinforcement Fine-Tuning To LLMs







Algorithm for Reinforcement Learning



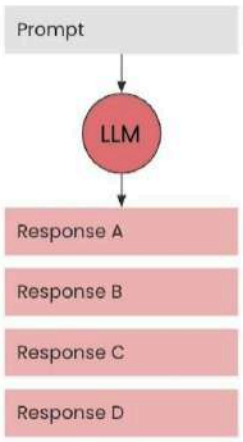
Reinforcement learning with human feedback (RLHF)

Predibase

DeepLearning.AI

Reinforcement Learning with Human Feedback (RLHF)

Step 1: Generate responses



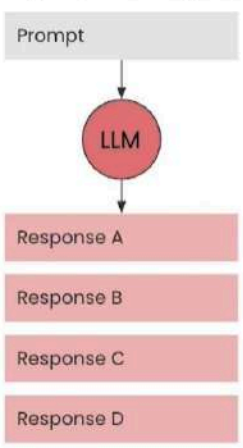
In step two,

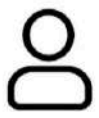
Predibase

DeepLearning.AI

Reinforcement Learning with Human Feedback (RLHF)

Step 1: Generate responses *Step 2: Get human feedback*





Human ranks responses:

C > D > A > B

This produces a preference ranking

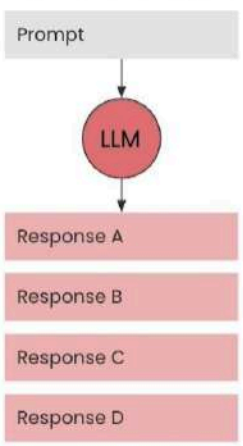
learn.deeplearning.ai – To exit full screen, drag from the top and touch the back button

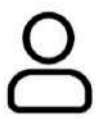
Predibase

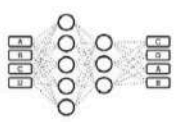
DeepLearning.AI

Reinforcement Learning with Human Feedback (RLHF)

Step 1: Generate responses *Step 2: Get human feedback* *Step 3: Train reward model*







Human ranks responses:

C > D > A > B

we train a separate reward model to learn to predict these human preferences.

Predibase

DeepLearning.AI

Reinforcement Learning with Human Feedback (RLHF)

Step 1: Generate responses

Prompt

LLM

Response A

Response B

Response C

Response D

Step 2: Get human feedback

Human ranks responses:

C > D > A > B

Step 3: Train reward model

Step 4: Update LLM weights using reward model and PPO algorithm

Prompt

LLM

Response A

Finally, in step four, we find in the original LLM

Predibase

DeepLearning.AI

Reinforcement Learning with Human Feedback (RLHF)

Step 1: Generate responses

Prompt

LLM

Response A

Response B

Response C

Response D

Step 2: Get human feedback

Human ranks responses:

C > D > A > B

the reward model scores it, and the LLM's weights are updated

Step 3: Train reward model

Step 4: Update LLM weights using reward model and PPO algorithm

Prompt

LLM

Response A

Reward score

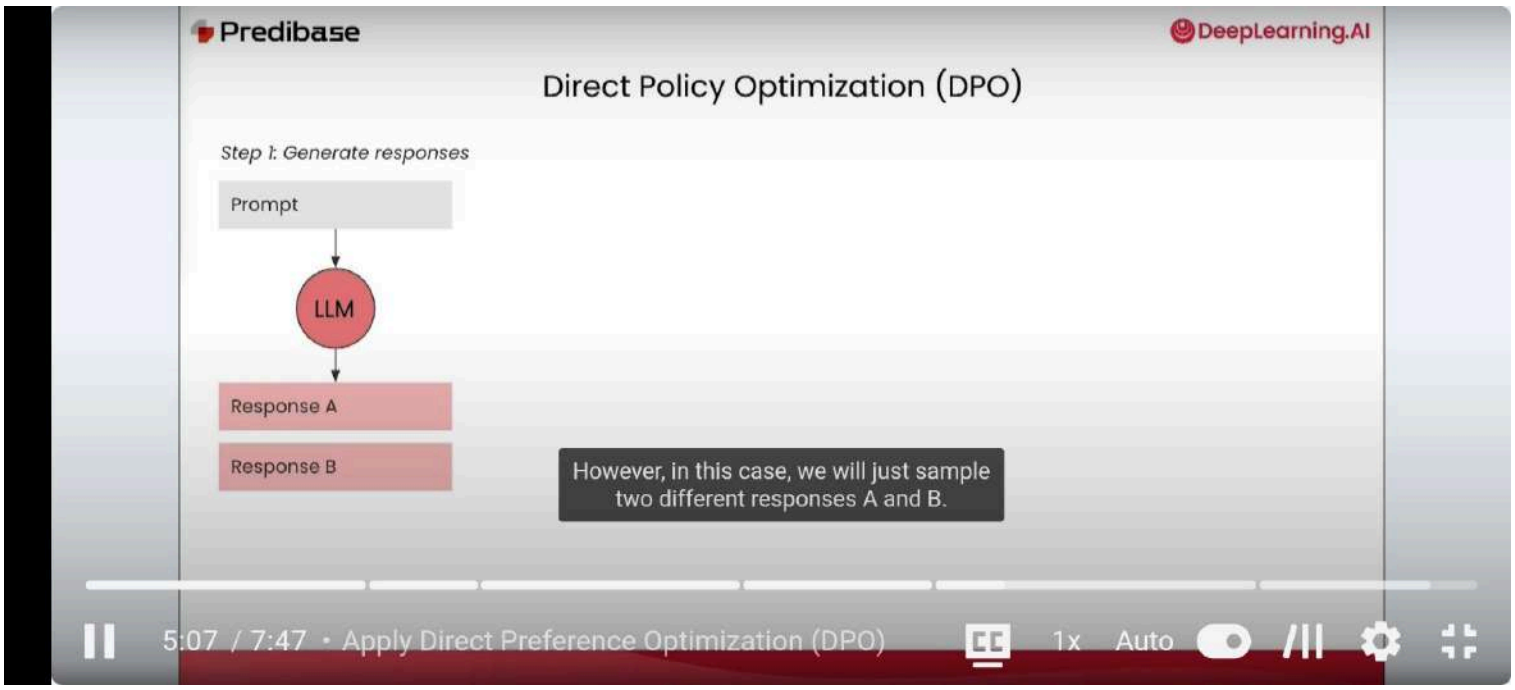
Predibase

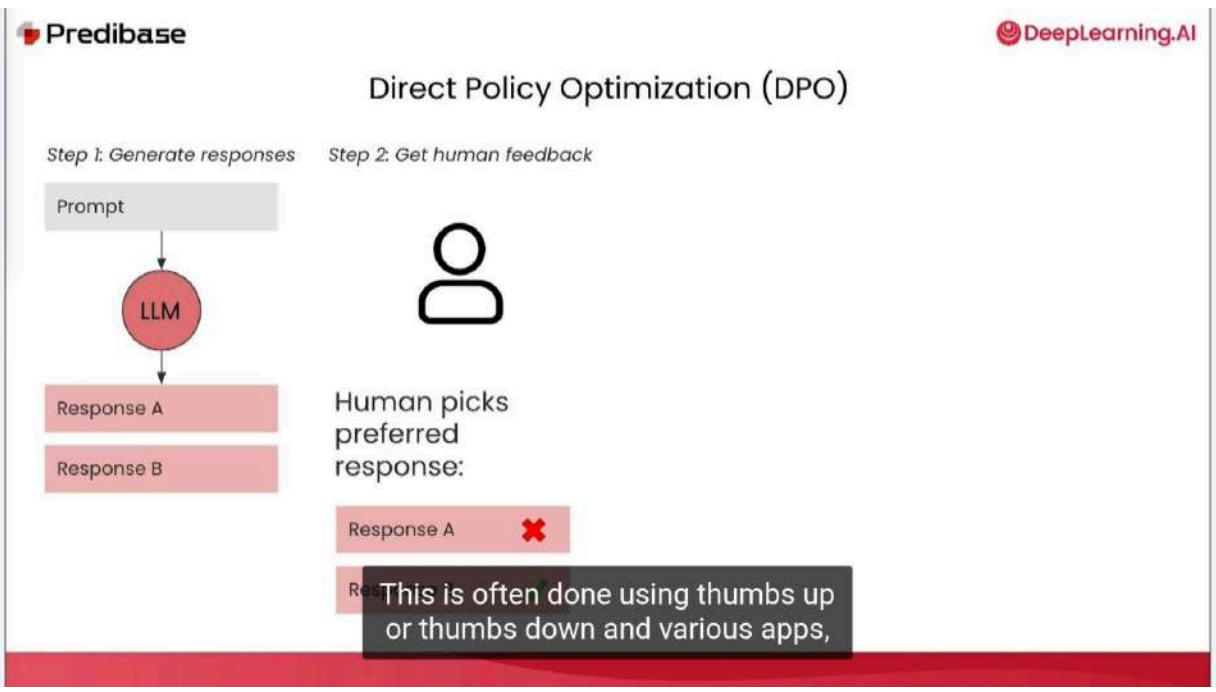
DeepLearning.AI

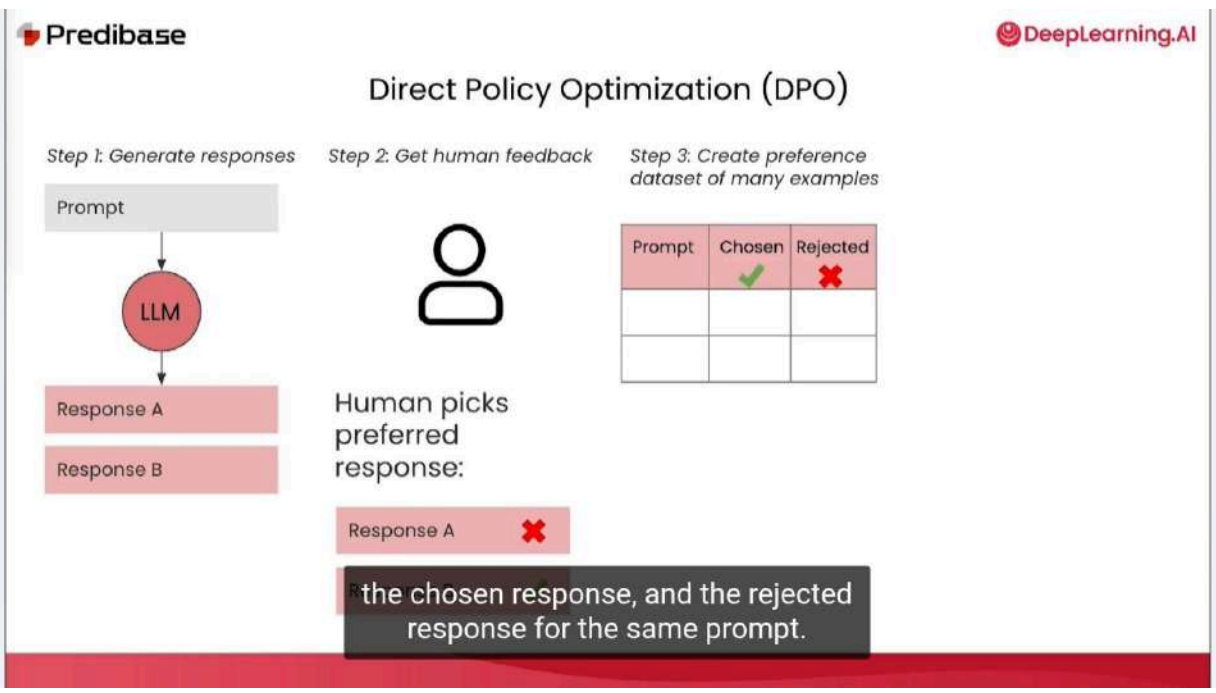
Algorithms for Reinforcement Learning

1. Reinforcement Learning with Human Feedback (RLHF)
2. Direct Policy Optimization (DPO)

But instead of first training a separate reward model, it directly fine-tunes the LLM


Predibase
DeepLearning.AI
Direct Policy Optimization (DPO)
Step 1: Generate responses
Prompt
LLM
Response A
Response B
However, in this case, we will just sample two different responses A and B.
5:07 / 7:47 • Apply Direct Preference Optimization (DPO)
1x
Auto


Predibase
DeepLearning.AI
Direct Policy Optimization (DPO)
Step 1: Generate responses
Step 2: Get human feedback
Prompt
LLM
Response A
Response B
Human picks preferred response:
Response A
This is often done using thumbs up or thumbs down and various apps,


Predibase
DeepLearning.AI
Direct Policy Optimization (DPO)
Step 1: Generate responses
Step 2: Get human feedback
Step 3: Create preference dataset of many examples
Prompt
LLM
Response A
Response B
Human picks preferred response:
Response A
Prompt
Chosen
Rejected
the chosen response, and the rejected response for the same prompt.

Predibase

DeepLearning.AI

Direct Policy Optimization (DPO)

Step 1: Generate responses

Prompt

LLM

Response A

Response B

Step 2: Get human feedback

Human picks preferred response:

Response A ✖

Response B

Step 3: Create preference dataset of many examples

Prompt	Chosen ✔	Rejected ✖

Step 4: Update LLM weights using preference dataset and DPO algorithm

Prompt ✖

Prompt ✔

LLM

P P

DPO updates weights

The idea behind the training

Predibase

DeepLearning.AI

Challenges With RLHF and DPO

Challenge	RLHF
Data Needed	Ranked generations for reward model
Compute / Memory	Very high
Training Stability	Often unstable—reward hacking, collapse risk
Limitation	Doesn't teach new tasks—only steers preferences

RLHF requires full rankings of the many candidate responses

Predibase

DeepLearning.AI

Challenges With RLHF and DPO

Challenge	RLHF	DPO
Data Needed	Ranked generations for reward model	Paired-preference labels (A > B)
Compute / Memory	Very high	Moderate
Training Stability	Often unstable—reward hacking, collapse risk	More stable, but still needs lots of labels
Limitation	Doesn't teach new tasks—only steers preferences	Doesn't teach new tasks—only steers preferences

DPO, in contrast, uses simple preference pairs, reducing computational load

Predibase

DeepLearning.AI

Challenges With RLHF and DPO

Challenge	RLHF	DPO
Data Needed	Ranked generations for reward model	Paired-preference labels (A > B)
Compute / Memory	Very high	Moderate
Training Stability	Often unstable—reward hacking, collapse risk	More stable, but still needs lots of labels
Limitation	Doesn't teach new tasks—only steers preferences	Doesn't teach new tasks—only steers preferences

They simply guide the model towards human preferred behaviors.

Predibase

DeepLearning.AI

Algorithms for Reinforcement Learning

1. Reinforcement Learning with Human Feedback (RLHF)
2. Direct Policy Optimization (DPO)
3. Group Relative Policy Optimization (GRPO)

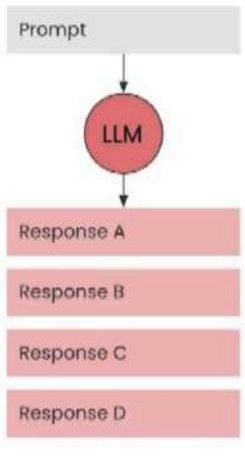
the DeepSeek team proposed a new alternative method called

Predibase

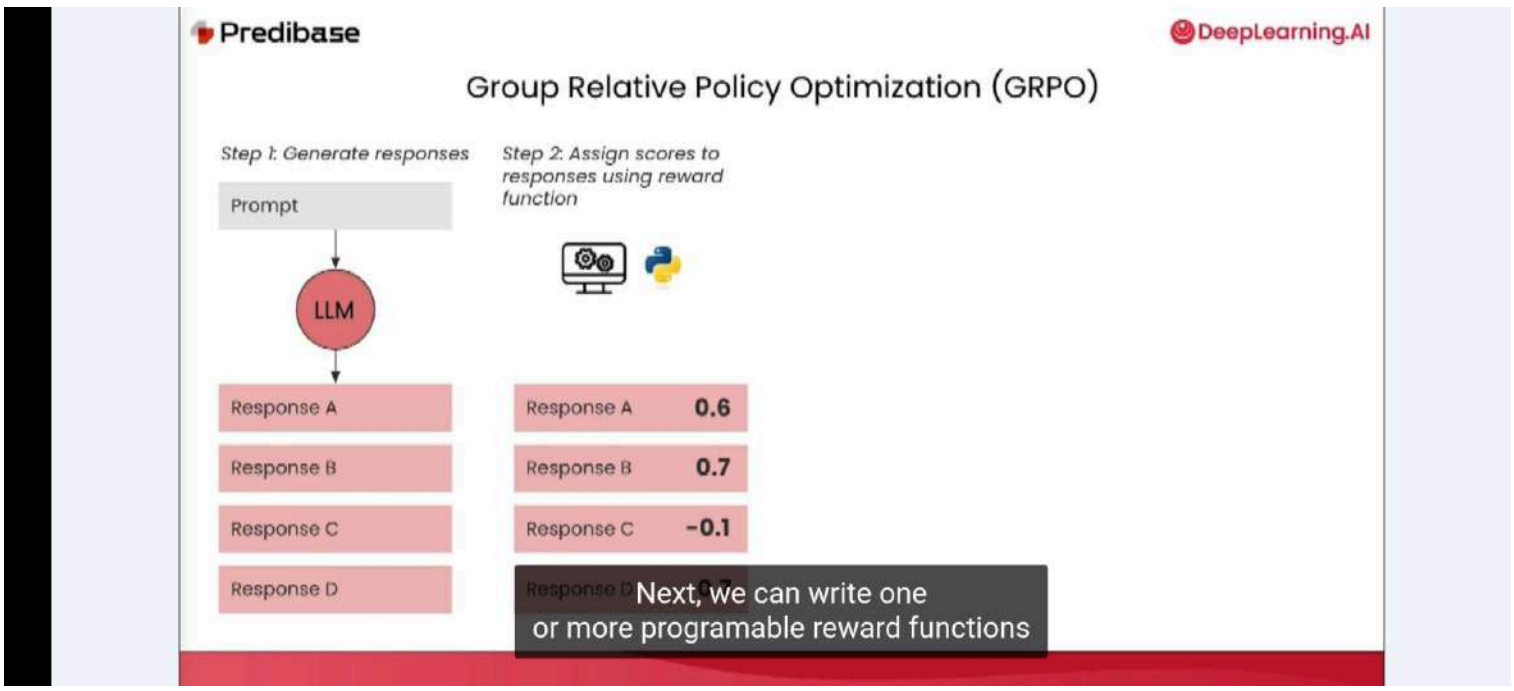
DeepLearning.AI

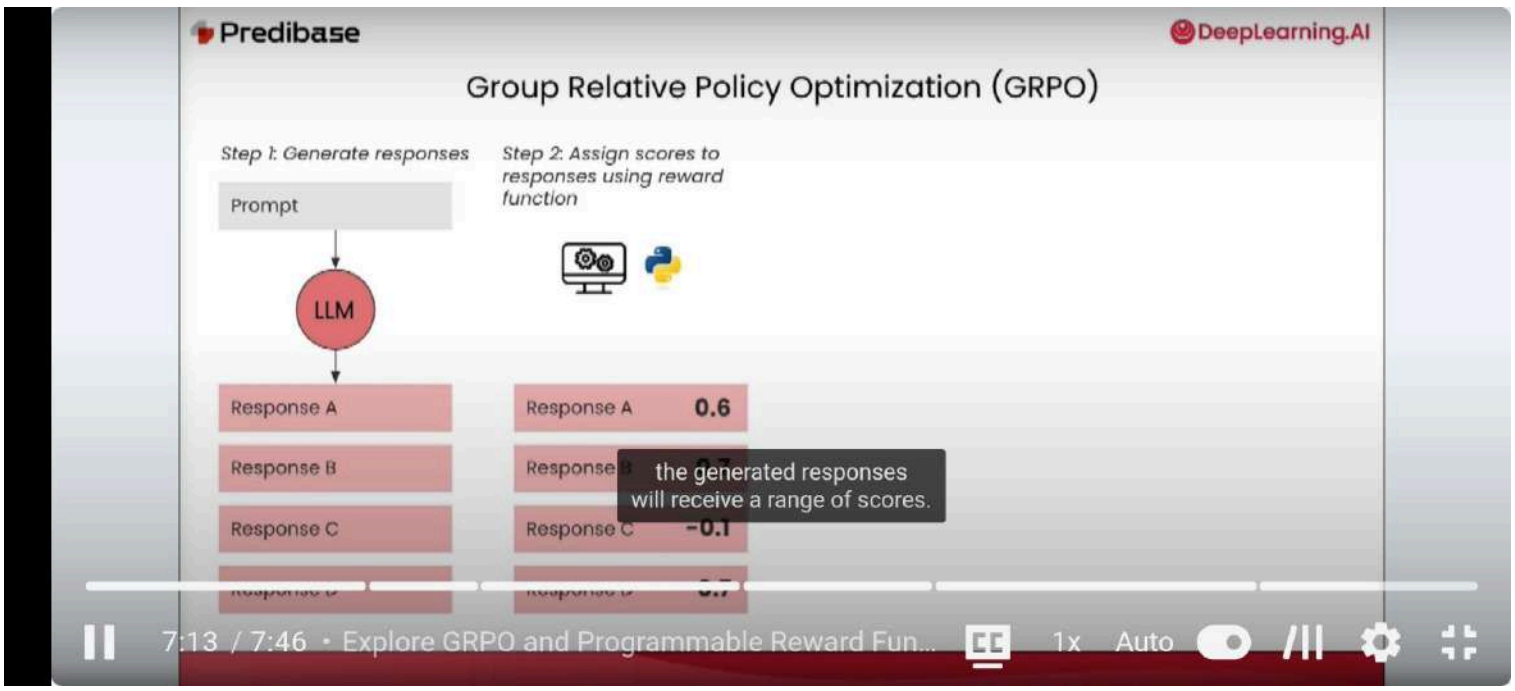
Group Relative Policy Optimization (GRPO)

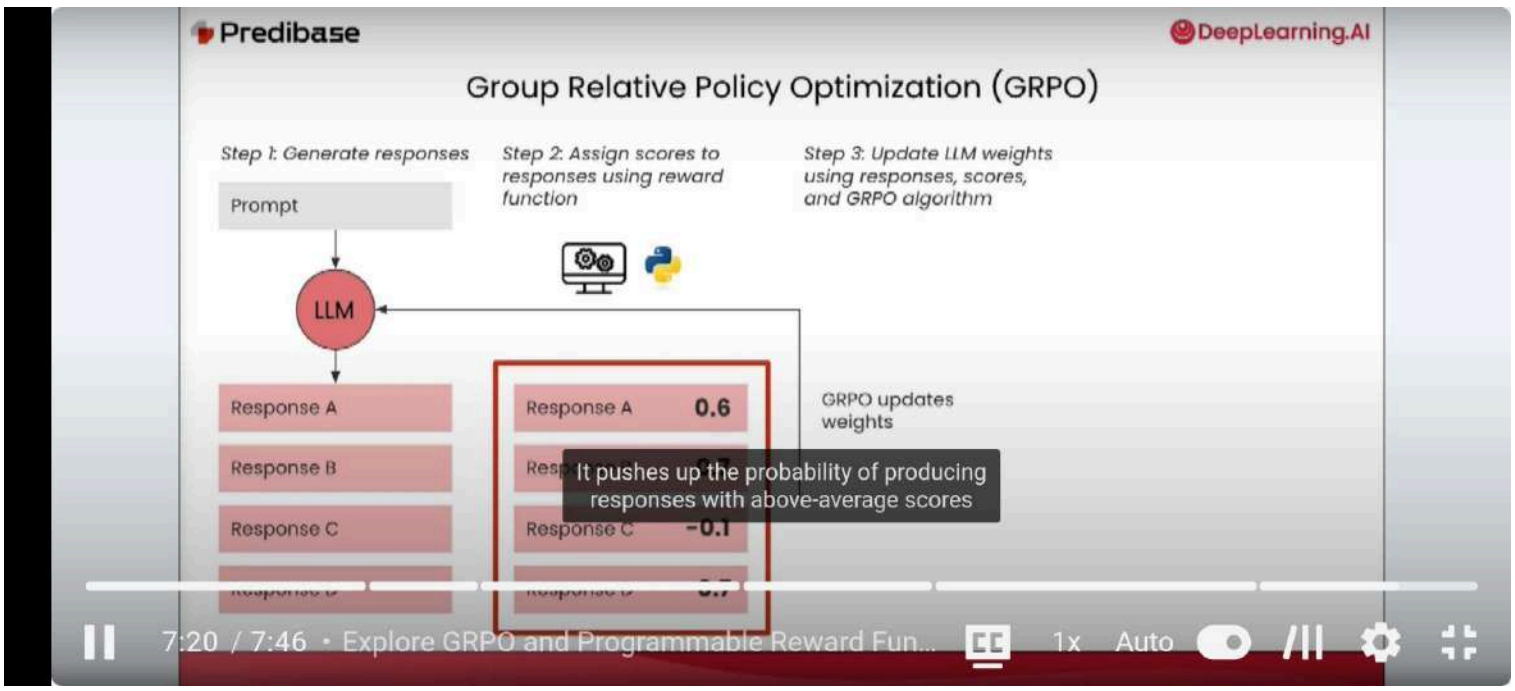
Step 1: Generate responses

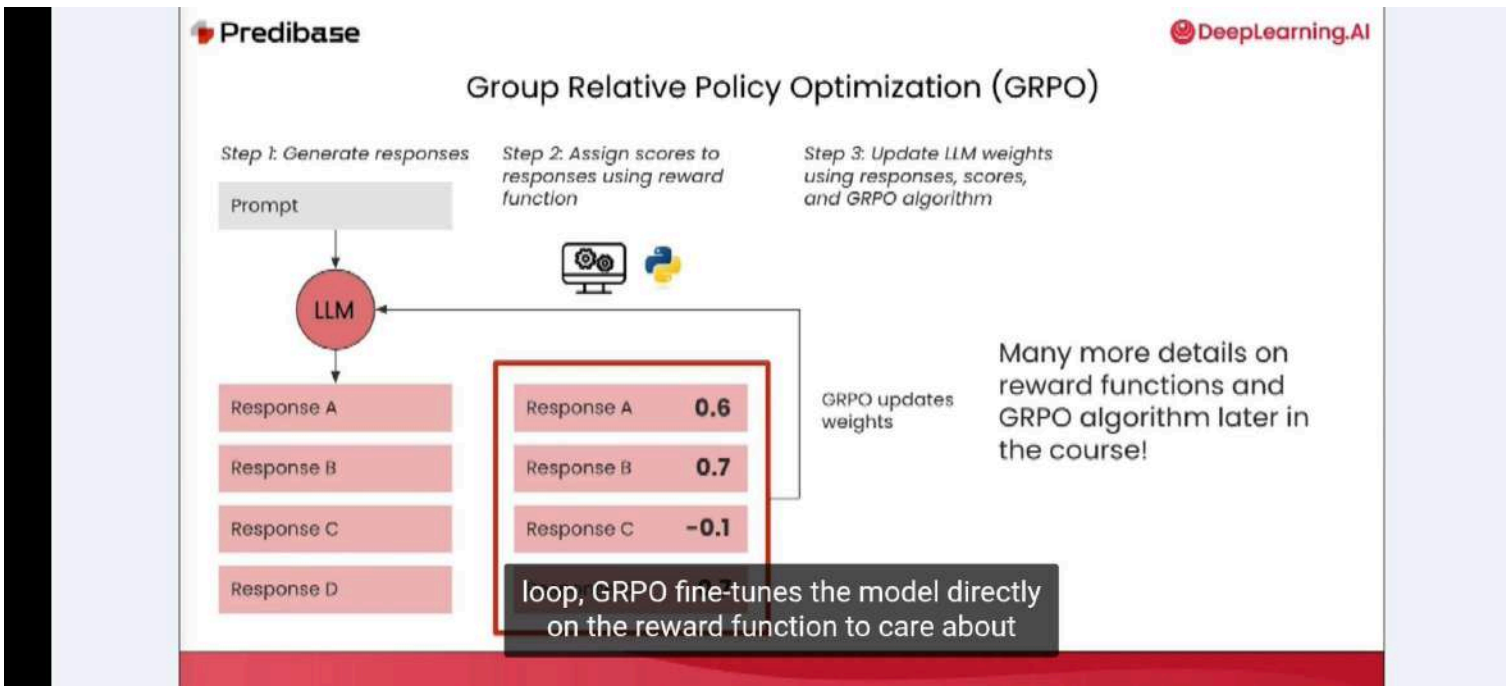


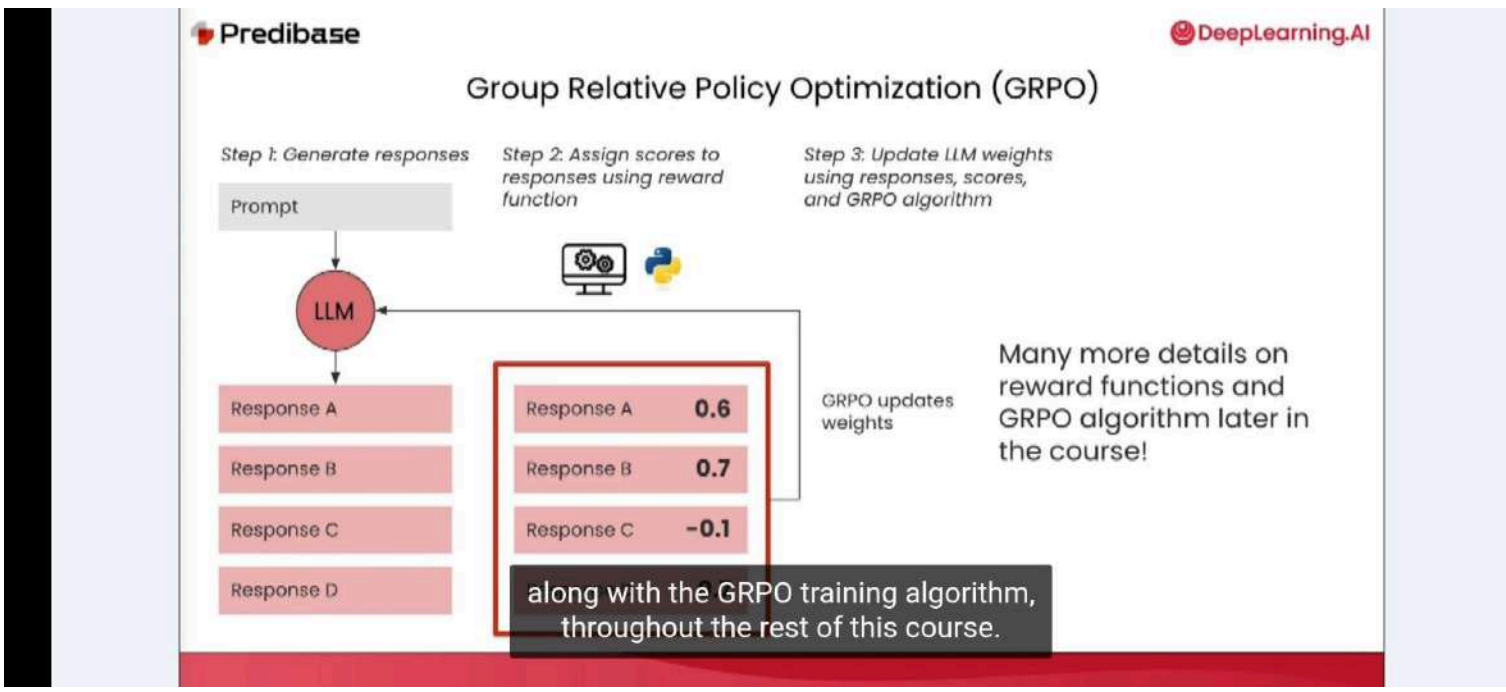
we first send a prompt to the LLM and sample multiple candidate responses.


Predibase
DeepLearning.AI
Group Relative Policy Optimization (GRPO)
Step 1: Generate responses
Step 2: Assign scores to responses using reward function
Prompt
LLM
Response A
Response B
Response C
Response D
Response A 0.6
Response B 0.7
Response C -0.1
Next, we can write one or more programable reward functions

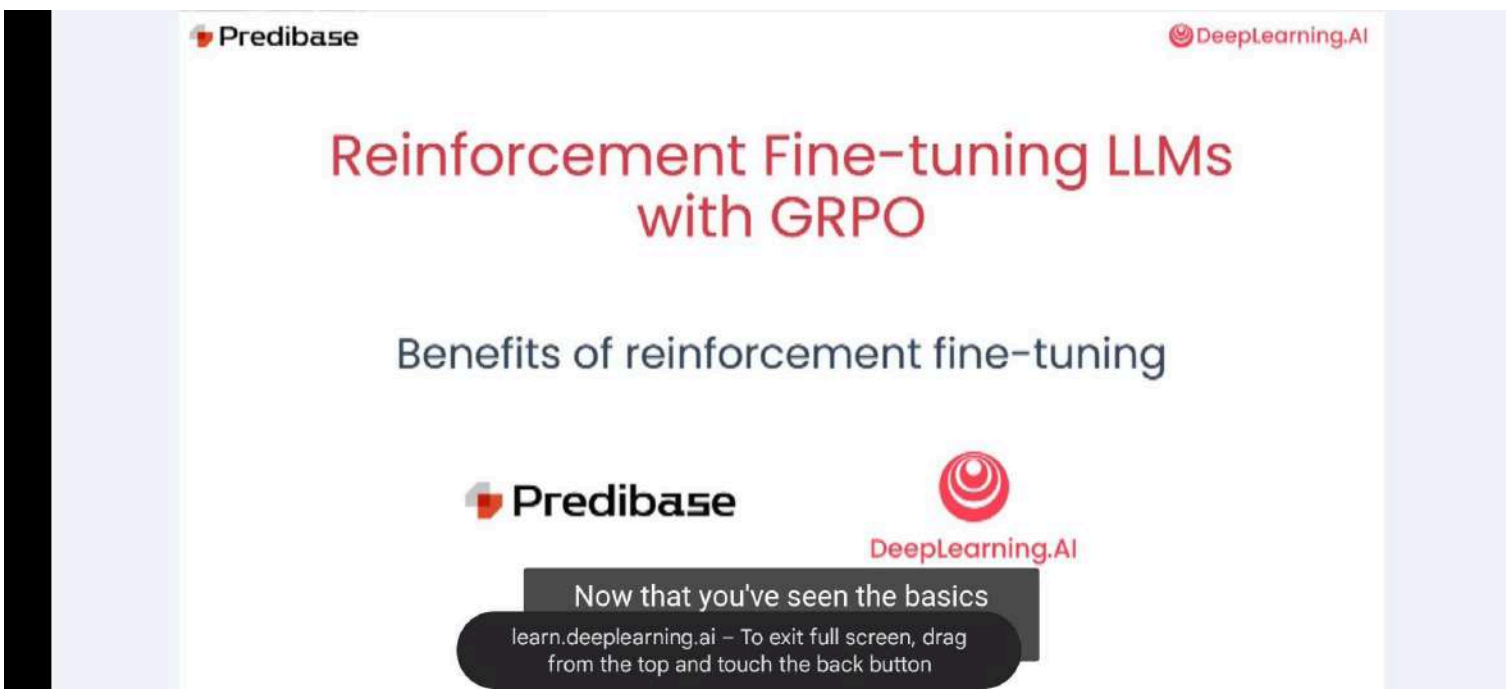

Predibase
DeepLearning.AI
Group Relative Policy Optimization (GRPO)
Step 1: Generate responses
Step 2: Assign scores to responses using reward function
Prompt
LLM
Response A
Response B
Response C
Response A 0.6
Response C -0.1
the generated responses will receive a range of scores.
7:13 / 7:46 • Explore GRPO and Programmable Reward Fun...
1x
Auto


Predibase
DeepLearning.AI
Group Relative Policy Optimization (GRPO)
Step 1: Generate responses
Step 2: Assign scores to responses using reward function
Step 3: Update LLM weights using responses, scores, and GRPO algorithm
Prompt
LLM
Response A
Response B
Response C
Response A 0.6
Response C -0.1
GRPO updates weights
It pushes up the probability of producing responses with above-average scores
7:20 / 7:46 • Explore GRPO and Programmable Reward Fun...
1x
Auto





Benefits of reinforcement fine-tuning



Doesn’t require labeled data, just a mean to ‘verify” correctness

Predibase

DeepLearning.AI

Benefits of Reinforcement Fine-Tuning

- ✓ Doesn’t require labeled data, just a means to “verify” correctness
- ✓ Works with as few as 10 examples

as you increase the number of prompts
that you show the model during training.

Predibase

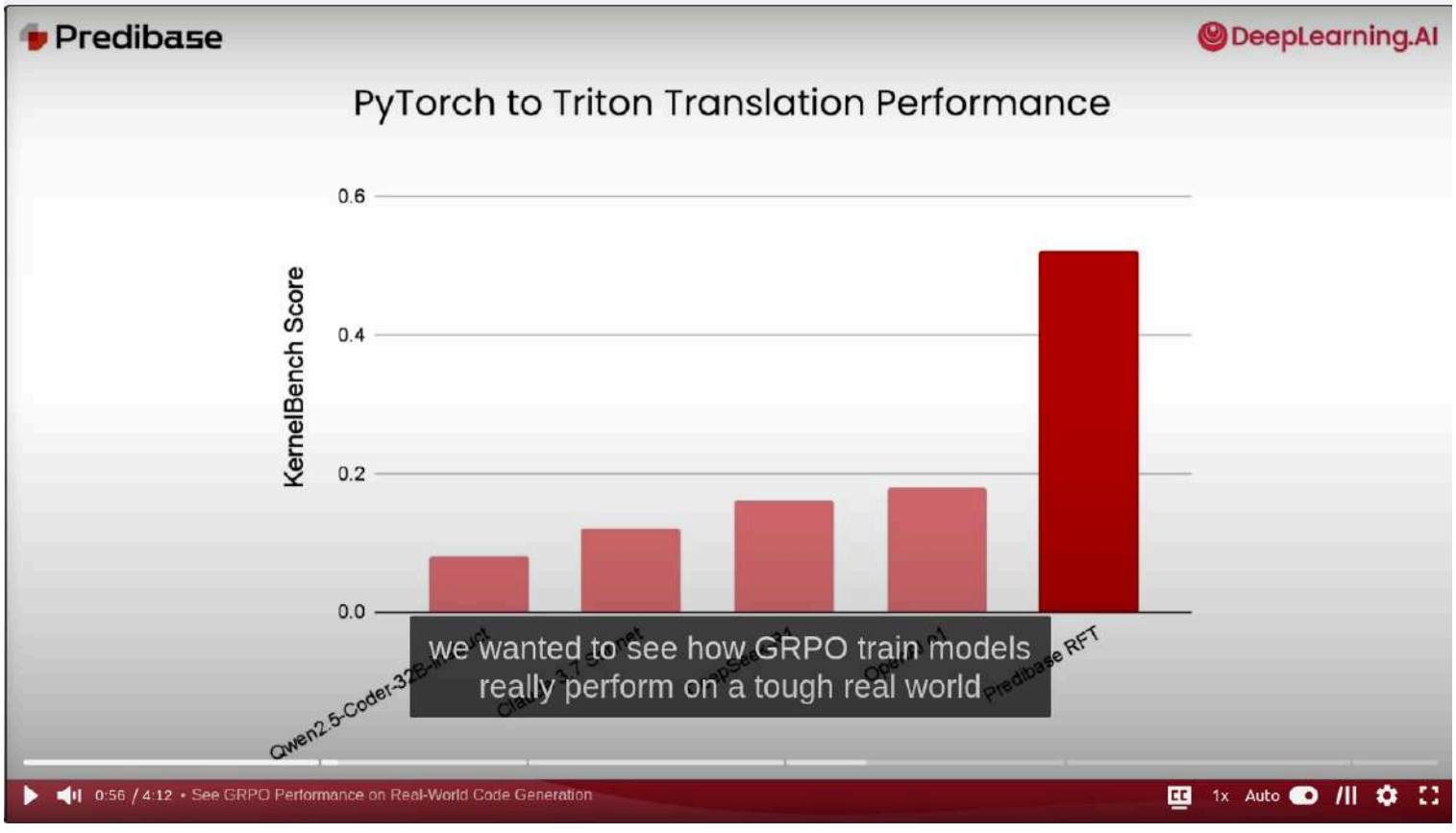
DeepLearning.AI

Benefits of Reinforcement Fine-Tuning

- ✓ Doesn’t require labeled data, just a means to “verify” correctness
- ✓ Works with as few as 10 examples
- ✓ More flexible than SFT because it learns *actively* from feedback rather than from *fixed* labeled examples
- ✓ Enables reasoning models to organically discover better strategies by improving its chain of thought

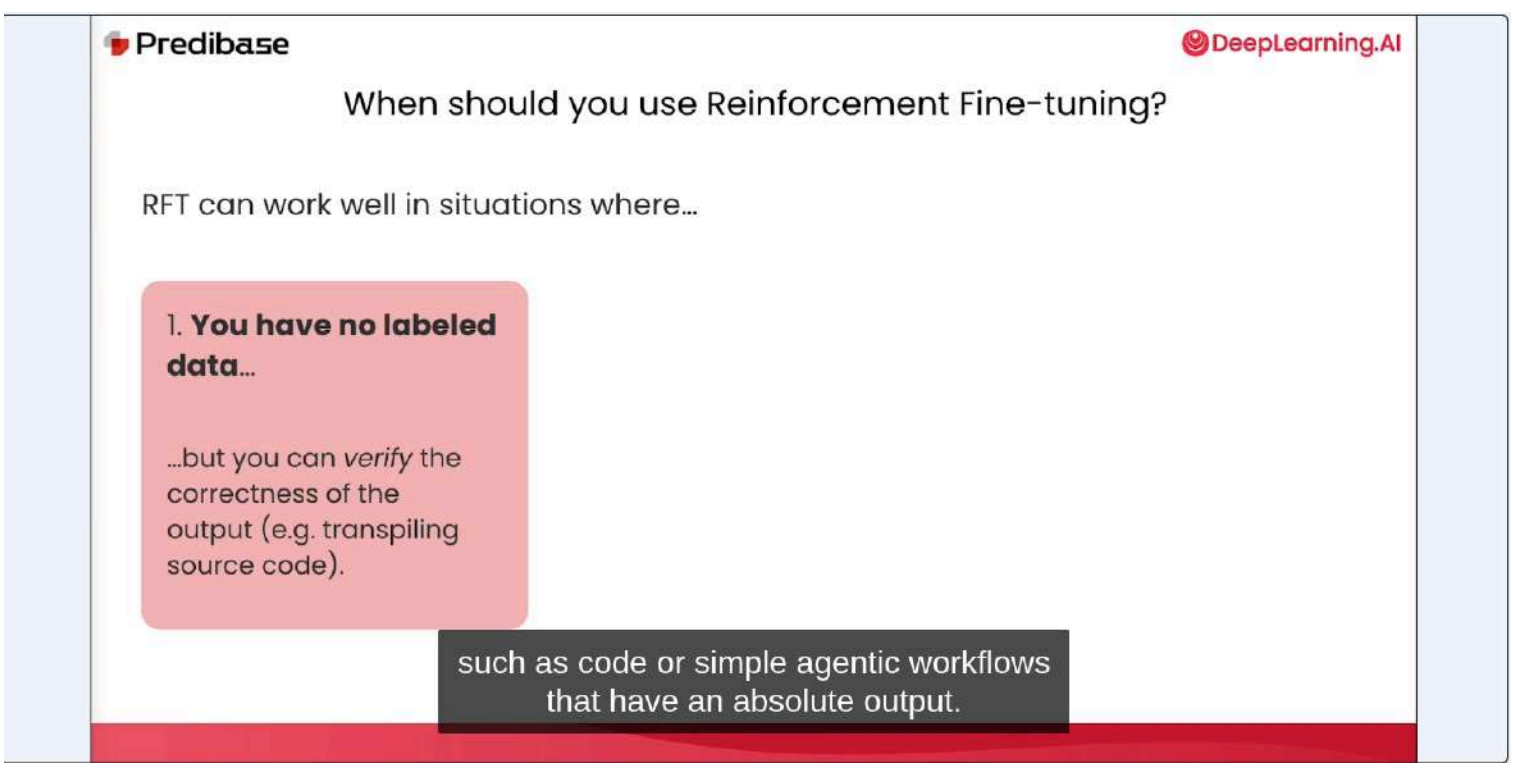
And because of this, it enables
reasoning models to organically discover

Pytorch to Triton Translation Performance

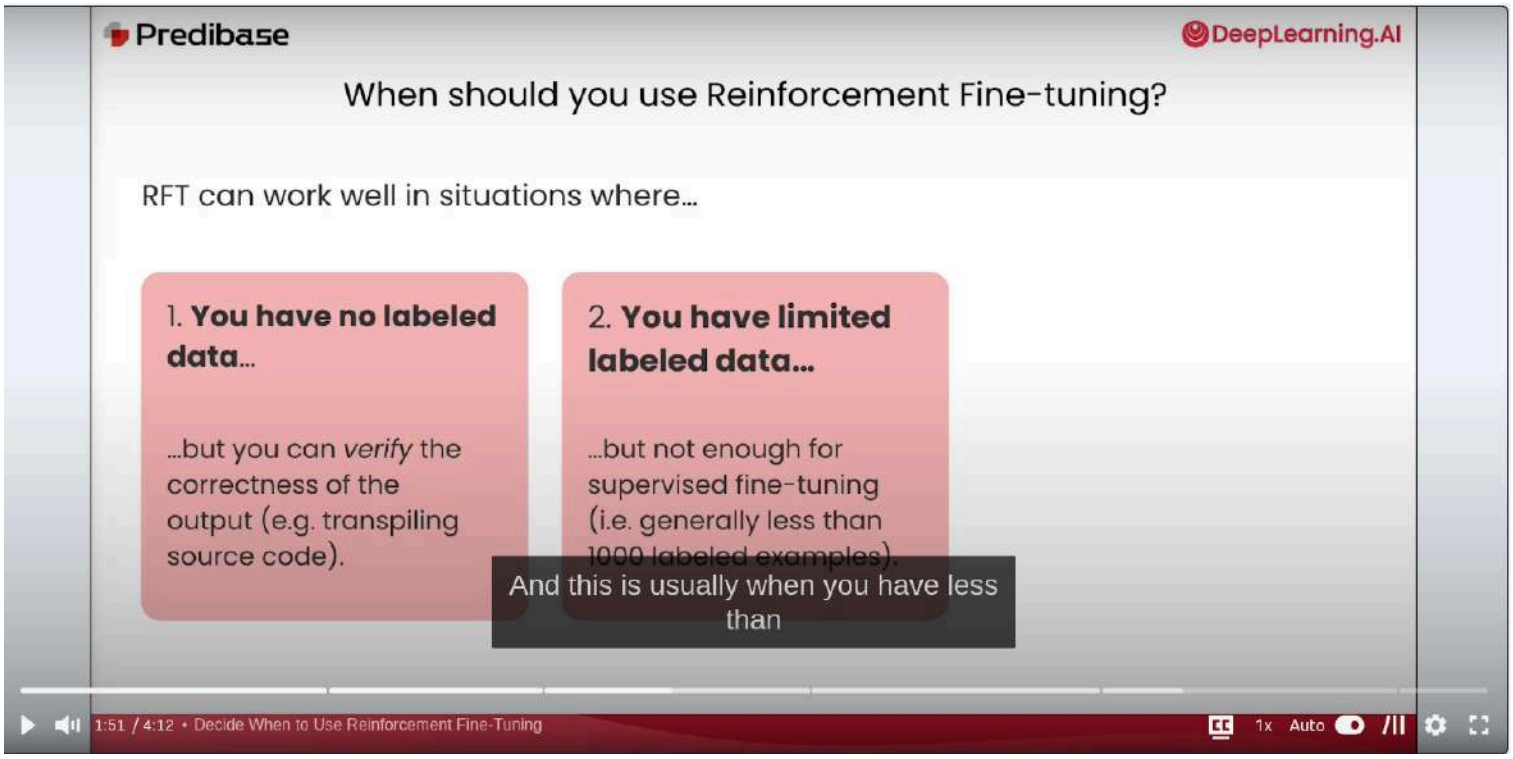


When should you use Reinforcement Fine-Tuning

You have no labeled data

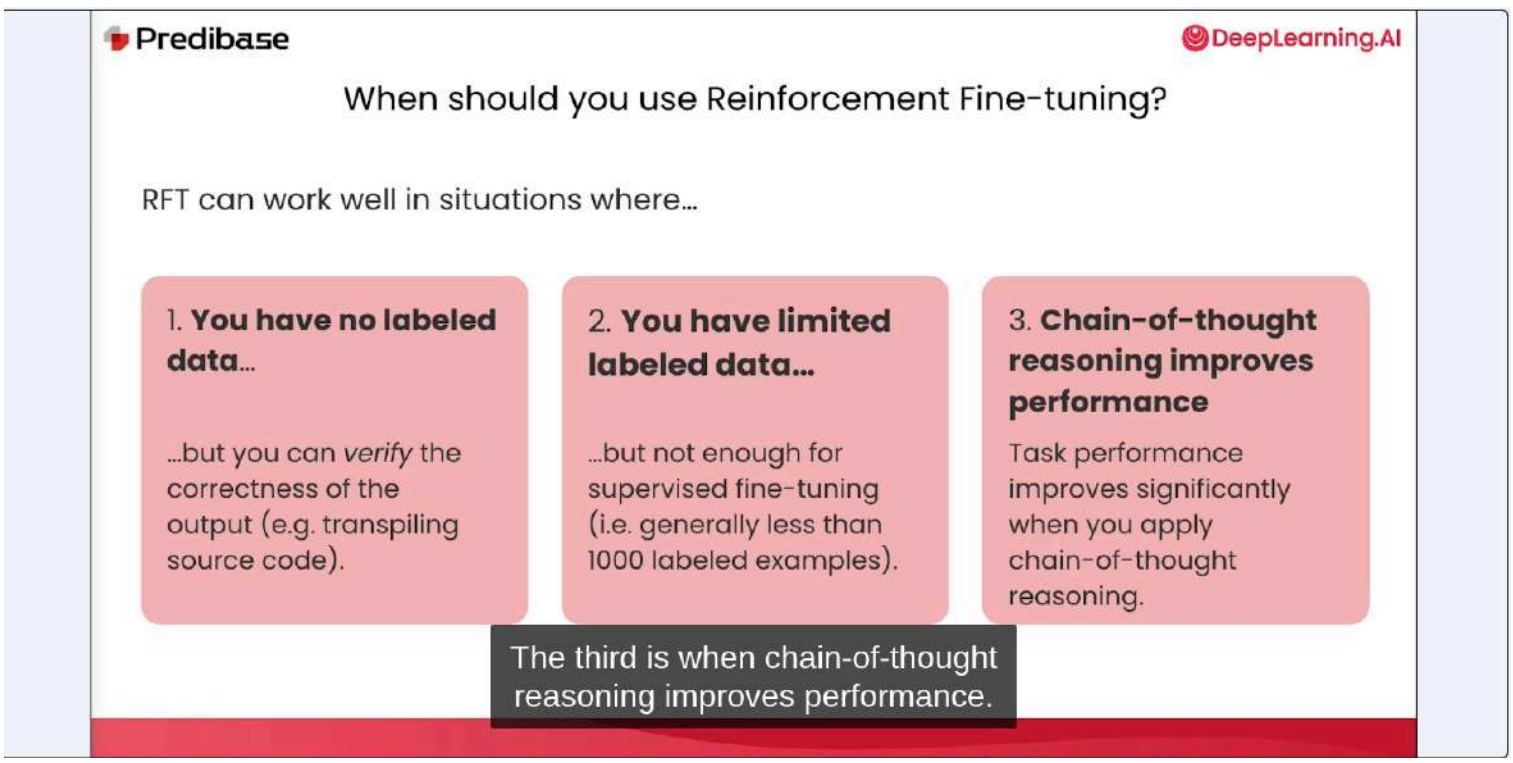


You have limited labeled data



\

Chain of thought (COT) reasoning improves performance

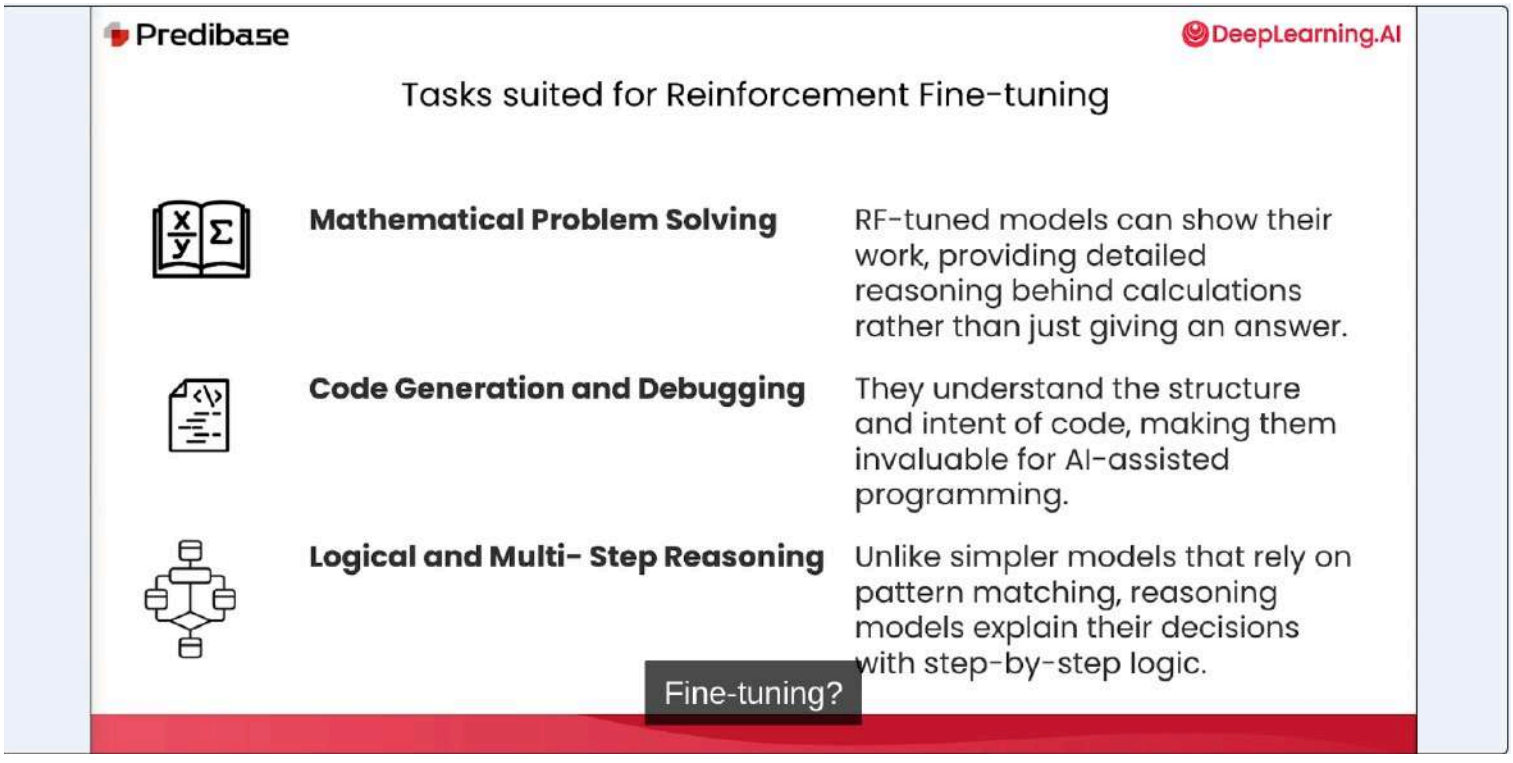


Task suited for Reinforcement Fine-tuning

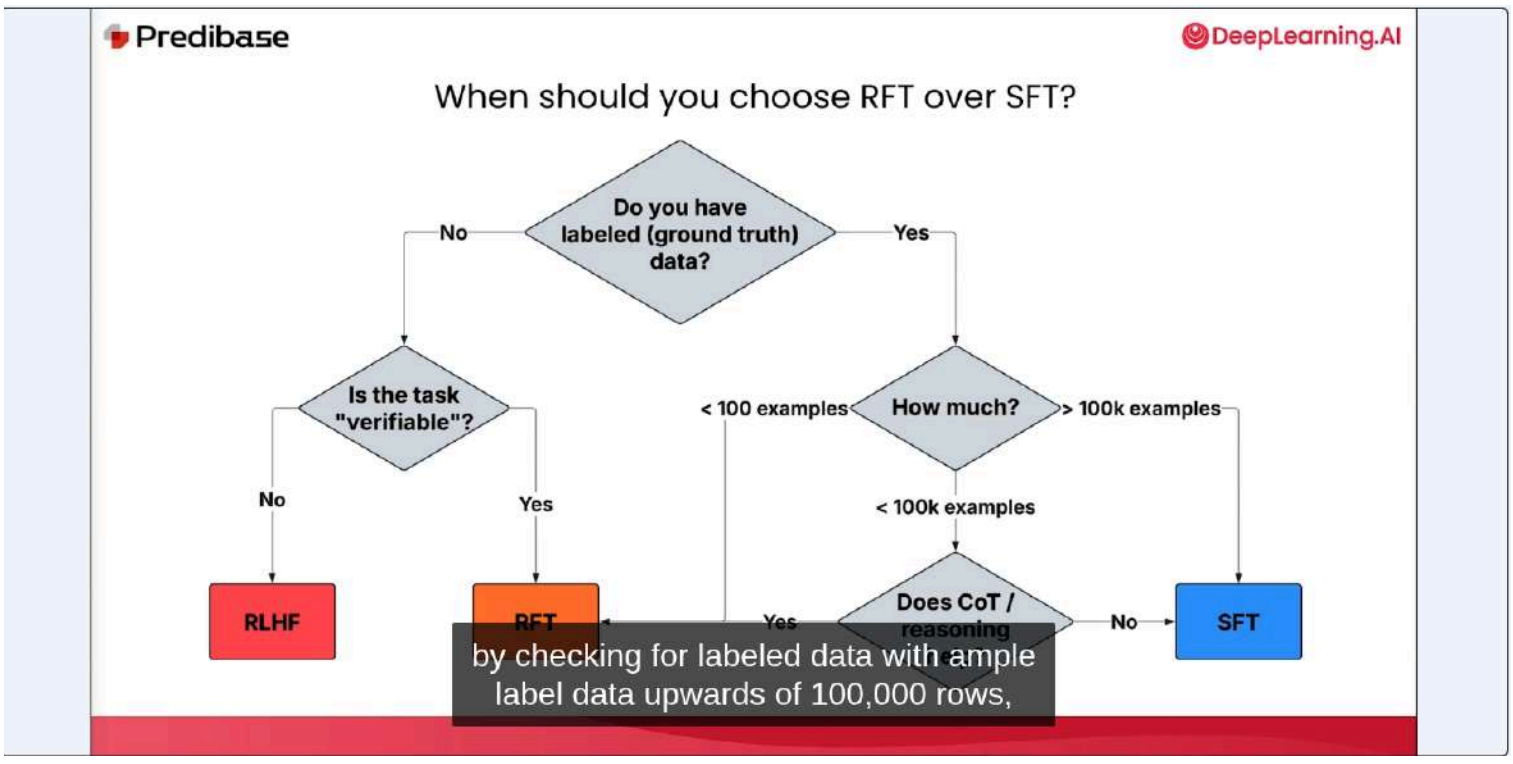
Mathematical Problem solving

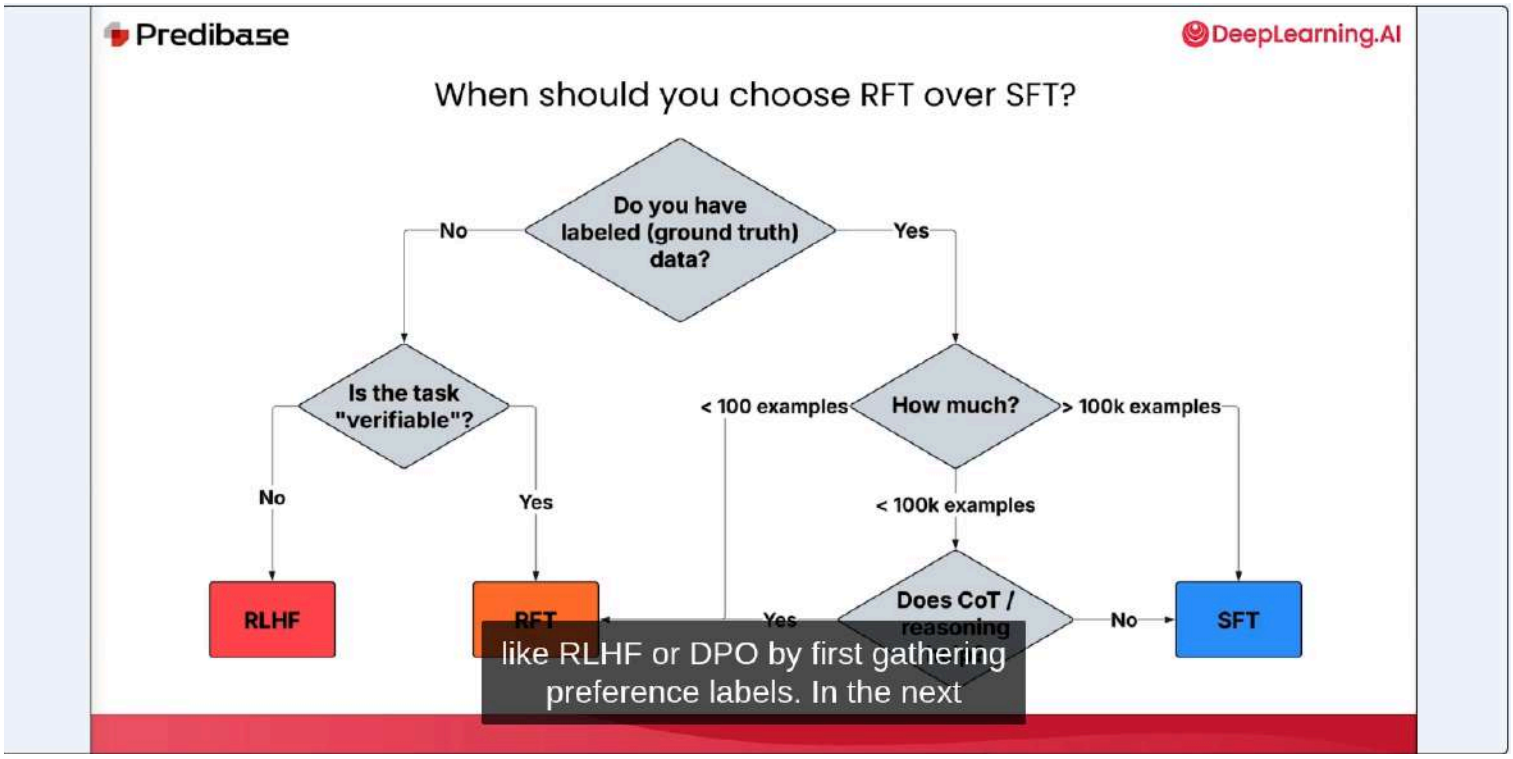
Code Generation and debugging

Logical and multi-step reasoning



When should you choose RFT ?





Can A Large language model master wordle?

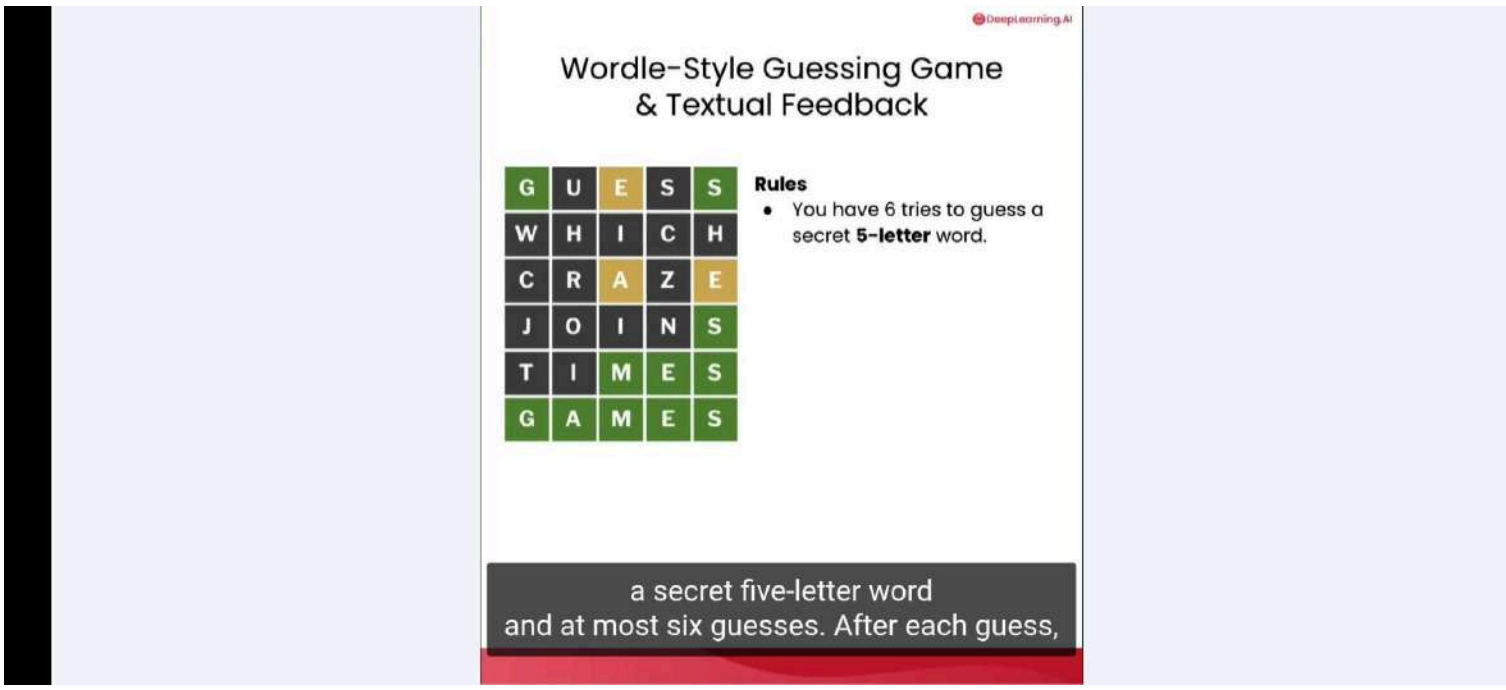
Reinforcement Fine-tuning LLMs

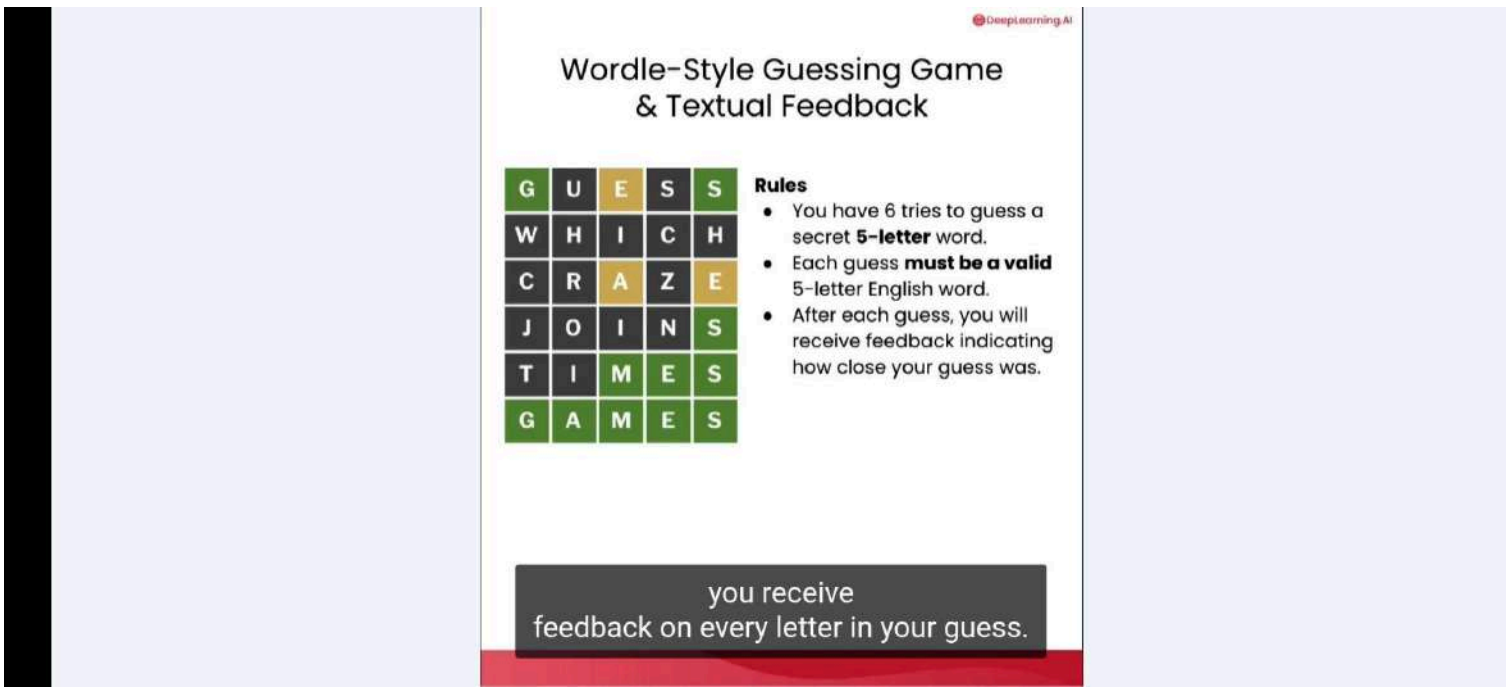
Can a large language model master Wordle?

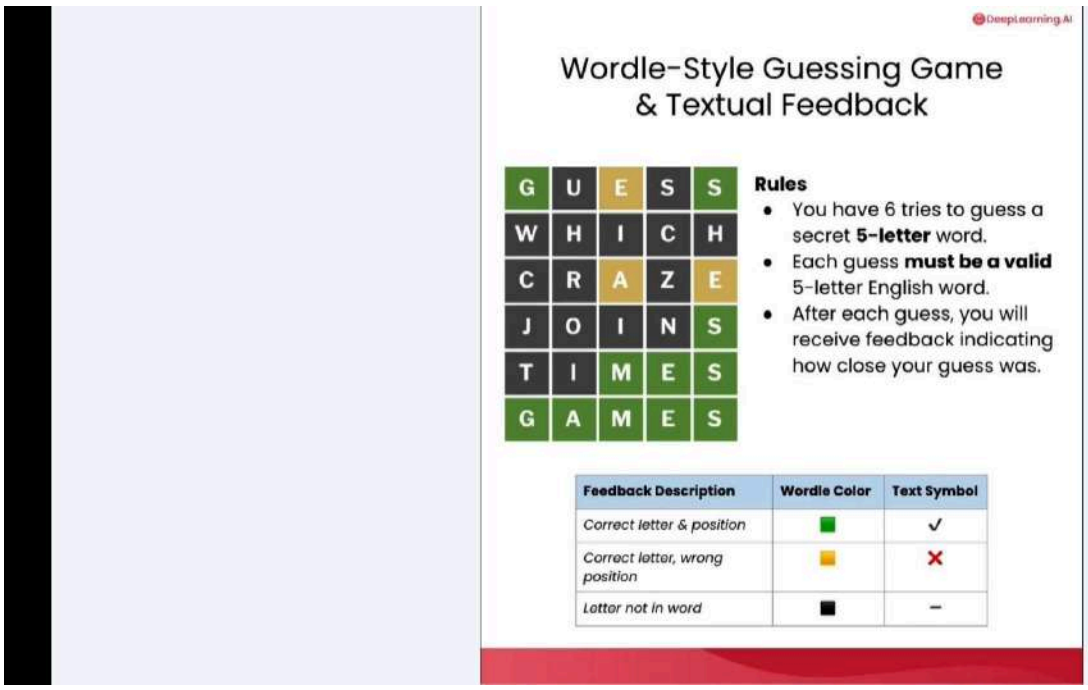
Predibase

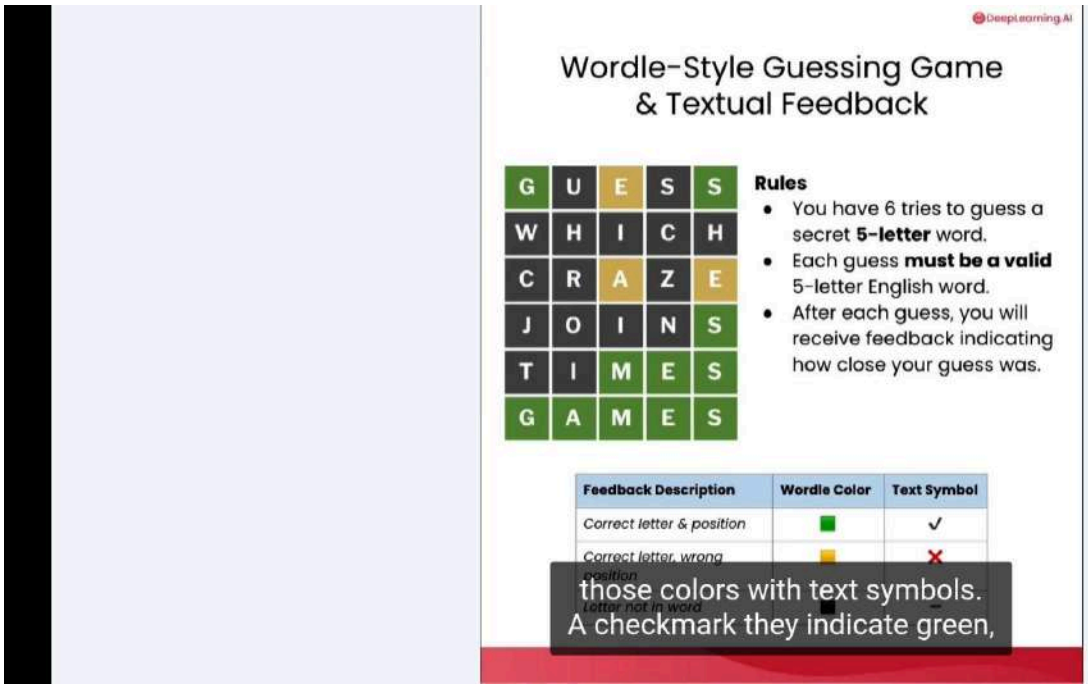
DeepLearning.AI

Wordle-style

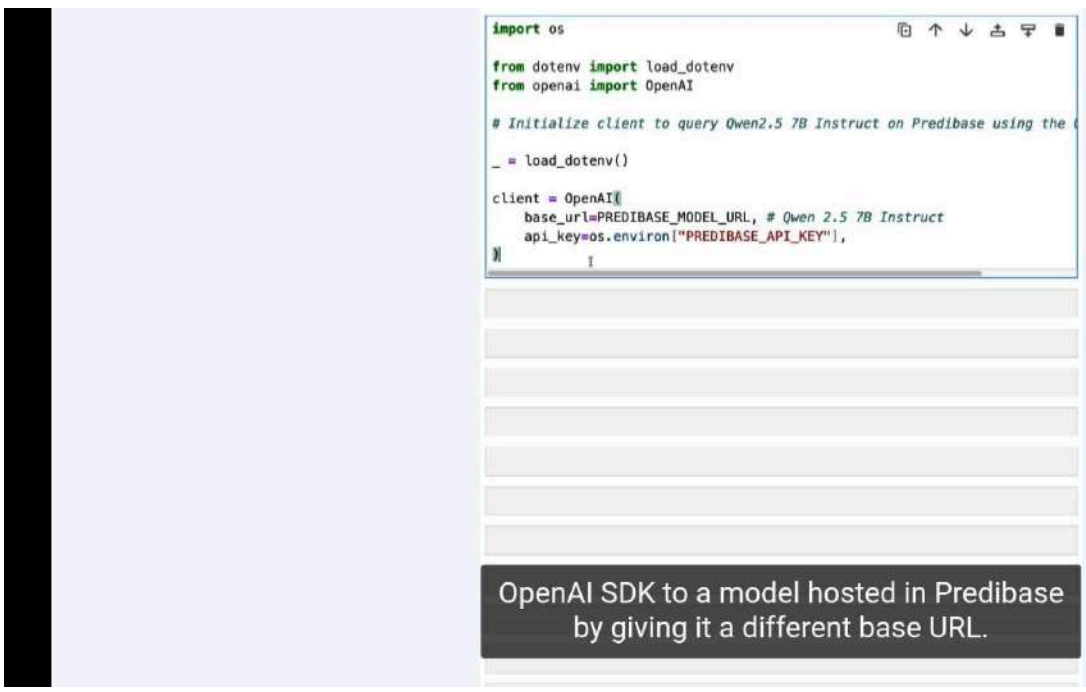








Load Qwen2.5 7B model



```
SYSTEM_PROMPT = """
You are playing Wordle, a word-guessing game.

### Game Rules:
- You have **6 tries** to guess a secret **5-letter** word.
- Each guess must be a valid **5-letter English word**.
- After each guess, you will receive feedback indicating how close
your guess was.

### Feedback Format:
Each letter in your guess will receive one of three symbols:
1. ✓ : The letter is in the word and in the CORRECT position.
2. - : The letter is in the word but in the WRONG position.
3. x : The letter is NOT in the word.

### Example:
Secret Word: BRISK

Guess 1: STORM → Feedback: S(-) T(x) O(x) R(-) M(x)
Guess 2: BRAVE → Feedback: B(✓) R(✓) A(x) V(x) E(x)
Guess 3: BRISK → Feedback: B(✓) R(✓) I(✓) S(✓) K(✓)

### Response Format:
Think through the problem and feedback step by step. Make sure to
first add your step by step thought process within <think> </think>
tags. Then, return your guessed word in the following format:
<guess> guessed-word </guess>.
"""
```

that we will pass to the model to play the game of Wordle.

Helper class

```python
from dataclasses import dataclass
from enum import Enum
from typing import List


class LetterFeedback(Enum):
    CORRECT = "✓"
    WRONG_POS = "–"
    WRONG_LETTER = "x"


@dataclass
class GuessWithFeedback:
    guess: str
    feedback: List[LetterFeedback]

    def __repr__(self) -> str:
        """Returns a readable string showing the guess alongside
        its letter-by-letter feedback."""
        feedback_str = " ".join(f"{letter}({fb.value})" for letter, fb i
        return f"{self.guess} → Feedback: {feedback_str}"
```

mpt Formatting

render_user_prompt

```
    WRONG_POS = "-"
    WRONG_LETTER = "x"


@dataclass
class GuessWithFeedback:
    guess: str
    feedback: List[LetterFeedback]

    def __repr__(self) -> str:
        """Returns a readable string showing the guess alongside
        its letter-by-letter feedback."""
        feedback_str = " ".join(f"{letter}({fb.value})" for letter, fb in
        return f"{self.guess} → Feedback: {feedback_str}"
```

```
def render_user_prompt(past_guesses: List[GuessWithFeedback]) -> str:
    """Creates a user-facing prompt that includes past guesses
    and their feedback."""
    prompt = "Make a new 5-letter word guess."
    if past_guesses:
        prompt += "\n\nHere is some previous feedback:"
        for i, guess in enumerate(past_guesses):
            prompt += f"\nGuess {i+1}: {guess}"
    return prompt
```

to create these feedback strings from the guess

Render_prmpt

```python
def render_prompt(past_guesses: List[GuessWithFeedback]):
    """Formats a full chat prompt using a system message, user
    prompt, and assistant preamble to start the model's
    step-by-step reasoning."""
    messages = [
        {
            "role": "system",
            "content": SYSTEM_PROMPT
        },
        {
            "role": "user",
            "content": render_user_prompt(past_guesses)
        },
        {
            "role": "assistant",
            "content": "Let me solve this step by step.\n<think>"
        }
    ]

    return tokenizer.apply_chat_template(
        messages, tokenize=False, continue_final_message=True
    )
```

So we'll define this messages object that has the system

Generate Stream

```python
def generate_stream(prompt: str, adapter_id: str = "") -> str:
    """Streams a model-generated response from a prompt in
    real-time and prints it as it arrives."""
    response = client.completions.create(
        model=adapter_id,
        prompt=prompt,
        # Produce deterministic responses for evaluation
        temperature=0.0,
        max_tokens=2048,
        stream=True,
    )

    completion = ""
    for chunk in response:
        if chunk.choices[0].text is not None:
            content = chunk.choices[0].text
            print(content, end="", flush=True)
            completion += content
    print()

    return completion
```

and an optional adapter ID.

```
secret_word = "CRAFT"

past_guesses = [
    GuessWithFeedback(
        "CRANE", [
            LetterFeedback.CORRECT,
            LetterFeedback.CORRECT,
            LetterFeedback.CORRECT,
            LetterFeedback.WRONG_LETTER,
            LetterFeedback.WRONG_LETTER,
        ]),
    GuessWithFeedback(
        "CRASH", [
            LetterFeedback.CORRECT,
            LetterFeedback.CORRECT,
            LetterFeedback.CORRECT,
            LetterFeedback.WRONG_LETTER,
            LetterFeedback.WRONG_LETTER,
        ]),
]

prompt = render_prompt(past_guesses)
print(prompt)
```


along with detail feedback
for each letter.

Generate Prompt

```
prompt = render_prompt(past_guesses)
print(prompt)
```

```
<|im_start|>system

You are playing Wordle, a word-guessing game.

### Game Rules:
- You have **6 tries** to guess a secret **5-letter** word.
- Each guess must be a valid **5-letter English word**.
- After each guess, you will receive feedback indicating how close
your guess was.

### Feedback Format:
Each letter in your guess will receive one of three symbols:
1. ✓ : The letter is in the word and in the CORRECT position.
2. - : The letter is in the word but in the WRONG position.
3. x : The letter is NOT in the word.

### Example:
Secret Word: BRISK

Guess 1: STORM → Feedback: S(-) T(x) O(x) R(-) M(x)
Guess 2: BRAVE → Feedback: B(✓) R(✓) A(x) V(x) E(x)
Guess 3: BRISK → Feedback: B(✓) R(✓) I(✓) S(✓) K(✓)

### Response Format:
Think through the problem and feedback step by step. Make sure to
first add your step by step thought process within <think> </think>
tags. Then, return your guessed word in the following format:
<guess> guessed-word </guess>.
<|im_end|>
<|im_start|>user
Make a new 5-letter word guess.

Here is some previous feedback:
Guess 1: CRANE → Feedback: C(✓) R(✓) A(✓) N(x) E(x)
Guess 2: CRASH → Feedback: C(✓) R(✓) A(✓) S(x) H(x)<|im_end|>
<|im_start|>assistant
```

that our prompt has all of the same stuff as our system prompt,

Sent prompt to LLM (1st chat completion)

```
# prompt base model
base_completion = generate_stream(prompt)
```

```
From the feedback, we know:
- C is in the correct position (1st letter)
- R is in the correct position (2nd letter)
- A is in the correct position (3rd letter)
- N is not in the word
- E is not in the word
- S is not in the word
- H is not in the word

This leaves us with the letters B, D, G, I, J, K, L, M, O, P, Q, T, U,
V, W, X, Y, Z to choose from for the 4th and 5th positions. We should tr
y to include letters that are common in 5-letter words and haven't been
ruled out yet.</think>
<guess>CRANE</guess>
```

our incorrect positions.

```python
# prompt fine-tuned model
ft_completion = generate_stream(prompt, adapter_id="wordle-dlai/2")
```

```
Let's analyze the feedback we have so far:

Guess 1: CRANE → Feedback: C(✓) R(✓) A(✓) N(x) E(x)
This means:
- C is in the correct position (1st position)
- R is in the correct position (2nd position)
- A is in the correct position (3rd position)
- N is not in the word
- E is not in the word

Guess 2: CRASH → Feedback: C(✓) R(✓) A(✓) S(x) H(x)
This confirms:
- C, R, A are in their correct positions (1st, 2nd, 3rd)
- S is not in the word
- H is not in the word

So we know:
- The first three letters are "CRA"
- The letters N, E, S, H are not in the word

Now I need to make a guess for the 4th and 5th positions. Let me think o
f 5-letter words that start with "CRA" and don't contain N,
```

that C, R and A are in the correct position and N and E are not in the word.

```
# prompt fine-tuned model
ft_completion = generate_stream(prompt, adapter_id="wordle-dlai/2")
```

```
Let's analyze the feedback we have so far:

Guess 1: CRANE → Feedback: C(✓) R(✓) A(✓) N(x) E(x)
This means:
- C is in the correct position (1st position)
- R is in the correct position (2nd position)
- A is in the correct position (3rd position)
- N is not in the word
- E is not in the word

Guess 2: CRASH → Feedback: C(✓) R(✓) A(✓) S(x) H(x)
This confirms:
- C, R, A are in their correct positions (1st, 2nd, 3rd)
- S is not in the word
- H is not in the word

So we know:
- The first three letters are "CRA"
- The letters N, E, S, H are not in the word

Now I need to make a guess for the 4th and 5th positions. Let me think o
f 5-letter words that start with "CRA" and don't contain N, E, S, H.

Possible words:
- CRAFT
- CRAMP
- CRANK (no, contains N)
- CRAP
- CRAZE (no, contains E)
- CRAZY (no, contains Z)
- CRAB
- CRACK
- CRAM
- CRAYON (no, contains N)

Let's eliminate those with letters we know are not in the word.

From the remaining options, CRAFT, CRAMP, CRAP, CRAB, CRACK, and CRAM se
em like good candidates.

CRAFT would check the letters F and T
CRAMP would check the letters M and P
CRAB would check the letters B
CRACK would check the letters C and K
CRAM would check the letter M

Since CRAM is only 4 letters, it's not a valid guess. CRAP is also only
4 letters.

Between CRAFT, CRAMP, CRAB, and CRACK, I think CRAFT is a good next gues
s because:
1. It tests two new letters (F and T)
2. It's a common English word
3. It might be the answer

So I'll go with CRAFT as my next guess.
Based on the feedback so far:

- The first three letters are definitely "CRA" (all marked with ✓)
- The letters N, E, S, and H are not in the word (all marked with x)

I need to find a 5-letter word that:
1. Starts with "CRA"
2. Doesn't contain N, E, S, or H
3. Is a valid English word

Looking at possible options, I could try words like:
- CRAFT (tests F and T)
- CRAMP (tests M and P in positions 4-5)
- CRACK (tests C and K)
- CRAB (tests B in position 4)

I'll try CRAFT as it's a common word and tests two new letters in the re
```

After producing this large chain of thought, it decides that craft

From the remaining options, CRAFT, CRAMP, CRAP, CRAB, CRACK, and CRAM se
em like good candidates.

CRAFT would check the letters F and T
CRAMP would check the letters M and P
CRAB would check the letters B
CRACK would check the letters C and K
CRAM would check the letter M

Since CRAM is only 4 letters, it's not a valid guess. CRAP is also only
4 letters.

Between CRAFT, CRAMP, CRAB, and CRACK, I think CRAFT is a good next gues
s because:
1. It tests two new letters (F and T)
2. It's a common English word
3. It might be the answer

So I'll go with CRAFT as my next guess.
Based on the feedback so far:

- The first three letters are definitely "CRA" (all marked with ✓)
- The letters N, E, S, and H are not in the word (all marked with x)

I need to find a 5-letter word that:
1. Starts with "CRA"
2. Doesn't contain N, E, S, or H
3. Is a valid English word

Looking at possible options, I could try words like:
- CRAFT (tests F and T in positions 4-5)
- CRAMP (tests M and P in positions 4-5)
- CRACK (tests C and K in positions 4-5)
- CRAB (tests B in position 4)

I'll try CRAFT as it's a common word and tests two new letters in the re
maining positions.</think><guess>CRAFT</guess>

is an optimal guess to make
based on all the criteria that it's left.

From the remaining options, CRAFT, CRAMP, CRAP, CRAB, CRACK, and CRAM se
em like good candidates.

CRAFT would check the letters F and T
CRAMP would check the letters M and P
CRAB would check the letters B
CRACK would check the letters C and K
CRAM would check the letter M

Since CRAM is only 4 letters, it's not a valid guess. CRAP is also only
4 letters.

Between CRAFT, CRAMP, CRAB, and CRACK, I think CRAFT is a good next gues
s because:
1. It tests two new letters (F and T)
2. It's a common English word
3. It might be the answer

So I'll go with CRAFT as my next guess.
Based on the feedback so far:

- The first three letters are definitely "CRA" (all marked with ✓)
- The letters N, E, S, and H are not in the word (all marked with x)

I need to find a 5-letter word that:
1. Starts with "CRA"
2. Doesn't contain N, E, S, or H
3. Is a valid English word

Looking at possible options, I could try words like:
- CRAFT (tests F and T in positions 4-5)
- CRAMP (tests M and P in positions 4-5)
- CRACK (tests C and K in positions 4-5)
- CRAB (tests B in position 4)

I'll try CRAFT as it's a common word and tests two new letters in the re
maining positions.</think><guess>CRAFT</guess>

use the past feedback to correctly
guess our secret word in three guesses.

Get Feedback function

```python
import re

def get_feedback(guess: str, secret_word: str) -> List[LetterFeedback]:
    valid_letters = set(secret_word)
    feedback = []
    for letter, secret_letter in zip(guess, secret_word):
        if letter == secret_letter:
            feedback.append(LetterFeedback.CORRECT)
        elif letter in valid_letters:
            feedback.append(LetterFeedback.WRONG_POS)
        else:
            feedback.append(LetterFeedback.WRONG_LETTER)
    return feedback
```

So. if the letter matches and the exact position we give it a correct symbol.

```python
import re

def get_feedback(guess: str, secret_word: str) -> List[LetterFeedback]:
    valid_letters = set(secret_word)
    feedback = []
    for letter, secret_letter in zip(guess, secret_word):
        if letter == secret_letter:
            feedback.append(LetterFeedback.CORRECT)
        elif letter in valid_letters:
            feedback.append(LetterFeedback.WRONG_POS)
        else:
            feedback.append(LetterFeedback.WRONG_LETTER)
    return feedback
```

The get feedback method will take a guess and secret word as input, and assign


```python
import re

def get_feedback(guess: str, secret_word: str) -> List[LetterFeedback]:
    valid_letters = set(secret_word)
    feedback = []
    for letter, secret_letter in zip(guess, secret_word):
        if letter == secret_letter:
            feedback.append(LetterFeedback.CORRECT)
        elif letter in valid_letters:
            feedback.append(LetterFeedback.WRONG_POS)
        else:
            feedback.append(LetterFeedback.WRONG_LETTER)
    return feedback
```


and if it's just not in the word at all, will mark it as a wrong letter.

Next turn function

```python
def next_turn(
    past_guesses: List[GuessWithFeedback],
    secret_word: str,
    adapter_id = ""
):
    prompt = render_prompt(past_guesses)
    completion = generate_stream(prompt, adapter_id)
    match = re.search(
        r"<guess>\s*(.*?)\s*</guess>", completion, re.DOTALL
    )
    if not match:
        raise RuntimeError("invalid guess")

    guess = match.group(1).upper()
    feedback = get_feedback(guess, secret_word)
    past_guesses.append(GuessWithFeedback(guess, feedback))
    print("\n\n")
    print(("-" * 100) + "\n")
    for past_guess in past_guesses:
        print(past_guess)

    if guess == secret_word:
        print("🎉 SUCCESS 🎉")
    elif len(past_guesses) >= 6:
        print("❌ better luck next time... ❌")
```

Once we have the response, we'll use regex matching

```python
def next_turn(
    past_guesses: List[GuessWithFeedback],
    secret_word: str,
    adapter_id = ""
):
    prompt = render_prompt(past_guesses)
    completion = generate_stream(prompt, adapter_id)
    match = re.search(
        r"<guess>\s*(.*?)\s*</guess>", completion, re.DOTALL
    )
    if not match:
        raise RuntimeError("invalid guess")

    guess = match.group(1).upper()
    feedback = get_feedback(guess, secret_word)
    past_guesses.append(GuessWithFeedback(guess, feedback))
    print("\n\n")
    print(("-" * 100) + "\n")
    for past_guess in past_guesses:
        print(past_guess)

    if guess == secret_word:
        print("🎉 SUCCESS 🎉")
    elif len(past_guesses) >= 6:
        print("❌ better luck next time... ❌")
```

which we'll call next turn.

```python
def next_turn(
    past_guesses: List[GuessWithFeedback],
    secret_word: str,
    adapter_id = ""
):
    prompt = render_prompt(past_guesses)
    completion = generate_stream(prompt, adapter_id)
    match = re.search(
        r"<guess>\s*(.*?)\s*</guess>", completion, re.DOTALL
    )
    if not match:
        raise RuntimeError("invalid guess")

    guess = match.group(1).upper()
    feedback = get_feedback(guess, secret_word)
    past_guesses.append(GuessWithFeedback(guess, feedback))
    print("\n\n")
    print(("-" * 100) + "\n")
    for past_guess in past_guesses:
        print(past_guess)

    if guess == secret_word:
        print("🎉 SUCCESS 🎉")
    elif len(past_guesses) >= 6:
        print("❌ better luck next time... ❌")
```

Next, it sends this to the model to generate an output.

```
    past_guesses: List[GuessWithFeedback],
    secret_word: str,
    adapter_id = ""
):
    prompt = render_prompt(past_guesses)
    completion = generate_stream(prompt, adapter_id)
    match = re.search(
        r"<guess>\s*(.*?)\s*</guess>", completion, re.DOTALL
    )
    if not match:
        raise RuntimeError("invalid guess")

    guess = match.group(1).upper()
    feedback = get_feedback(guess, secret_word)
    past_guesses.append(GuessWithFeedback(guess, feedback))
    print("\n\n")
    print(("-" * 100) + "\n")
    for past_guess in past_guesses:
        print(past_guess)

    if guess == secret_word:
        print("🎉 SUCCESS 🎉")
    elif len(past_guesses) >= 6:
        print("❌ better luck next time... ❌")
```

and if the guess matches,
a secret word will mark it as a success

Start game: “BRICK”

```
secret_word = "BRICK"
past_guesses = []
adapter_id = ""
```

```
next_turn(past_guesses, secret_word, adapter_id)
```

```
- The word is 5 letters long.
- We have 6 tries to guess the word.
- We need to use valid 5-letter English words.
- We should try to use words that cover a wide range of letters to maxim
ize our chances of getting feedback.
- A good starting word could be a common word that includes a mix of vow
els and consonants, such as "CRANE".
</think>
<guess> CRANE </guess>



--------------------------------------------------------------------------
--------------------------

CRANE → Feedback: C(-) R(✓) A(x) N(x) E(x)
```

So for the first guess,
the model decides that it's a good idea

Second guess

```
secret_word = "BRICK"
past_guesses = []
adapter_id = ""
```

```
next_turn(past_guesses, secret_word, adapter_id)
```

```
From the feedback, we know:
- R is in the correct position (2nd letter).
- C, A, N, and E are not in the word at all.
- The word is 5 letters long and must include R.

Given these constraints, we can eliminate any words that contain C, A,
N, or E. Let's try a word that includes R and avoids these letters. A go
od candidate could be "BRICK". This word includes R and doesn't contain
any of the letters C, A, N, or E.</think>
<guess>BRICK</guess>



--------------------------------------------------------------------------
--------------------------

CRANE → Feedback: C(-) R(✓) A(x) N(x) E(x)
BRICK → Feedback: B(✓) R(✓) I(✓) C(✓) K(✓)
🎉 SUCCESS 🎉
```

we can see that
it utilized some of the feedback,

For fine tuning

inital

```
secret_word = "BRICK"
past_guesses = []
adapter_id = "wordle-dlai/2"
```

Load fine tuned model

```
adapter_id = "wordle-dlai/2"
```

```
next_turn(past_guesses, secret_word, adapter_id)
```

```
I need to make a first guess for a 5-letter word in Wordle. For the firs
t guess, it's best to choose a word that:

1. Contains common letters
2. Has vowels
3. Has minimal repeated letters

Some good starting words include:
- ARISE (has A, I, E and common consonants R, S)
- TEARS (has E, A and common consonants T, R, S)
- STARE (same letters as TEARS, just rearranged)
- ADIEU (has many vowels: A, I, E, U)
- CRANE (has A, E and common consonants C, R, N)

I think I'll go with "STARE" as my first guess. It has:
- S: a very common letter
- T: another common letter
- A: the most common vowel
- R: another common letter
- E: the most common letter in English

This gives me a good mix of common consonants and vowels, which should p
rovide useful information for my subsequent guesses.
For my first guess in Wordle, I'll choose a word with common letters tha
t will give me good information.</think><guess>STARE</guess>
```

model decides that it wants to pick a first word and it needs to contain

```
--------------------------------------------------------------------------
--------------------------------------------------------------------------

STARE → Feedback: S(x) T(x) A(x) R(-) E(x)
```

After first guess, analyze result

```
adapter_id = "wordle-dlai/2"
```

```
next_turn(past_guesses, secret_word, adapter_id)
```

```
Alright, I'm playing Wordle and need to make a second guess based on the
feedback from my first guess.

Let's analyze what we know:
- Guess 1: STARE → Feedback: S(x) T(x) A(x) R(-) E(x)

This means:
- The letters S, T, A, and E are NOT in the secret word.
- The letter R IS in the word but in the WRONG position (it's not the 4t
h letter).

So we need to find a 5-letter word that:
1. Contains R (but not in the 4th position)
2. Does NOT contain S, T, A, or E

First, let me think about common letters I haven't tried yet: I, O, N,
L, U, C, D, P, M, H, G, B, F, Y, W, K, V, X, Z, J, Q

I should pick a word that:
- Has R in a position other than 4th
- Contains several common letters I haven't tried yet
- Is a valid 5-letter English word

Some possible words:
- PROUD (R in position 2)
- BRING (R in position 2)
- GROWN (R in position 2)
- WORLD (R in position 3)
- CROWD (R in position 2)
- BRICK (R in position 2)
- PRINT (R in position 2) - No, this has T which we know isn't in the wo
rd
- DRILL (R in position 2)
- BRONG (R in position 2)
```

So the model starts by analyzing its first guess,

List of guess word

- The letters S, T, A, and E are NOT in the secret word.
- The letter R IS in the word but in the WRONG position (it's not the 4th letter).

So we need to find a 5-letter word that:
1. Contains R (but not in the 4th position)
2. Does NOT contain S, T, A, or E

First, let me think about common letters I haven't tried yet: I, O, N, L, U, C, D, P, M, H, G, B, F, Y, W, K, V, X, Z, J, Q

I should pick a word that:
- Has R in a position other than 4th
- Contains several common letters I haven't tried yet
- Is a valid 5-letter English word

Some possible words:
- PROUD (R in position 2)
- BRING (R in position 2)
- GROWN (R in position 2)
- WORLD (R in position 3)
- CROWD (R in position 2)
- BRICK (R in position 2)
- PRINT (R in position 2) - No, this has T which we know isn't in the word
- DRILL (R in position 2)
- BRUNG (R in position 2)

Let me choose POUND. Wait, this doesn't have R. Let me try again.

Let me choose PROUD. This has:
- P (not tested yet)
- R (in position 2 - correct position)
- O (not tested yet)
- U (not tested yet)
- D (not tested yet)

it hasn't tried yet, and then thinks about how to use this information.

"PROUD" seems like a good guess to test multiple new letters while properly positioning the R.

- PROUD (R in position 2)
- BRING (R in position 2)
- GROWN (R in position 2)
- WORLD (R in position 3)
- CROWD (R in position 2)
- BRICK (R in position 2)
- PRINT (R in position 2) - No, this has T which we know isn't in the wo
rd
- DRILL (R in position 2)
- BRUNG (R in position 2)

Let me choose POUND. Wait, this doesn't have R. Let me try again.

Let me choose PROUD. This has:
- P (not tested yet)
- R (in position 2, not position 4 where we know it's not)
- O (not tested yet)
- U (not tested yet)
- D (not tested yet)

"PROUD" seems like a good guess to test multiple new letters while prope
rly positioning the R.
Based on the feedback from my first guess, I can see that:
- The letters S, T, A, and E are NOT in the word
- The letter R IS in the word but NOT in the 4th position

For my second guess, I should:
- Include R but place it in a different position than before
- Avoid using S, T, A, and E completely
- Try some new common letters</think><guess>PROUD</guess>

because it has multiple new letters, which will eliminate a lot of words.

STARE → Feedback: S(x) T(x) A(x) R(-) E(x)
PROUD → Feedback: P(x) R(✓) O(x) U(x) D(x)

Second guess

```
adapter_id = "wordle-dlai/2"
```

```
next_turn(past_guesses, secret_word, adapter_id)
```

```
Let's analyze the feedback I have received:

Guess 1: STARE → Feedback: S(x) T(x) A(x) R(-) E(x)
- S, T, A, and E are not in the word
- R is in the word but in the wrong position (it's not the 4th letter)

Guess 2: PROUD → Feedback: P(x) R(✓) O(x) U(x) D(x)
- P, O, U, and D are not in the word
- R is in the correct position (2nd letter)

So far I know:
- R is the 2nd letter
- The letters S, T, A, E, P, O, U, D are not in the word

The word follows the pattern: ?R???

Let me think of some 5-letter words with R as the 2nd letter, avoiding a
ll the letters we know are not in the word:
- BRICK
- BRINGING (too long)
- BRIM
- CRIB (too short)
- CRIME
- CRIMP
- CRISP
- DRILL
- DRINK
- DRIVE
- FRILL
- FRISK
- GRILL
- GRIME
- GRIND
- GRIPS
```

it starts by analyzing the two guesses and tries to use this to think about

List of guess word

```
Let me think of some 5-letter words with R as the 2nd letter, avoiding a
ll the letters we know are not in the word:
- BRICK
- BRINGING (too long)
- BRIM
- CRIB (too short)
- CRIME
- CRIMP
- CRISP
- DRILL
- DRINK
- DRIVE
- FRILL
- FRISK
- GRILL
- GRIME
- GRIND
- GRIPS
- PRICE
- PRINT
- PRIZE
- WRINK (not a valid word)
- WRIST
- WRITE
- WRONG

I need to choose a word that:
1. Has R as the 2nd letter
2. Does not contain S, T, A, E, P, O, U, D
3. Is 5 letters long

Looking at my list, these words fit the criteria:
- BRICK
- BRIM (too short)
- CRIB (too short and R isn't 2nd)
- CRIMP (too long)
- DRINK
- FRICK (contains S which is not in the word)
```

by three question marks, and it comes up with the list of candidate words.

After filter out invalid option

- GRILL (contains L twice, may not be efficient)
- GRIND
- PRICE (contains E which is not in the word)
- PRINT (contains T which is not in the word)
- PRIZE (contains E which is not in the word)
- WRIST (contains S and T which are not in the word)
- WRITE (contains T and E which are not in the word)
- WRONG (contains O which is not in the word)

After filtering out invalid options, good candidates include:
- BRICK
- DRINK
- GRIND

Let me choose BRICK as my next guess. It has R in the correct position (2nd letter) and introduces new letters (B, I, C, K) that we haven't tes
ted yet.
Looking at the feedback for the previous guesses:

From the first guess (STARE):
- S, T, A, and E are not in the word
- R is in the word but in the wrong position

From the second guess (PROUD):
- P, O, U, and D are not in the word
- R is in the correct position (2nd letter)

I now know:
- R is definitely the 2nd letter
- The letters S, T, A, E, P, O, U, D are not in the word

The word follows the pattern: ?R???

I need a 5-letter word with R as the 2nd letter that does not use any of the eliminated letters. I'll try BRICK to test some new letters (B, I, C, K) while keeping R in its confirmed position.</think><guess>BRICK</guess>

this guess is a valid guess based on all the criteria from the additional feedback.

Finial answer

After filtering out invalid options, good candidates include:
- BRICK
- DRINK
- GRIND

Let me choose BRICK as my next guess. It has R in the correct position (2nd letter) and introduces new letters (B, I, C, K) that we haven't tes
ted yet.
Looking at the feedback for the previous guesses:

From the first guess (STARE):
- S, T, A, and E are not in the word
- R is in the word but in the wrong position

From the second guess (PROUD):
- P, O, U, and D are not in the word
- R is in the correct position (2nd letter)

I now know:
- R is definitely the 2nd letter
- The letters S, T, A, E, P, O, U, D are not in the word

The word follows the pattern: ?R???

I need a 5-letter word with R as the 2nd letter that doesn't use any of the eliminated letters. I'll try BRICK to test some new letters (B, I, C, K) while keeping R in its confirmed position.</think><guess>BRICK</guess>

STARE → Feedback: S(x) T(x) A(x) R(-) E(x)
PROUD → Feedback: P(x) R(✓) O(x) U(x) D(x)
BRICK → Feedback: B(✓) R(✓) I(✓) C(✓) K(✓)
🎉 SUCCESS 🎉

Compare base model, fine tuning model iterative thought though reasoning process

```
After filtering out invalid options, good candidates include:
- BRICK
- DRINK
- GRIND

Let me choose BRICK as my next guess. It has R in the correct position
(2nd letter) and introduces new letters (B, I, C, K) that we haven't tes
ted yet.
Looking at the feedback for the previous guesses:

From the first guess (STARE):
- S, T, A, and E are not in the word
- R is in the word but in the wrong position

From the second guess (PROUD):
- P, O, U, and D are not in the word
- R is in the correct position (2nd letter)

I now know:
- R is definitely the 2nd letter
- The letters S, T, A, E, P, O, U, D are not in the word

The word follows the pattern: ?R???

I need a 5-letter word with R as the 2nd letter that doesn't use any of
the eliminated letters. I'll try BRICK to test some new letters (B, I,
C, K) while keeping R in its confirmed position.</think><guess>BRICK</gu
ess>
```

is that the fine-tuned model iteratively thought through its reasoning process

```
STARE → Feedback: S(x) T(x) A(x) R(-) E(x)
PROUD → Feedback: P(x) R(✓) O(x) U(x) D(x)
BRICK → Feedback: B(✓) R(✓) I(✓) C(✓) K(✓)
🎉 SUCCESS 🎉
```

Reinforcement Fine-tuning LLMs

Reward functions

Load deployment for base model

```
from utils import *
import numpy as np

import warnings
warnings.filterwarnings("ignore")
```

```
import torch
```

```
create_deployment()
```

```
WARN: Using a post-release / prod version of the SDK in staging ca
installing from latest master.
```

```
WARN: Using a post-release / prod version of the SDK in staging ca
installing from latest master.
```

```
Connected to Predibase as User(id=497a26a0-c2a8-4476-9a55-5f462a3b
```

```
Deployment qwen2-5-7b-instruct-dlai already exists
```

And this base model for this

Model : qwen2.5 7B instruct

```
model_id = "Qwen/Qwen2.5-7B-Instruct"
```

World_reward

```python
def wordle_reward(guess: str, secret_word: str) -> int:
    if guess.upper() == secret_word.upper():
        return 1   # correct guess
    else:
        return 0   # incorrect guess
```

So this is analogous to in the supervised fine-tuning world,

Run guesses word

```
def wordle_reward(guess: str, secret_word: str) -> int:
    if guess.upper() == secret_word.upper():
        return 1   # correct guess
    else:
        return 0   # incorrect guess
```

```
secret_word = "POUND"

past_guesses = [
    GuessWithFeedback.from_secret(guess="CRANE", secret=secret_word),
    GuessWithFeedback.from_secret(guess="BLOND", secret=secret_word),
    GuessWithFeedback.from_secret(guess="FOUND", secret=secret_word),
]
past_guesses
```

```
[CRANE → Feedback: C(x) R(x) A(x) N(✓) E(x),
 BLOND → Feedback: B(x) L(x) O(-) N(✓) D(✓),
 FOUND → Feedback: F(x) O(✓) U(✓) N(✓) D(✓)]
```

```
response = generate(get_messages(past_guesses))[0]
guess = extract_guess(response)
reward = wordle_reward(guess, secret_word)

print(f"Guessed Word: {guess} -> Reward: {reward}")
```

go ahead and take all these past guesses

Model guess words → reward =0 incorrect

```python
def wordle_reward(guess: str, secret_word: str) -> int:
    if guess.upper() == secret_word.upper():
        return 1   # correct guess
    else:
        return 0   # incorrect guess
```

```python
secret_word = "POUND"

past_guesses = [
    GuessWithFeedback.from_secret(guess="CRANE", secret=secret_word),
    GuessWithFeedback.from_secret(guess="BLOND", secret=secret_word),
    GuessWithFeedback.from_secret(guess="FOUND", secret=secret_word),
]
past_guesses
```

```
[CRANE → Feedback: C(x) R(x) A(x) N(✓) E(x),
 BLOND → Feedback: B(x) L(x) O(-) N(✓) D(✓),
 FOUND → Feedback: F(x) O(✓) U(✓) N(✓) D(✓)]
```

```python
response = generate(get_messages(past_guesses))[0]
guess = extract_guess(response)
reward = wordle_reward(guess, secret_word)

print(f"Guessed Word: {guess} -> Reward: {reward}")
```

```
Guessed Word: GONE -> Reward: 0
```

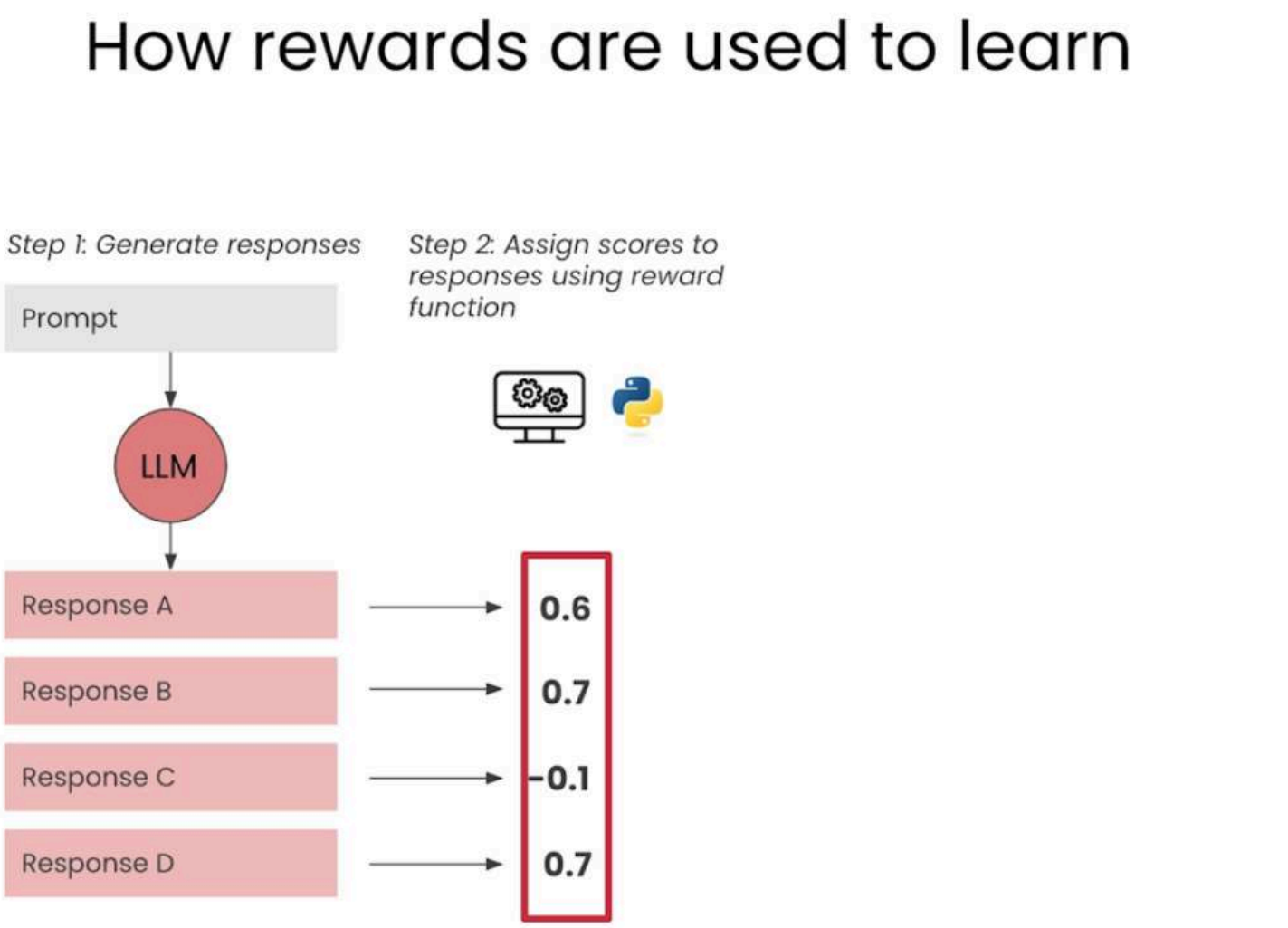
meaning that from the perspective

How rewards are used to learn

Step 1: Generate response

Step 2: Assign **score** to response using **reward function**

DeepLearning.AI

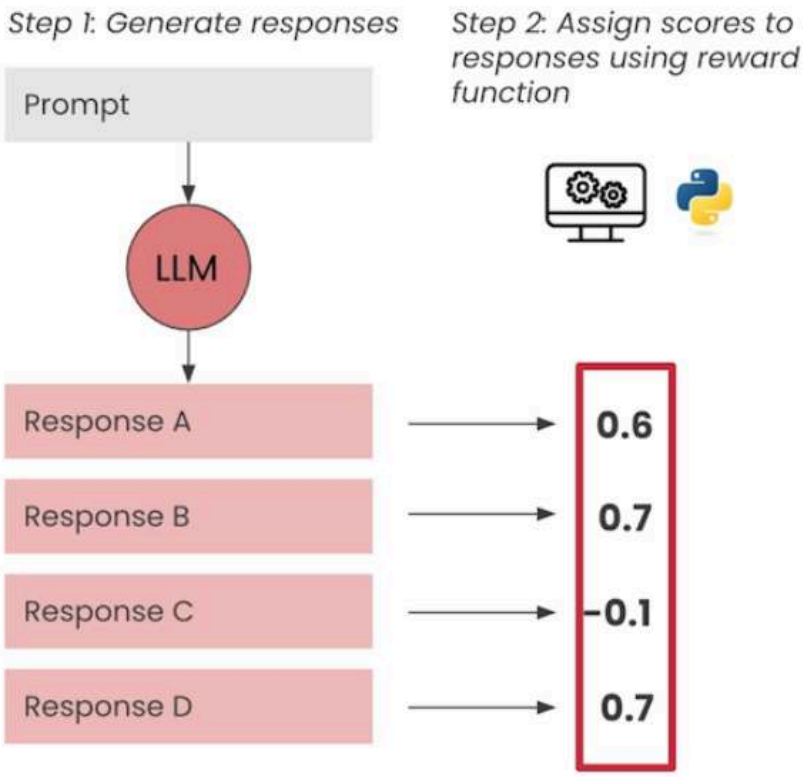


What we're ultimately doing is
we are taking all the different guesses

Agent Goal : maximum reward received

DeepLearning.AI

How rewards are used to learn

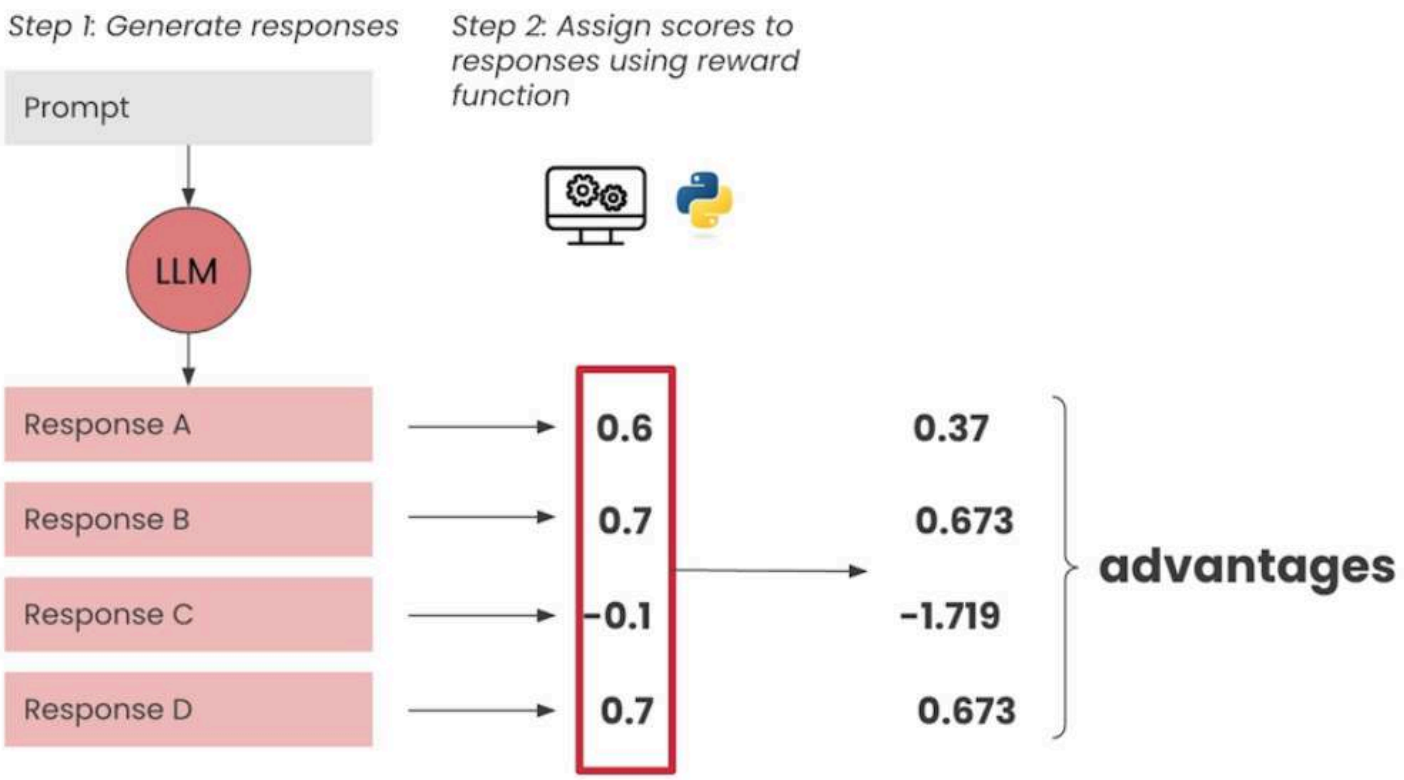


Agent goal: to maximize rewards received over time

Requires: *Diversity in responses* and *diversity of rewards*

DeepLearning.AI

How rewards are used to learn



Agent goal: to maximize rewards received over time

Requires: *Diversity in responses* and *diversity of rewards*

is was something we called the advantage.

Calculating Advantages

Ai = ri - mean ({r1, r2, …, rG}) / std({r1, r2, … , rG}) , where i =---

DeepLearning.AI

Calculating advantages

$$A_i = \frac{r_i - \text{mean}(\{r_1, r_2, \ldots, r_G\})}{\text{std}(\{r_1, r_2, \ldots, r_G\})}$$

A_i - Advantage of i^{th} completion

r_i - Reward assigned to i^{th} completion

All we're really doing here
is we're taking all the rewards

Compute_advantages

```python
def compute_advantages(rewards: list):
    rewards = np.array(rewards)

    # Compute the mean and standard deviation of the rewards
    mean_reward = np.mean(rewards)
    std_reward = np.std(rewards)

    # Avoid division by zero in case of zero variance (typically happens
    # Note: In the GRPO implementation, we add 1e-4 to the std_reward to
    if std_reward == 0:
        return [0] * len(rewards)

    # Divide by stddev of rewards to normalize range to 0
    advantages = (rewards - mean_reward) / std_reward
    return advantages.tolist()
```

just getting all zeros in the event that the standard deviation is zero.

Calculate Rewards

```python
def compute_advantages(rewards: list):
    rewards = np.array(rewards)

    # Compute the mean and standard deviation of the rewards
    mean_reward = np.mean(rewards)
    std_reward = np.std(rewards)

    # Avoid division by zero in case of zero variance (typically happens
    # Note: In the GRPO implementation, we add 1e-4 to the std_reward to
    if std_reward == 0:
        return [0] * len(rewards)

    # Divide by stddev of rewards to normalize range to 0
    advantages = (rewards - mean_reward) / std_reward
    return advantages.tolist()
```

```python
rewards = [0.0, 0.2, 0.4, 0.5, 0.5, 0.6, 0.8, 1.0]
compute_advantages(rewards)
```

```
[-1.6903085094570331,
 -1.01418510567422,
 -0.33806170189140655,
 0.0,
 0.0,
 0.33806170189140655,
 1.0141851056742202,
 1.6903085094570331]
```

from generating responses that look like the things that scored zero.

render_guess_table

```
def render_guess_table(response, reward_fn):
    guesses = [extract_guess(guess) for guess in response]
    rewards = [reward_fn(guess, secret_word) for guess in guesses]
    print_guesses_table(guesses, rewards)
```

```
print(f"Secret: {secret_word}")
response = generate(get_messages(past_guesses), num_guesses=8)
render_guess_table(response, wordle_reward)
```

```
Secret: POUND
+---------+---------+----------+-------------+
|   Index | Guess   |   Reward |   Advantage |
+=========+=========+==========+=============+
|       0 | CRANE   |        0 |           0 |
+---------+---------+----------+-------------+
|       1 | TOWER   |        0 |           0 |
+---------+---------+----------+-------------+
|       2 | WORDN   |        0 |           0 |
+---------+---------+----------+-------------+
|       3 | NOUSE   |        0 |           0 |
+---------+---------+----------+-------------+
|       4 | SORT    |        0 |           0 |
+---------+---------+----------+-------------+
|       5 | FLEND   |        0 |           0 |
+---------+---------+----------+-------------+
|       6 | SOUND   |        0 |           0 |
+---------+---------+----------+-------------+
|       7 | FOOD    |        0 |           0 |
+---------+---------+----------+-------------+
```

Here we can see that again for our secret word a pound.

```
def render_guess_table(response, reward_fn):
    guesses = [extract_guess(guess) for guess in response]
    rewards = [reward_fn(guess, secret_word) for guess in guesses]
    print_guesses_table(guesses, rewards)
```

```
print(f"Secret: {secret_word}")
response = generate(get_messages(past_guesses), num_guesses=8)
render_guess_table(response, wordle_reward)
```

```
Secret: POUND
+---------+---------+----------+-------------+
|   Index | Guess   |   Reward |   Advantage |
+=========+=========+==========+=============+
|       0 | CRANE   |        0 |           0 |
+---------+---------+----------+-------------+
|       1 | TOWER   |        0 |           0 |
+---------+---------+----------+-------------+
|       2 | WORDN   |        0 |           0 |
+---------+---------+----------+-------------+
|       3 | NOUSE   |        0 |           0 |
+---------+---------+----------+-------------+
|       4 | SORT    |        0 |           0 |
+---------+---------+----------+-------------+
|       5 | FLEND   |        0 |           0 |
+---------+---------+----------+-------------+
|       6 | SOUND   |        0 |           0 |
+---------+---------+----------+-------------+
|       7 | FOOD    |        0 |           0 |
+---------+---------+----------+-------------+
```

actually are not going to result in any learning at all.


```
def render_guess_table(response, reward_fn):
    guesses = [extract_guess(guess) for guess in response]
    rewards = [reward_fn(guess, secret_word) for guess in guesses]
    print_guesses_table(guesses, rewards)
```

```
print(f"Secret: {secret_word}")
response = generate(get_messages(past_guesses), num_guesses=8)
render_guess_table(response, wordle_reward)
```

```
Secret: POUND
+---------+---------+----------+-------------+
|   Index | Guess   |   Reward |   Advantage |
+=========+=========+==========+=============+
|       0 | CRANE   |        0 |           0 |
+---------+---------+----------+-------------+
|       1 | TOWER   |        0 |           0 |
+---------+---------+----------+-------------+
|       2 | WORDN   |        0 |           0 |
+---------+---------+----------+-------------+
|       3 | NOUSE   |        0 |           0 |
+---------+---------+----------+-------------+
|       4 | SORT    |        0 |           0 |
+---------+---------+----------+-------------+
|       5 | FLEND   |        0 |           0 |
+---------+---------+----------+-------------+
```


like Crane, which has far fewer correct letters in the correct position.

New wordle reward partial credit function

```python
def wordle_reward_partial_credit(guess: str, secret_word: str) -> float:
    if len(guess) != len(secret_word):
        # no reward for having the wrong number of letters
        return 0.0

    valid_letters = set(secret_word)
    reward = 0.0
    for letter, secret_letter in zip(guess, secret_word):
        if letter == secret_letter:
            # right letter, right location
            reward += 0.2
        elif letter in valid_letters:
            # right letter, wrong location
            reward += 0.1
        else:
            # no reward
            pass
    return reward
```

and the types of reward scores we're getting from this process.

Calculate render_guess_table

temperature = 0

```
print(f"Secret: {secret_word}")
response = generate(get_messages(past_guesses), num_guesses=8, temperatu
render_guess_table(response, wordle_reward_partial_credit)
```

```
Secret: POUND
+---------+---------+----------+-------------+
|   Index | Guess   |   Reward |   Advantage |
+=========+=========+==========+=============+
|       0 | FROWN   |      0.2 |           0 |
+---------+---------+----------+-------------+
|       1 | FROWN   |      0.2 |           0 |
+---------+---------+----------+-------------+
|       2 | FROWN   |      0.2 |           0 |
+---------+---------+----------+-------------+
|       3 | FROWN   |      0.2 |           0 |
+---------+---------+----------+-------------+
|       4 | FROWN   |      0.2 |           0 |
+---------+---------+----------+-------------+
|       5 | FROWN   |      0.2 |           0 |
+---------+---------+----------+-------------+
|       6 | FROWN   |      0.2 |           0 |
+---------+---------+----------+-------------+
|       7 | FROWN   |      0.2 |           0 |
+---------+---------+----------+-------------+
```

And so every time
the model guesses the same thing.

Temperature = 1.3

```
print(f"Secret: {secret_word}")
response = generat                                    esses=8, temperatu
render_guess_table(response, wordle_reward_partial_credit)
```



```
Secret: POUND
+---------+---------+----------+-------------+
|   Index | Guess   |   Reward |   Advantage |
+=========+=========+==========+=============+
|       0 | NOTED   |      0.5 |     2.59248 |
+---------+---------+----------+-------------+
|       1 | END     |        0 |   -0.457496 |
+---------+---------+----------+-------------+
|       2 |         |        0 |   -0.457496 |
+---------+---------+----------+-------------+
|       3 |         |        0 |   -0.457496 |
+---------+---------+----------+-------------+
|       4 |         |        0 |   -0.457496 |
+---------+---------+----------+-------------+
|       5 | TRAIN   |      0.1 |    0.152499 |
+---------+---------+----------+-------------+
|       6 |         |        0 |   -0.457496 |
+---------+---------+----------+-------------+
|       7 | BONY    |        0 |   -0.457496 |
+---------+---------+----------+-------------+
```

has successfully resulted in more variety and the reward scores that are generated.

Temperature = 0.7

```
print(f"Secret: {secret_word}")
response = generate(get_messages(past_guesses), num_guesses=8, temperatu
render_guess_table(response, wordle_reward_partial_credit)
```

Secret: POUND

Index	Guess	Reward	Advantage
0	STONE	0.3	-0.457496
1	ONORD	0.5	0.762493
2	CHINA	0.2	-1.06749
3	OUTDO	0.4	0.152499
4	NOUDI	0.6	1.37249
5	MOULD	0.6	1.37249
6	THIRD	0.2	-1.06749
7	FLOOR	0.2	-1.06749

So now we can see that we're finally starting to get something that looks a lot

```
print(f"Secret: {secret_word}")
response = generat                                    esses=8, temperatu
render_guess_table(response, wordle_reward_partial_credit)
```



```
Secret: POUND
+---------+---------+----------+-------------+
|   Index | Guess   |   Reward |   Advantage |
+=========+=========+==========+=============+
|       0 | STONE   |      0.3 |   -0.457496 |
+---------+---------+----------+-------------+
|       1 | ONORD   |      0.5 |    0.762493 |
+---------+---------+----------+-------------+
|       2 | CHINA   |      0.2 |   -1.06749  |
+---------+---------+----------+-------------+
|       3 | OUTDO   |      0.4 |    0.152499 |
+---------+---------+----------+-------------+
|       4 | NOUDI   |      0.6 |    1.37249  |
+---------+---------+----------+-------------+
|       5 | MOULD   |      0.6 |    1.37249  |
+---------+---------+----------+-------------+
|       6 | THIRD   |      0.2 |   -1.06749  |
+---------+---------+----------+-------------+
|       7 | FLOOR   |      0.2 |   -1.06749  |
+---------+---------+----------+-------------+
```

that are more likely
to receive higher reward

```
print(f"Secret: {secret_word}")
response = generate(get_messages(past_guesses), num_guesses=8, temperatu
render_guess_table(response, wordle_reward_partial_credit)
```

Secret: POUND

Index	Guess	Reward	Advantage
0	STONE	0.3	-0.457496
1	ONORD	0.5	0.762493
2	CHINA	0.2	-1.06749
3	OUTDO	0.4	0.152499
4	NOUDI	0.6	1.37249
5	MOULD	0.6	1.37249
6	THIRD	0.2	-1.06749
7	FLOOR	0.2	-1.06749

and therefore more likely to ultimately get the word correct.

Reward functions with LLM as a judge

Reinforcement Fine-tuning LLMs

Reward functions with LLM as a judge

In this lesson, you'll write a reward function for more subjective

Initial LLM

```
import os
import re
from datasets import load_dataset
from dotenv import load_dotenv
from openai import OpenAI

from utils import *

load_dotenv("../.env")

client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])

pb_client = OpenAI(
    base_url="https://serving.staging.predibase.com/c1c29f/deployments/
    api_key=os.environ["PREDIBASE_API_KEY"],
)
```

Let's start
by importing our standard dependencies.

Load dataset

```
ds = load_dataset("mrSoul7766/ECTSum")
transcript = ds["train"][1]["text"]
print(transcript[:1983])
```

```
I'm joined by Tom Greco, our President and Chief Executive Officer; an
Jeff Shepherd, our Executive Vice President and Chief Financial Office
We also hope that you and your families are healthy and safe.
The health and safety of our team members and customers has been a top
riority over the past year.
With strength across all channels, we delivered comparable store sales
rowth of 24.7%, and margin expansion of 478 basis points versus the pr
r year.
On a two-year stack, our comp sales growth was 15.4%.
Adjusted diluted earnings per share of $3.34 represented an all-time q
rterly high for AAP, and improved more than 230% compared to Q1 2020.
Free cash flow of $259 million was up significantly versus the prior y
r, and we returned over $203 million to our shareholders through a com
nation of share repurchases and our quarterly cash dividend.
In addition, we recently announced an updated capital allocation frame
rk targeting top quartile total shareholder return, highlighted by ope
ting income growth, share repurchases and an increase in our dividend.
This further reinforces our confidence in future cash generation and o
commitment to returning excess cash to shareholders.
As outlined in April, we are building an ownership culture, as well as
differentiated operating model at Advance.
Over the past few years, we've made substantial investments in our bra
s, our digital and physical assets, and our team.
These investments, along with external factors, enabled us to post a s
ong start to 2021.
Clearly, the federal stimulus package, along with our first real winte
weather in three years, was a benefit to our industry.
From a categ                                                    eries, ap
arance chemicals and wipers.
Geographically, all                                      growth.
Importantly, over the past year, the Northeast, our largest region, ha
been below our overall reported growth rate and well below that of our
op-performing regions.
```

You can see that these transcripts tend to be quite long.

Summarize prompt

```
SUMMARIZE_PROMPT = """Generate a concise summary of the information in

Only respond with the summary, do not include any extraneous text.

Transcript:

{transcript}
"""
```

The following earnings call transcript only respond to the summary.

Summarize function

```
SUMMARIZE_PROMPT = """Generate a concise summary of the information in

Only respond wit                                                    eous text.

Transcript:

{transcript}
"""

def summarize(transcript, n=1):
    prompt = SUMMARIZE_PROMPT.format(transcript=transcript)
    messages = [
        {"role": "user", "content": prompt},
    ]

    return pb_client.chat.completions.create(
        model=MODEL_NAME,
        messages=messages,
        n=n,
        temperature=0.9,
    )
```

learn.deeplearning.ai – To exit full screen, press Esc

We're then going to convert this into the chat API format.

Run summarize,

A lot of unwanted summary

```
resp = summarize(transcript)
summary = resp.choices[0].message.content
print(summary)
```

```
Here is a concise summary of the earnings call transcript:

Advance Auto Parts (AAP) reports Q1 2021 earnings:

* Comparable store sales growth of 24.7%, with a 15.4% two-year stack
* Adjusted diluted EPS of $3.34, up 230% from Q1 2020
* Free cash flow of $259 million, up significantly from Q1 2020
* Returned $203 million to shareholders through share repurchases and di
vidend

AAP highlights its growth in various areas:

* Professional business grew 20% in Q1, with momentum into Q2
* DIY omnichannel business led growth for the fourth consecutive quarter
* DieHard batteries continue to have strong momentum
* Loyalty program drives growth in VIP and Elite membership

The company outlines its plans to expand margins:

* Category management initiatives aim to drive up to 200 basis points of
margin expansion by 2023
* Supply chain initiatives include tiering, transforming in-market deliv
ery, and improving customer fulfillment

AAP also provides Q1 financial highlights:

* Net sales increased 23.4% to $3.3 billion
* Adjusted gross profit margin expanded 91 basis points to 44.8%
* Adjusted operating income increased 478 basis points to 9%

The company raises its comp sales guidance to up 4% to 6% and updates it
s adjusted OI margin range to 9% to 9.2%.
```

and it is a lot shorter
than the original transcript.

Judge prompt : rating

```
JUDGE_PROMPT_V1 = """
Rate the following summary of an earnings call transcript on a
scale from 1 to 10.

1 means the summary is very poor, 10 means the summary is very good.

Provide reasoning followed by the final score at the end
surrounded by <score> tags.

For example:

<score>1</score>

Transcript:

{transcript}

Summary:

{summary}
"""
```

where 1 is very poor
and 10 is very good.

Jude reward function

```python
def judge_reward_v1(
    transcript: str,
    summary: str,
    model: str = "gpt-4o-mini",
    verbose: bool = False,
) -> float:
    prompt = JUDGE_PROMPT_V1.format(
        transcript=transcript,
        summary=summary,
    )
    messages = [
        {"role": "user", "content": prompt},
    ]

    resp = client.chat.completions.create(
        model=model,
        messages=messages,
        n=1,
        temperature=0,
        )
    completion = resp.choices[0].message.content

    if verbose:
        print(completion)

try:
    match = re.search(r"<score>(\d+)<\/score>", completion)
    if match is None:
        return 0

    # Extract the "score" part from the completion
    score = match.group(1).strip()
    score = int(score)
except:
    score = 0
```

and the summary.

Putting this all behind a reward function

```
    summary: str,
    model: str = "gpt-4o-mini",
    verbose: bool = False,
) -> float:
    prompt = JUDGE_PROMPT_V1.format(
        transcript=transcript,
        summary=summary,
    )
    messages = [
        {"role": "user", "content": prompt},
    ]

    resp = client.chat.completions.create(
        model=model,
        messages=messages,
        n=1,
        temperature=0,
        )
    completion = resp.choices[0].message.content

    if verbose:
        print(completion)

    try:
        match = re.search(r"<score>(\d+)<\/score>", completion)
        if match is None:
            return 0

        # Extract the "score" part from the completion
        score = match.group(1)
        score = int(score)
    except:
        score = 0

    return score / 10
```

And then we're going to extract out the final score using this regular expression.

Run judge_reward score function

```
score = judge_reward_v1(transcript, summary, verbose=True)
print(score)
```

```
The summary provided captures the key points from the earnings call tran
script effectively. It highlights the significant financial metrics such
as comparable store sales growth, adjusted diluted earnings per share, a
nd free cash flow, which are crucial for understanding the company's per
formance. Additionally, it mentions the growth in both the Professional
and DIY segments, as well as the success of the DieHard brand and the lo
yalty program, which are important for assessing the company's market po
sition and customer engagement.

The summary also addresses the company's strategic initiatives aimed at
margin expansion and supply chain improvements, which are essential for
understanding the future direction of the business. Furthermore, it incl
udes the updated guidance for comparable sales and operating income marg
in, providing a forward-looking perspective.

However, while the summary is comprehensive, it could benefit from a bit
more context or analysis regarding the implications of these results and
strategies. For instance, discussing how the growth in the Professional
segment might impact the overall business strategy or the competitive la
ndscape could add depth.

Overall, the summary is clear, concise, and covers the essential aspects
of the earnings call, making it a strong representation of the original
transcript.

<score>9</score>
0.9
```

this is reasonable, and then provides this final score at the end, which is 0.9.

Generate 8 summarize output sample, calculate scores

```
resp = summarize(transcript, n=8)
summaries = [choice.message.content for choice in resp.choices]
```

```
scores = [judge_reward_v1(transcript, summary) for summary in summaries]
scores
```

```
[0.8, 0.7, 0.8, 0.9, 0.8, 0.8, 0.7, 0.8]
```

But importantly,
you'll notice that it never really notices

Generate multiple voice

```
from pydantic import BaseModel
from random import shuffle


QUIZ_PROMPT = """
Generate a multiple-choice quiz based on the information
in the following earnings call transcript.

Example:

```
1. What was the q1 adjusted earnings per share?
a) $3.34
b) $5.32
c) $2.49
d) $7.78

2. By what percent did same store sales rise in q1?
a) 29.4%
b) 32.1%
c) 24.7%
d) 21.2%

===== ANSWERS =====
1. a
2. c
```

Limit the length of the quiz to the top 10 most relevant questions for f

Transcript:

{transcript}
"""
```

is most relevant to a financial analyst should be looking at these summaries.

```python
from pydantic import BaseModel
from random import shuffle


QUIZ_PROMPT = """
Generate a multiple-choice quiz based on the information
in the following earnings call transcript.

Example:

```
1. What was the q1 adjusted earnings per share?
a) $3.34
b) $5.32
c) $2.49
d) $7.78

2. By what percent did same store sales rise in q1?
a) 29.4%
b) 32.1%
c) 24.7%
d) 21.2%

===== ANSWERS =====
1. a
2. c
```

Limit the length of the quiz to the top 10 most relevant questions for f

Transcript:

{transcript}
"""
```

and objective
task for the LLM to construct this quiz.

```python
class Question(BaseModel):
    text: str
    options: list[str]
    answer: int

    def shuffle_options(self) -> None:
        """Shuffle the options while preserving the correct answer"""
        # Get the correct answer text
        correct = self.options[self.answer]

        # Shuffle the options
        shuffled = self.options.copy()
        shuffle(shuffled)

        # Update the answer index to match new position
        self.options = shuffled
        self.answer = shuffled.index(correct)

    def __str__(self) -> str:
        """Pretty print a single question"""
        output = [self.text]
        for i, option in enumerate(self.options):
            output.append(f"{chr(65+i)}. {option}")
        return "\n".join(output)
```

Now we could go about generating this quiz by having the model

Pydantic schema

q

```python
class Question(BaseModel):
    text: str
    options: list[str]
    answer: int

    def shuffle_options(self) -> None:
        """Shuffle the options while preserving the correct answer"""
        # Get the correct answer text
        correct = self.options[self.answer]

        # Shuffle the options
        shuffled = self.options.copy()
        shuffle(shuffled)

        # Update the answer index to match new position
        self.options = shuffled
        self.answer = shuffled.index(correct)

    def __str__(self) -> str:
        """Pretty print a single question"""
        output = [self.text]
        for i, option in enumerate(self.options):
            output.append(f"{chr(65+i)}. {option}")
        return "\n".join(output)
```

that defines the output structure that we're looking for.

Str

```python
class Question(BaseModel):
    text: str
    options: list[str]
    answer: int

    def shuffle_options(self) -> None:
        """Shuffle the options while preserving the correct answer"""
        # Get the correct answer text
        correct = self.options[self.answer]

        # Shuffle the options
        shuffled = self.options.copy()
        shuffle(shuffled)

        # Update the answer index to match new position
        self.options = shuffled
        self.answer = shuffled.index(correct)

    def __str__(self) -> str:
        """Pretty print a single question"""
        output = [self.text]
        for i, option in enumerate(self.options):
            output.append(f"{chr(65+i)}. {option}")
        return "\n".join(output)
```

these options
and these questions as a string.

Quiz

```python
class Quiz(BaseModel):
    questions: list[Question]

    def shuffle_all_questions(self) -> None:
        """Shuffle the options for all questions in the quiz"""
        for question in self.questions:
            question.shuffle_options()

    def __str__(self) -> str:
        """Pretty print the entire quiz"""
        output = []
        for i, question in enumerate(self.questions, 1):
            output.append(f"\nQuestion {i}:")
            output.append(str(question))
        return "\n".join(output)
```

that we can use to shuffle
all of the options for every question.

```python
class Quiz(BaseModel):
    questions: list[Question]

    def shuffle_all_questions(self) -> None:
        """Shuffle the options for all questions in the quiz"""
        for question in self.questions:
            question.shuffle_options()

    def __str__(self) -> str:
        """Pretty print the entire quiz"""
        output = []
        for i, question in enumerate(self.questions, 1):
            output.append(f"\nQuestion {i}:")
            output.append(str(question))
        return "\n".join(output)
```

so that it can be inserted
directly into the prompt.

```python
def create_quiz(transcript: str):
    prompt = QUIZ_PROMPT.format(transcript=transcript)
    messages = [
        {"role": "user", "content": prompt},
    ]
    resp = client.beta.chat.completions.parse(
        model="gpt-4o-mini",
        messages=messages,
        temperature=0.7,
        response_format=Quiz,
    )

    quiz = resp.choices[0].message.parsed
    quiz.shuffle_all_questions()

    return quiz
```

It's going to take a string as input
which is the transcript.

```python
def create_quiz(transcript: str):
    prompt = QUIZ_PROMPT.format(transcript=transcript)
    messages = [
        {"role": "user", "content": prompt},
    ]
    resp = client.beta.chat.completions.parse(
        model="gpt-4o-mini",
        messages=messages,
        temperature=0.7,
        response_format=Quiz,
    )

    quiz = resp.choices[0].message.parsed
    quiz.shuffle_all_questions()

    return quiz
```

As we might want to play around with different variations of the quiz.

7:56 / 12:35 • Create Objective Quiz-Based Rewards CC 1x Auto

```python
def create_quiz(transcript: str):
    prompt = QUIZ_PROMPT.format(transcript=transcript)
    messages = [
        {"role": "user", "content": prompt},
    ]
    resp = client.beta.chat.completions.parse(
        model="gpt-4o-mini",
        messages=messages,
        temperature=0.7,
        response_format=Quiz,
    )

    quiz = resp.choices[0].message.parsed
    quiz.shuffle_all_questions()

    return quiz
```

which is one of these quiz objects.

```python
def create_quiz(transcript: str):
    prompt = QUIZ_PROMPT.format(transcript=transcript)
    messages = [
        {"role": "user", "content": prompt},
    ]
    resp = client.beta.chat.completions.parse(
        model="gpt-4o-mini",
        messages=messages,
        temperature=0.7,
        response_format=Quiz,
    )

    quiz = resp.choices[0].message.parsed
    quiz.shuffle_all_questions()

    return quiz
```

tend to be pre predictable in terms of where they put the right answer.

```python
def create_quiz(transcript: str):
    prompt = QUIZ_PROMPT.format(transcript=transcript)
    messages = [
        {"role": "user", "content": prompt},
    ]
    resp = client.beta.chat.completions.parse(
        model="gpt-4o-mini",
        messages=messages,
        temperature=0.7,
        response_format=Quiz,
    )

    quiz = resp.choices[0].message.parsed
    quiz.shuffle_all_questions()

    return quiz
```

So in order to account

```python
quiz = create_quiz(transcript)
print(quiz)
```

```
Question 1:
What was the Q1 adjusted diluted earnings per share for the company?
A. $2.49
B. $3.34
C. $1.00
D. $5.00

Question 2:
What percentage did comparable store sales grow in Q1?
A. 29.4%
B. 24.7%
C. 20.1%
D. 32.1%

Question 3:
How much free cash flow was generated in Q1?
A. $330 million
B. $500 million
C. $259 million
D. $203 million

Question 4:
What was the adjusted operating income margin in Q1?
A. 8.5%
B. 9%
C. 10%
D. 7.5%

Question 5:
By how many basis points did the adjusted gross profit margin expand in
Q1?
A. 80 ba
B. 150 basis points
C. 91 basis points
D. 100 basis points
```

As you can see, the quiz consists

```
C. $1.00
D. $5.00

Question 2:
What percentage did comparable store sales grow in Q1?
A. 29.4%
B. 24.7%
C. 20.1%
D. 32.1%

Question 3:
How much free cash flow was generated in Q1?
A. $330 million
B. $500 million
C. $259 million
D. $203 million

Question 4:
What was the adjusted operating income margin in Q1?
A. 8.5%
B. 9%
C. 10%
D. 7.5%

Question 5:
By how many basis points did the adjusted gross profit margin expand in
Q1?
A. 80 basis points
B. 150 basis points
C. 91 basis points
D. 100 basis points

Question 6:
How much did the
s and dividends?
A. $100 million
B. $300 million
C. $500 million
D. $203 million
```

to the particular earnings call transcripts.

```
Question 5:
By how many basis points did the adjusted gross profit margin expand in
Q1?
A. 80 basis points
B. 150 basis points
C. 91 basis points
D. 100 basis points

Question 6:
How much did the company return to shareholders through share repurchase
s and dividends?
A. $100 million
B. $300 million
C. $500 million
D. $203 million

Question 7:
What is the expected comp sales growth rate for the year?
A. 5% to 7%
B. 0% to 2%
C. 4% to 6%
D. 2% to 4%

Question 8:
What was the increase in sales per store targeted by 2023?
A. $1.8 million
B. $2.0 million
C. $1.5 million
D. $1.2 million

Question 9:
How many new independent locations are being added to the Carquest famil
y?
A. 15
B. 29
C. 35
D. 50

Question 10:
```

```
B. $300 million
C. $500 million
D. $203 million

Question 7:
What is the expected comp sales growth rate for the year?
A. 5% to 7%
B. 0% to 2%
C. 4% to 6%
D. 2% to 4%

Question 8:
What was the increase in sales per store targeted by 2023?
A. $1.8 million
B. $2.0 million
C. $1.5 million
D. $1.2 million

Question 9:
How many new independent locations are being added to the Carquest famil
y?
A. 15
B. 29
C. 35
D. 50

Question 10:
What was the increase in VIP members from the loyalty program in Q1?
A. 10%
B. 14%
C. 20%
D. 5%
```

We won't show it explicitly
in this lesson in the interest of time,

```
B. $300 million
C. $500 million
D. $203 million

Question 7:
What is the expected comp sales growth rate for the year?
A. 5% to 7%
B. 0% to 2%
C. 4% to 6%
D. 2% to 4%

Question 8:
What was the increase in sales per store targeted by 2023?
A. $1.8 million
B. $2.0 million
C. $1.5 million
D. $1.2 million

Question 9:
How many new independent locations are being added to the Carquest famil
y?
A. 15
B. 29
C. 35
D. 50

Question 10:
What was the increase in VIP members from the loyalty program in Q1?
A. 10%
B. 14%
C. 20%
D. 5%
```

Now that we've generated the quiz,
we're going to write the helper function

```
letter_to_index = {"A": 0, "B": 1, "C": 2, "D": 3}
index_to_letter = ["A", "B", "C", "D"]


TAKE_QUIZ_PROMPT = """Use the provided summary of a transcript
to answer the following quiz.

Quiz:

{quiz}

Summary:

{summary}

Respond with just a list of answers and no additional text,
for example:

[A, D, C, B, B, C, D, A, A, B]

You must provide an answer for all 10 questions.
If you don't know the answer, answer with "0" for that question.
Example:

[A, D, 0, B, B, C, D, A, A, B]
"""
```

So let's go ahead and define our prompt for this use case.

```
letter_to_index = {"A": 0, "B": 1, "C": 2, "D": 3}
index_to_letter = ["A", "B", "C", "D"]


TAKE_QUIZ_PROMPT = """Use the provided summary of a transcript
to answer the following quiz.

Quiz:

{quiz}

Summary:

{summary}

Respond with just a list of answers and no additional text,
for example:

[A, D, C, B, B, C, D, A, A, B]

You must provide an answer for all 10 questions.
If you don't know the answer, answer with "0" for that question.
Example:

[A, D, 0, B, B, C, D, A, A, B]
"""
```

we don't want the model to take a random guess.

9:55 / 12:35 • Have the Model Take and Score the Quiz 1x Auto

```
letter_to_index = {"A": 0, "B": 1, "C": 2, "D": 3}
index_to_letter = ["A", "B", "C", "D"]


TAKE_QUIZ_PROMPT = """Use the provided summary of a transcript
to answer the following quiz.

Quiz:

{quiz}

Summary:

{summary}

Respond with just a list of answers and no additional text,
for example:

[A, D, C, B, B, C, D, A, A, B]

You must provide an answer for all 10 questions.
If you don't know the answer, answer with "0" for that question.
Example:

[A, D, 0, B, B, C, D, A, A, B]
"""
```

If the model legitimately doesn't know what the answer to a particular question

```python
def take_quiz(summary, quiz):
    question_strs = []
    for question in quiz.questions:
        question_str = question.text
        for i, option in enumerate(question.options):
            letter = index_to_letter[i]
            question_str += f"\n{letter}. {option}"
        question_strs.append(question_str)
    quiz_str = "\n\n".join(question_strs)

    prompt = TAKE_QUIZ_PROMPT.format(quiz=quiz_str, summary=summary)
    resp = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": prompt}],
        temperature=0,
    )
    resp_str = resp.choices[0].message.content

    # Convert string representation of list to actual list of strings
    answers = resp_str.strip('[]').split(', ')

    return answers
```

Now we take the summary as input as well as the quiz.

```python
def take_quiz(summary, quiz):
    question_strs = []
    for question in quiz.questions:
        question_str = question.text
        for i, option in enumerate(question.options):
            letter = index_to_letter[i]
            question_str += f"\n{letter}. {option}"
        question_strs.append(question_str)
    quiz_str = "\n\n".join(question_strs)

    prompt = TAKE_QUIZ_PROMPT.format(quiz=quiz_str, summary=summary)
    resp = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": prompt}],
        temperature=0,
    )
    resp_str = resp.choices[0].message.content

    # Convert string representation of list to actual list of strings
    answers = resp_str.strip('[]').split(', ')

    return answers
```

GPT-4o-mini here with temperature zero.

```python
def take_quiz(summary, quiz):
    question_strs = []
    for question in quiz.questions:
        question_str = question.text
        for i, option in enumerate(question.options):
            letter = index_to_letter[i]
            question_str += f"\n{letter}. {option}"
        question_strs.append(question_str)
    quiz_str = "\n\n".join(question_strs)

    prompt = TAKE_QUIZ_PROMPT.format(quiz=quiz_str, summary=summary)
    resp = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": prompt}],
        temperature=0,
    )
    resp_str = resp.choices[0].message.content

    # Convert string representation of list to actual list of strings
    answers = resp_str.strip('[]').split(', ')

    return answers
```

```python
answers = take_quiz(summaries[0], quiz)
answers
```

Let's run it and see what we get.

```python
def take_quiz(summary, quiz):
    question_strs = []
    for question in quiz.questions:
        question_str = question.text
        for i, option in enumerate(question.options):
            letter = index_to_letter[i]
            question_str += f"\n{letter}. {option}"
        question_strs.append(question_str)
    quiz_str = "\n\n".join(question_strs)

    prompt = TAKE_QUIZ_PROMPT.format(quiz=quiz_str, summary=summary)
    resp = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": prompt}],
        temperature=0,
    )
    resp_str = resp.choices[0].message.content

    # Convert string representation of list to actual list of strings
    answers = resp_str.strip('[]').split(', ')

    return answers
```

```python
answers = take_quiz(summaries[0], quiz)
answers
```

```
['B', 'B', 'C', 'B', 'A', 'C', 'C', 'A', '0', 'B']
```

As you can see, there's
a good variety of different answers here.


```python
def take_quiz(summary, quiz):
    question_strs = []
    for question in quiz.questions:
        question_str = question.text
        for i, option in enumerate(question.options):
            letter = index_to_letter[i]
            question_str += f"\n{letter}. {option}"
        question_strs.append(question_str)
    quiz_str = "\n\n".join(question_strs)

    prompt = TAKE_QUIZ_PROMPT.format(quiz=quiz_str, summary=summary)
    resp = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": prompt}],
        temperature=0,
    )
    resp_str = resp.choices[0].message.content

    # Convert string representation of list to actual list of strings
    answers = resp_str.strip('[]').split(', ')

    return answers
```

```python
answers = take_quiz(summaries[0], quiz)
answers
```

```
['B', 'B', 'C', 'B', 'A', 'C', 'C', 'A', '0', 'B']
```


the answers that came out of the
take quiz function above.

```python
def score_quiz_answers(answers, quiz):
    assert len(answers) == len(quiz.questions)

    total = len(answers)
    correct = 0
    for answer, question in zip(answers, quiz.questions):
        expected_answer = index_to_letter[question.answer]
        if answer == expected_answer:
            correct += 1
    return correct / total
```

So let's write this helper
function score quiz answers.

```python
def score_quiz_answers(answers, quiz):
    assert len(answers) == len(quiz.questions)

    total = len(answers)
    correct = 0
    for answer, question in zip(answers, quiz.questions):
        expected_answer = index_to_letter[question.answer]
        if answer == expected_answer:
            correct += 1
    return correct / total
```

And then we're going to iterate over every one of the answers

```python
def score_quiz_answers(answers, quiz):
    assert len(answers) == len(quiz.questions)

    total = len(answers)
    correct = 0
    for answer, question in zip(answers, quiz.questions):
        expected_answer = index_to_letter[question.answer]
        if answer == expected_answer:
            correct += 1
    return correct / total
```

Divide that by the total number of questions in the quiz.

```python
def score_quiz_answers(answers, quiz):
    assert len(answers) == len(quiz.questions)

    total = len(answers)
    correct = 0
    for answer, question in zip(answers, quiz.questions):
        expected_answer = index_to_letter[question.answer]
        if answer == expected_answer:
            correct += 1
    return correct / total
```

```python
score_quiz_answers(answers, quiz)
```

Let's go ahead and run that.

```python
def score_quiz_answers(answers, quiz):
    assert len(answers) == len(quiz.questions)

    total = len(answers)
    correct = 0
    for answer, question in zip(answers, quiz.questions):
        expected_answer = index_to_letter[question.answer]
        if answer == expected_answer:
            correct += 1
    return correct / total
```

```python
score_quiz_answers(answers, quiz)
```

```
0.7
```

So that was computing the scores for just one of our summaries.

```python
all_answers = []
quiz_rewards = []
for summary in summaries:
    answers = take_quiz(summary, quiz)
    all_answers.append(answers)
    quiz_rewards.append(score_quiz_answers(answers, quiz))
```

those answers using our scoring function and keep track of that as well.

```python
all_answers = []
quiz_rewards = []
for summary in summaries:
    answers = take_quiz(summary, quiz)
    all_answers.append(answers)
    quiz_rewards.append(score_quiz_answers(answers, quiz))
```

```python
print_quiz_table(all_answers, quiz_rewards)
```

```
+---------+----------+--------------+
|   Index |   Reward |   Advantage |
+=========+==========+==============+
|       0 |      0.7 |     1.04407  |
+---------+----------+--------------+
|       1 |      0.4 |    -1.23391  |
+---------+----------+--------------+
|       2 |      0.6 |     0.284747 |
+---------+----------+--------------+
|       3 |      0.3 |    -1.99323  |
+---------+----------+--------------+
|       4 |      0.6 |     0.284747 |
+---------+----------+--------------+
|       5 |      0.6 |     0.284747 |
+---------+----------+--------------+
|       6 |      0.7 |     1.04407  |
+---------+----------+--------------+
|       7 |      0.6 |     0.284747 |
+---------+----------+--------------+
```

We can see that our quiz based approach provides

```
all_answers = []
quiz_rewards = []
for summary in summaries:
    answers = take_quiz(summary, quiz)
    all_answers.append(answers)
    quiz_rewards.append(score_quiz_answers(answers, quiz))
```

```
print_quiz_table(all_answers, quiz_rewards)
```

Index	Reward	Advantage
0	0.7	1.04407
1	0.4	-1.23391
2	0.6	0.284747
3	0.3	-1.99323
4	0.6	0.284747
5	0.6	0.284747
6	0.7	1.04407
7	0.6	0.284747

And so as a result, we can expect that we'll get a nice amount of learning

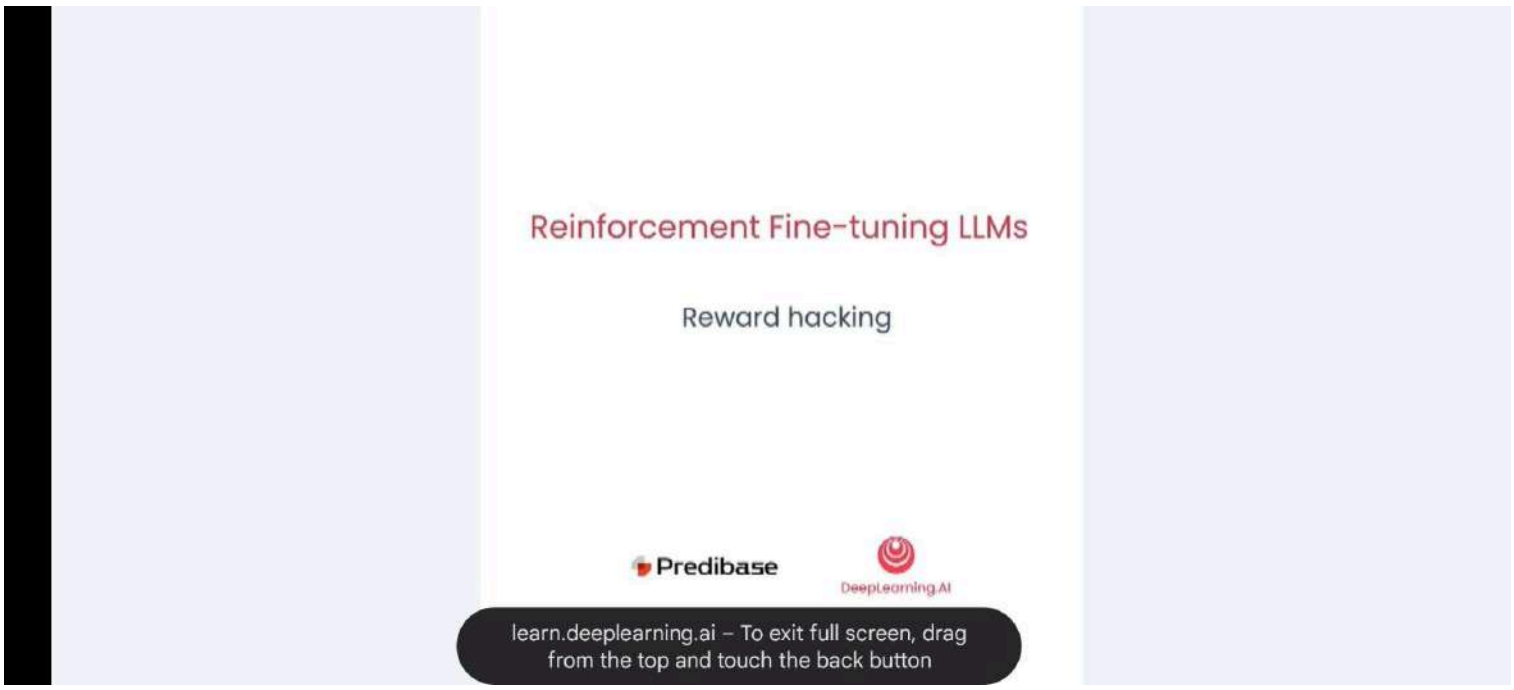

```
all_answers = []
quiz_rewards = []
for summary in summaries:
    answers = take_quiz(summary, quiz)
    all_answers.append(answers)
    quiz_rewards.append(score_quiz_answers(answers, quiz))
```

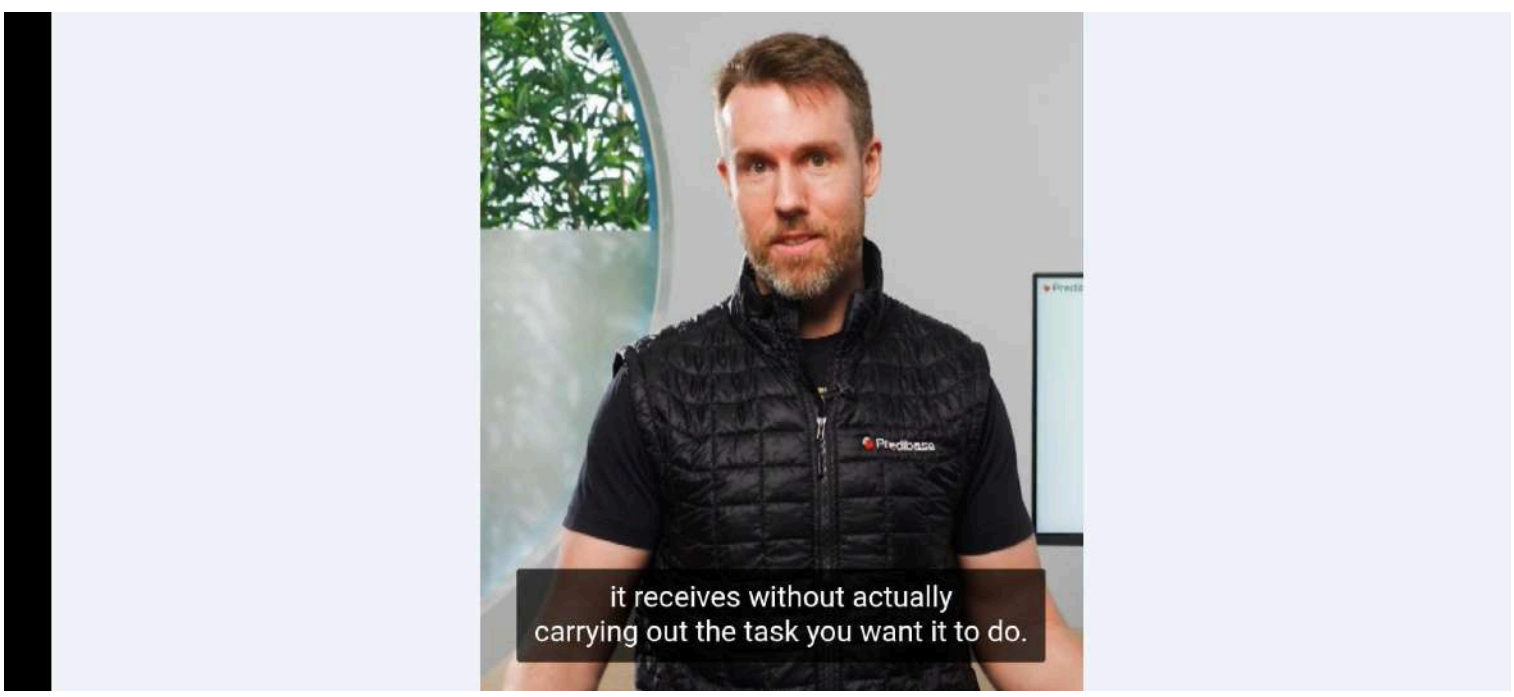
```
print_quiz_table(all_answers, quiz_rewards)
```

Index	Reward	Advantage
0	0.7	1.04407
1	0.4	-1.23391
2	0.6	0.284747
3	0.3	-1.99323
4	0.6	0.284747
5	0.6	0.284747
6	0.7	1.04407
7	0.6	0.284747

to encourage kind of bad behavior, so-called reward hacking.

Reward Hacking





Load Dataset

Prompt transcrpt

```
prompt = f"""Generate a concise bulleted summary of the
information in the following earnings call transcript.

Only respond with the summary, do not include any extraneous text.

Transcript:

{transcript}
"""

completions = pb_client.chat.completions.create(
    model=MODEL_NAME,
    messages=[
        {"role": "user", "content": prompt},
    ],
    n=8,
    temperature=0.9,
)
```

Let's start by generating eight summaries using the same prompt as before.

```
ds = load_dataset("mrSoul7766/ECTSum")
transcript = ds["train"][1]["text"]
```

```
quiz = generate_quiz(transcript)
print(quiz)
```

```
Question 1:
What was the adjusted diluted earnings per share for Q1?
A. $2.49
B. $1.00
C. $3.34
D. $5.32

Question 2:
By what percentage did comparable store sales grow in Q1?
A. 32.1%
B. 21.2%
C. 29.4%
D. 24.7%

Question 3:
What was the net sales increase percentage for Q1?
A. 25.1%
B. 23.4%
C. 20.5%
D. 28.6%

Question 4:
What is the adjusted operating income margin for Q1?
A. 9%
B. 10%
C. 8.5%
D. 7%

Question 5:
How much was the free cash flow for Q1?
A. $203 million
```

to construct our quiz.

```
prompt = f"""Generate a concise bulleted summary of the
information in the following earnings call transcript.

Only respond with the summary, do not include any extraneous text.

Transcript:

{transcript}
"""

completions = pb_client.chat.completions.create(
    model=MODEL_NAME,
    messages=[
        {"role": "user", "content": prompt},
    ],
    n=8,
    temperature=0.9,
)
```

Generate a concise bulleted summary information and earnings call transcript.

```
responses = [choice.message.content for choice in completions.choices]
quiz_rewards = [quiz_reward(resp, quiz) for resp in responses]
quiz_rewards
```

Now let's see how these different summaries score on the quiz.

```
responses = [choice.message.content for choice in completions.choices]
quiz_rewards = [quiz_reward(resp, quiz) for resp in responses]
quiz_rewards
```

```
[0.625, 0.625, 0.875, 0.75, 0.625, 0.5, 0.625, 0.625]
```

As we can see we have some variety in the outputs for this quiz.

```
responses = [choice.message.content for choice in completions.choices]
quiz_rewards = [quiz_reward(resp, quiz) for resp in responses]
quiz_rewards
```

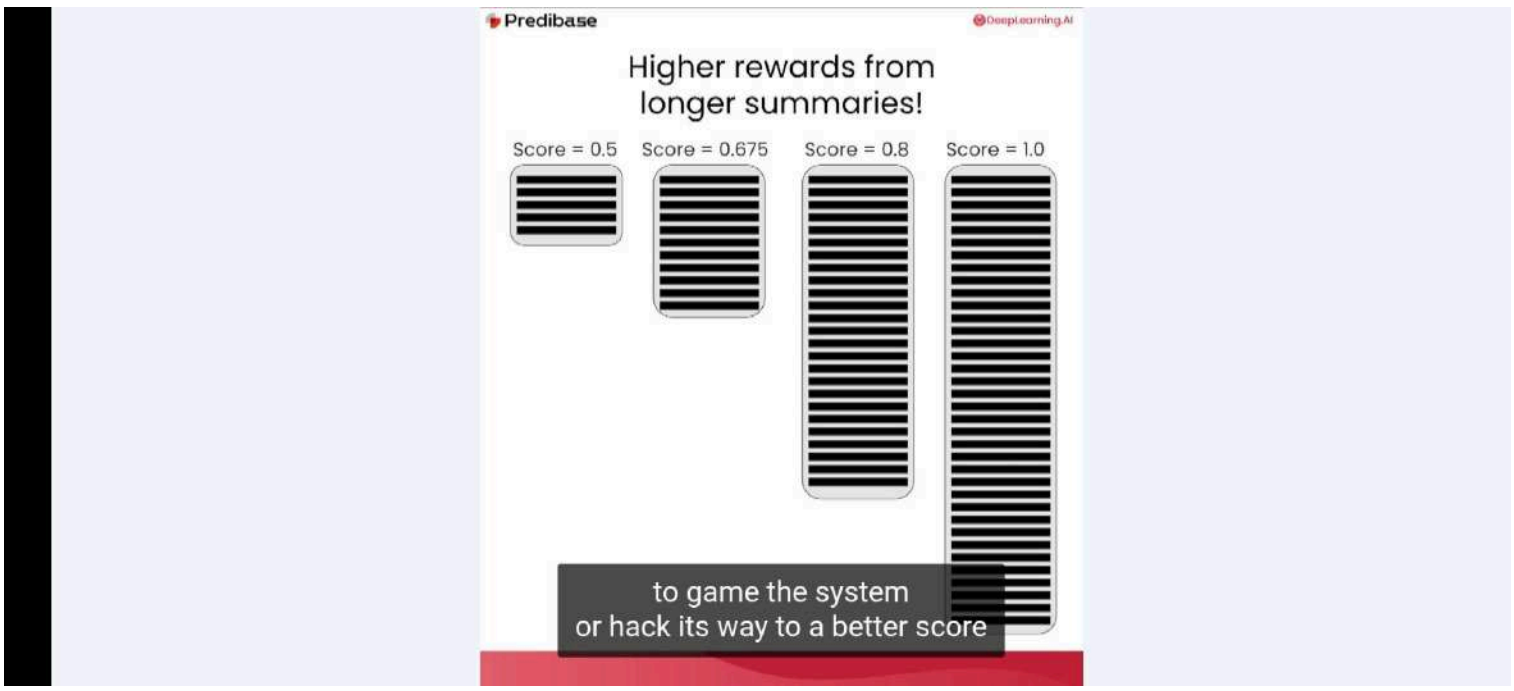
```
[0.625, 0.625, 0.875, 0.75, 0.625, 0.5, 0.625, 0.625]
```

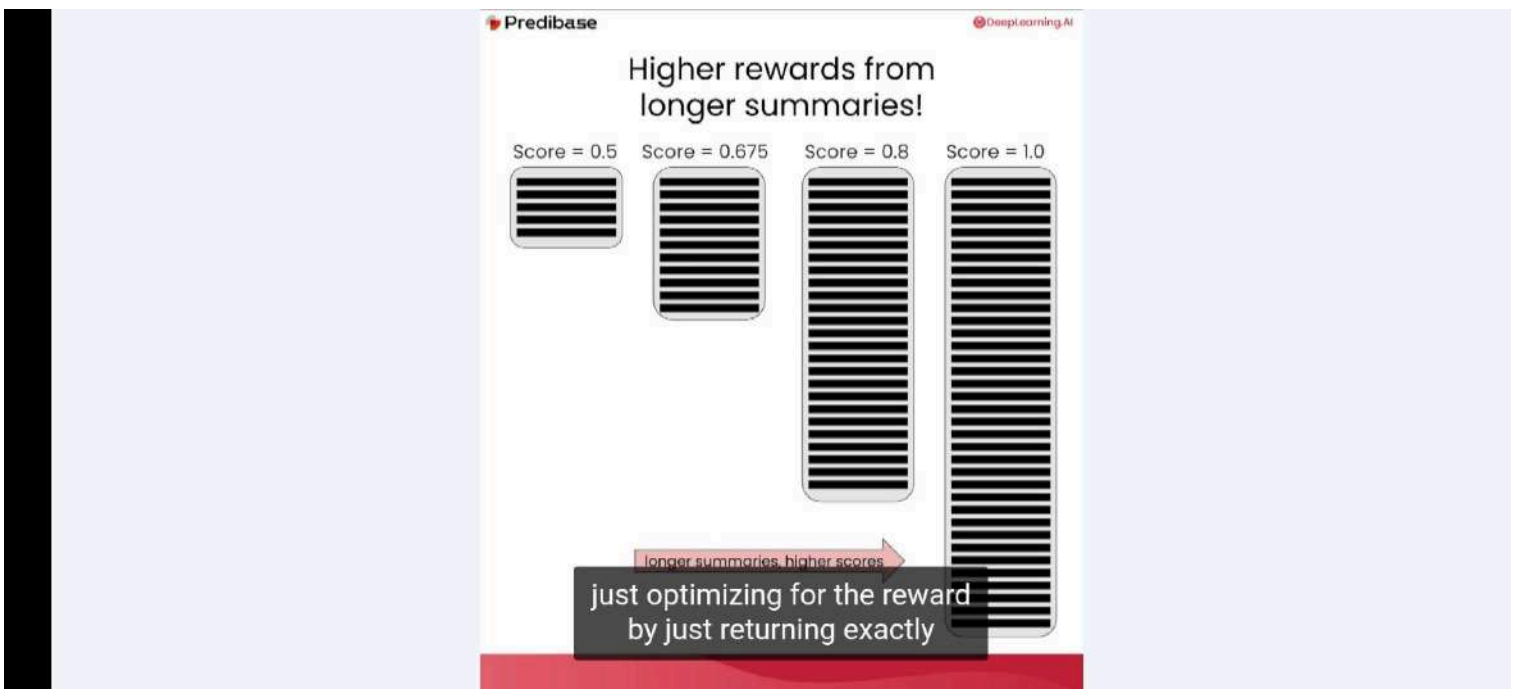
```
transcript_score = quiz_reward(transcript, quiz)
transcript_score
```

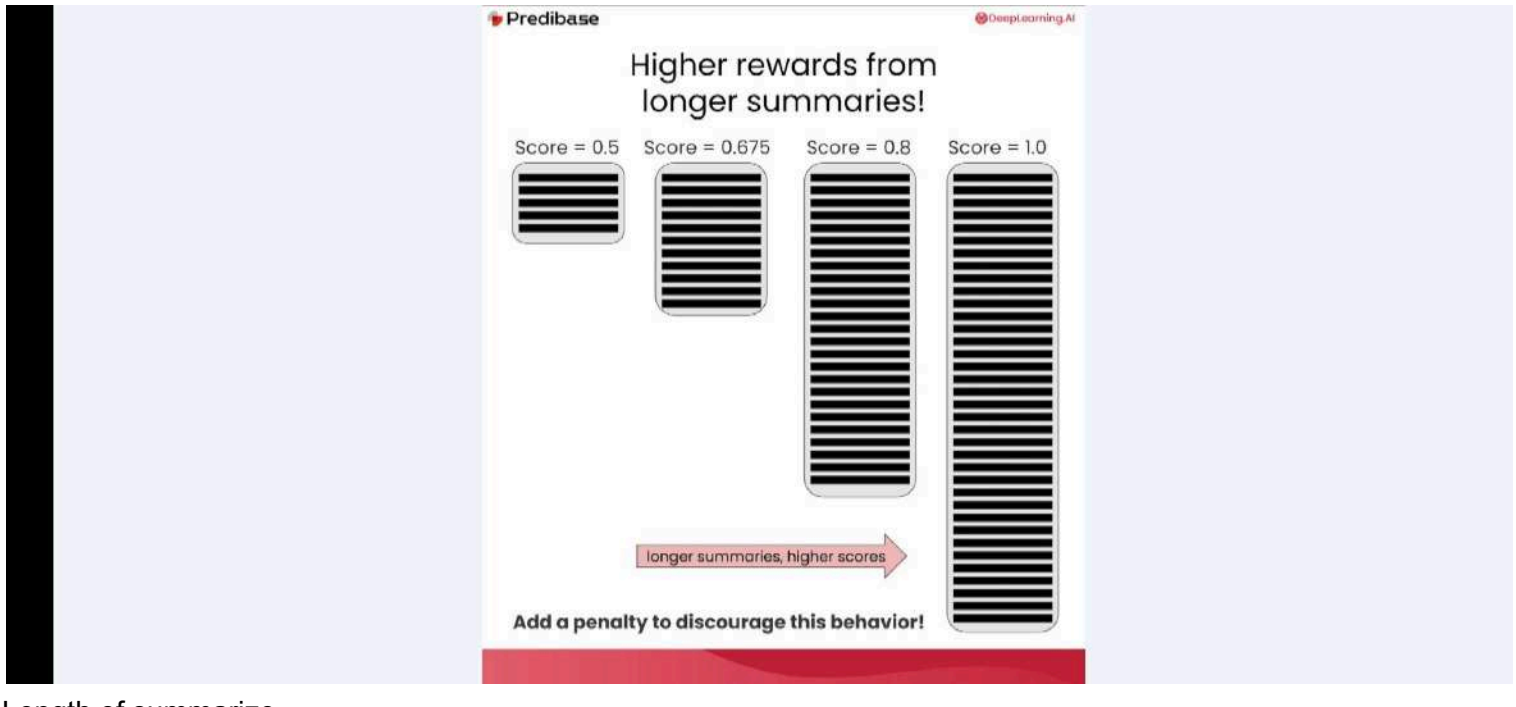
```
1.0
```

And lo and behold, the transcript actually gets a perfect score.

Higher rewards from longer summaries







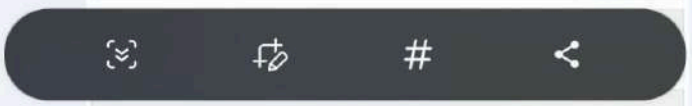
Length of summarize

```
lengths = [len(resp) for resp in responses]
lengths
```

```
[1365, 1150, 941, 1249, 1179, 1261, 1126, 1269]
```

```
len(transcript)
```

```
21810
```




```
lengths = [len(resp) for resp in responses]
lengths
```

```
[1365, 1150, 941, 1249, 1179, 1261, 1126, 1269]
```

```
len(transcript)
```

```
21810
```


the purpose of actually penalize the model for being too long and exceeding

```python
def length_penalty_reward(response: str) -> float:
    length = len(response)
    target_length = 1024
    if length <= target_length:
        return 0.0
    else:
        # Linearly decrease reward as length increases beyond max
        return max(
            (target_length - length) / target_length,
            -10
        )
```

So we expect its value to be negative
called the length penalty reward it takes

```python
def length_penalty_reward(response: str) -> float:
    length = len(response)
    target_length = 1024
    if length <= target_length:
        return 0.0
    else:
        # Linearly decrease reward as length increases beyond max
        return max(
            (target_length - length) / target_length,
            -10
        )
```

the response computes its length
and the number of characters and compares

Length penalty reward

```python
def length_penalty_reward(response: str) -> float:
    length = len(response)
    target_length = 1024
    if length <= target_length:
        return 0.0
    else:
        # Linearly decrease reward as length increases beyond max
        return max(
            (target_length - length) / target_length,
            -10
        )
```

the response computes its length
and the number of characters and compares

```python
def length_penalty_reward(response: str) -> float:
    length = len(response)
    target_length = 1024
    if length <= target_length:
        return 0.0
    else:
        # Linearly decrease reward as length increases beyond max
        return max(
            (target_length - length) / target_length,
            -10
        )
```

we're going to return zero,
which means that there's no penalty.

3:19 / 7:00 • Design a Length Penalty to Enforce Concise S... 1x Auto

```python
transcript_reward = length_penalty_reward(transcript)
transcript_reward
```

```
-10
```

As you can see,
because the transcript is very long,

```
transcript_reward = length_penalty_reward(transcript)
transcript_reward
```

```
-10
```

from generating summaries of that length.


```
transcript_reward = length_penalty_reward(transcript)
transcript_reward
```

```
-10
```


```
lengths = [len(resp) for resp in responses]
length_rewards = [
    length_penalty_reward(resp) for resp in responses
]
print_length_table(lengths, length_rewards)
```

and see how the length penalty reward would affect each of them.


```
transcript_reward = length_penalty_reward(transcript)
transcript_reward
```

```
-10
```

```
lengths = [len(resp) for resp in responses]
length_rewards = [
    length_penalty_reward(resp) for resp in responses
]
print_length_table(lengths, length_rewards)
```


```
+---------+----------+-------------+--------------+
|   Index |   Length |      Reward |    Advantage |
+=========+==========+=============+==============+
|       0 |     1365 | -0.333008   |   -1.64788   |
+---------+----------+-------------+--------------+
|       1 |     1150 | -0.123047   |    0.537436  |
+---------+----------+-------------+--------------+
|       2 |      941 |  0          |    1.81814   |
+---------+----------+-------------+--------------+
|       3 |     1249 | -0.219727   |   -0.468827  |
+---------+----------+-------------+--------------+
|       4 |     1179 | -0.151367   |    0.242672  |
+---------+----------+-------------+--------------+
|       5 |     1261 | -0.231445   |   -0.590799  |
+---------+----------+-------------+--------------+
|       6 |     1126 | -0.0996094  |    0.781379  |
+---------+----------+-------------+--------------+
|       7 |     1269 | -0.239258   |   -0.672113  |
+---------+----------+-------------+--------------+
```

summaries, 941 characters in this case, which is below our target of 1024,

```
transcript_reward = length_penalty_reward(transcript)
transcript_reward
```

```
-10
```

```
lengths = [len(resp) for resp in responses]
length_rewards = [
    length_penalty_reward(resp) for resp in responses
]
print_length_table(lengths, length_rewards)
```

```
+---------+----------+------------+-------------+
|   Index |   Length |     Reward |   Advantage |
+=========+==========+============+=============+
|       0 |     1365 | -0.333008  |   -1.64788  |
+---------+----------+------------+-------------+
|       1 |     1150 | -0.123047  |    0.537436 |
+---------+----------+------------+-------------+
|       2 |      941 | ∅          |    1.81814  |
+---------+----------+------------+-------------+
|       3 |     1249 | -0.219727  |   -0.468827 |
+---------+----------+------------+-------------+
|       4 |     1179 | -0.151367  |    0.242672 |
+---------+----------+------------+-------------+
|       5 |     1261 | -0.231445  |   -0.590799 |
+---------+----------+------------+-------------+
|       6 |     1126 | -0.0996094 |    0.781379 |
+---------+----------+------------+-------------+
|       7 |     1269 | -0.239258  |   -0.672113 |
+---------+----------+------------+-------------+
```

Again, note that even though the summary with 941 characters got a reward of zero,

```
def total_reward(length_reward, quiz_reward):
    return length_reward + quiz_reward
```

So in this case
we're going to take the reward penalty

```python
def total_reward(length_reward, quiz_reward):
    return length_reward + quiz_reward
```

functions together into a final total reward function.

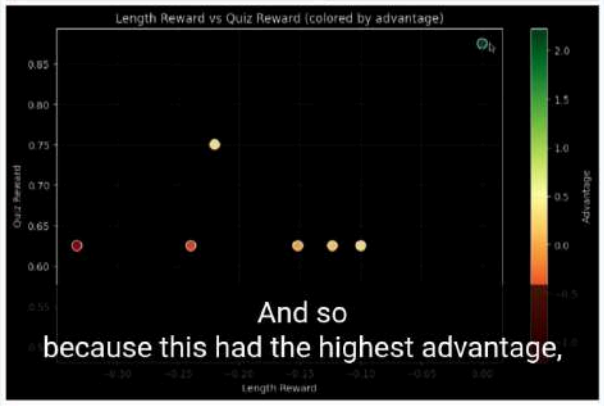
```python
def total_reward(length_reward, quiz_reward):
    return length_reward + quiz_reward

total_rewards = [
    total_reward(length_reward, quiz_reward)
    for length_reward, quiz_reward
    in zip(length_rewards, quiz_rewards)
]
```

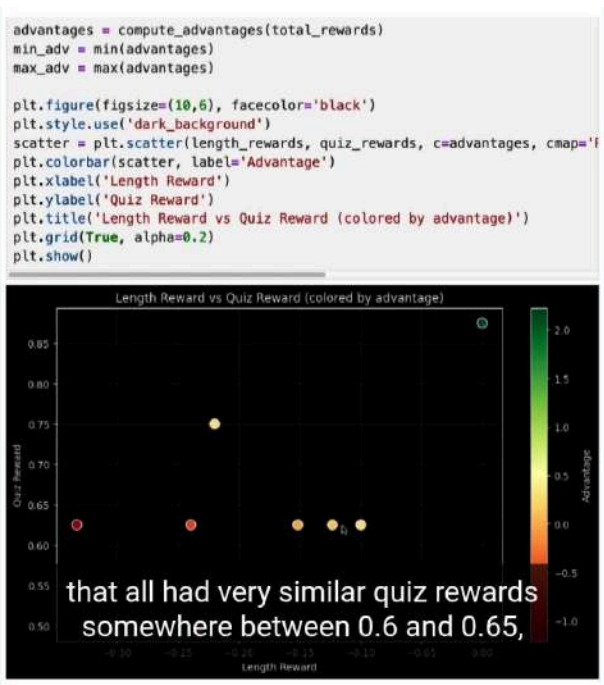
Now we can go ahead and compute these.

Advantages calculate

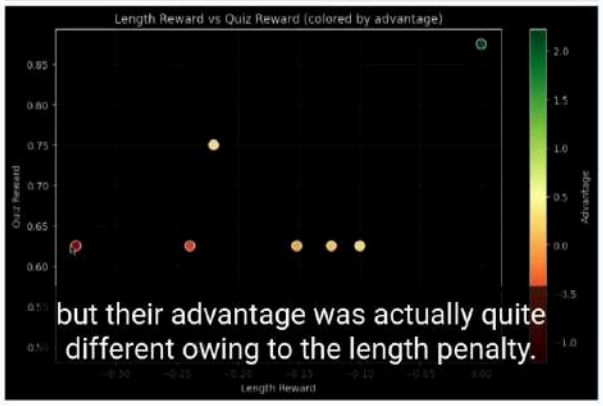
```
advantages = compute_advantages(total_rewards)
min_adv = min(advantages)
max_adv = max(advantages)

plt.figure(figsize=(10,6), facecolor='black')
plt.style.use('dark_background')
scatter = plt.scatter(length_rewards, quiz_rewards, c=advantages, cmap='
plt.colorbar(scatter, label='Advantage')
plt.xlabel('Length Reward')
plt.ylabel('Quiz Reward')
plt.title('Length Reward vs Quiz Reward (colored by advantage)')
plt.grid(True, alpha=0.2)
plt.show()
```

relationship between the length reward and the quiz reward.

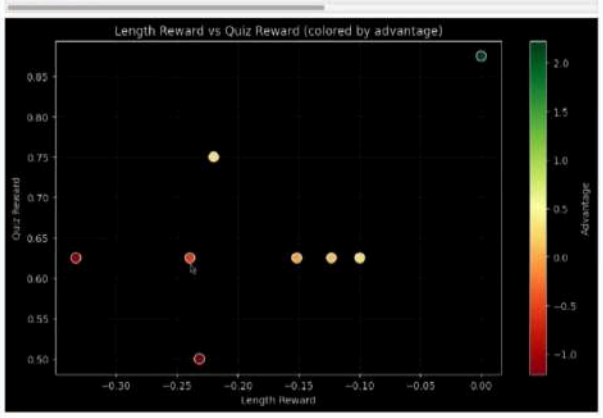




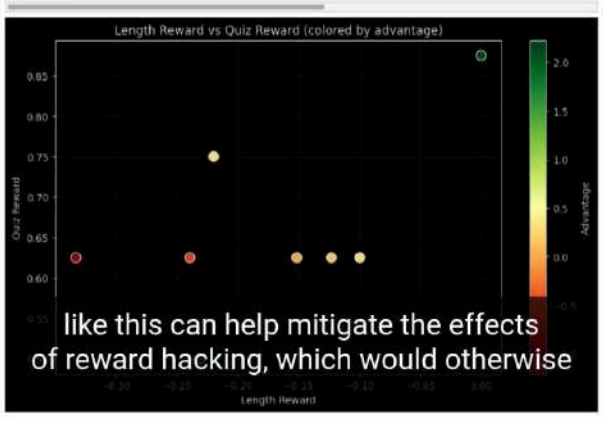
```
advantages = compute_advantages(total_rewards)
min_adv = min(advantages)
max_adv = max(advantages)

plt.figure(figsize=(10,6), facecolor='black')
plt.style.use('dark_background')
scatter = plt.scatter(length_rewards, quiz_rewards, c=advantages, cmap='
plt.colorbar(scatter, label='Advantage')
plt.xlabel('Length Reward')
plt.ylabel('Quiz Reward')
plt.title('Length Reward vs Quiz Reward (colored by advantage)')
plt.grid(True, alpha=0.2)
plt.show()
```



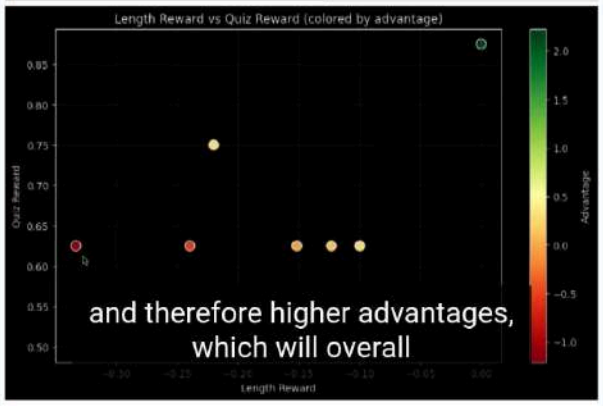

```
advantages = compute_advantages(total_rewards)
min_adv = min(advantages)
max_adv = max(advantages)

plt.figure(figsize=(10,6), facecolor='black')
plt.style.use('dark_background')
scatter = plt.scatter(length_rewards, quiz_rewards, c=advantages, cmap='
plt.colorbar(scatter, label='Advantage')
plt.xlabel('Length Reward')
plt.ylabel('Quiz Reward')
plt.title('Length Reward vs Quiz Reward (colored by advantage)')
plt.grid(True, alpha=0.2)
plt.show()
```

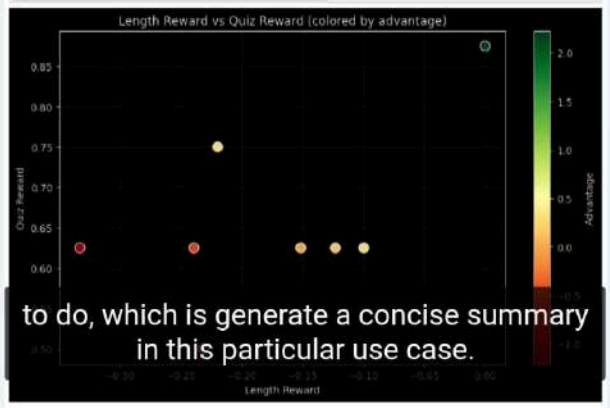

```
advantages = compute_advantages(total_rewards)
min_adv = min(advantages)
max_adv = max(advantages)

plt.figure(figsize=(10,6), facecolor='black')
plt.style.use('dark_background')
scatter = plt.scatter(length_rewards, quiz_rewards, c=advantages, cmap='
plt.colorbar(scatter, label='Advantage')
plt.xlabel('Length Reward')
plt.ylabel('Quiz Reward')
plt.title('Length Reward vs Quiz Reward (colored by advantage)')
plt.grid(True, alpha=0.2)
plt.show()
```

```
advantages = compute_advantages(total_rewards)
min_adv = min(advantages)
max_adv = max(advantages)

plt.figure(figsize=(10,6), facecolor='black')
plt.style.use('dark_background')
scatter = plt.scatter(length_rewards, quiz_rewards, c=advantages, cmap='
plt.colorbar(scatter, label='Advantage')
plt.xlabel('Length Reward')
plt.ylabel('Quiz Reward')
plt.title('Length Reward vs Quiz Reward (colored by advantage)')
plt.grid(True, alpha=0.2)
plt.show()
```

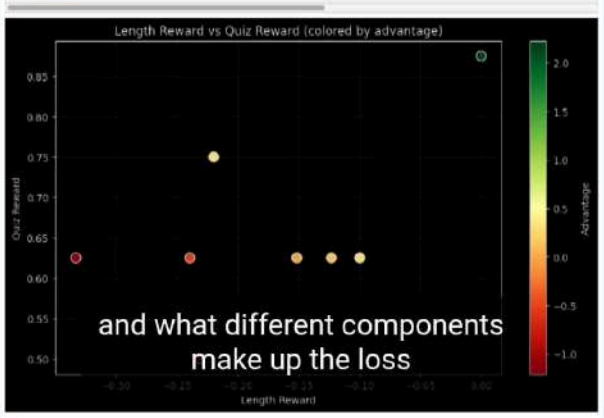



```
advantages = compute_advantages(total_rewards)
min_adv = min(advantages)
max_adv = max(advantages)

plt.figure(figsize=(10,6), facecolor='black')
plt.style.use('dark_background')
scatter = plt.scatter(length_rewards, quiz_rewards, c=advantages, cmap='
plt.colorbar(scatter, label='Advantage')
plt.xlabel('Length Reward')
plt.ylabel('Quiz Reward')
plt.title('Length Reward vs Quiz Reward (colored by advantage)')
plt.grid(True, alpha=0.2)
plt.show()
```

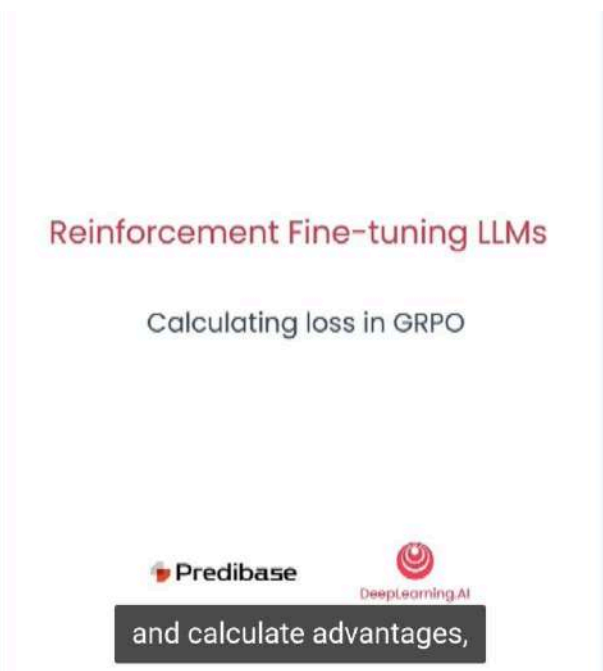



```
advantages = compute_advantages(total_rewards)
min_adv = min(advantages)
max_adv = max(advantages)

plt.figure(figsize=(10,6), facecolor='black')
plt.style.use('dark_background')
scatter = plt.scatter(length_rewards, quiz_rewards, c=advantages, cmap='
plt.colorbar(scatter, label='Advantage')
plt.xlabel('Length Reward')
plt.ylabel('Quiz Reward')
plt.title('Length Reward vs Quiz Reward (colored by advantage)')
plt.grid(True, alpha=0.2)
plt.show()
```

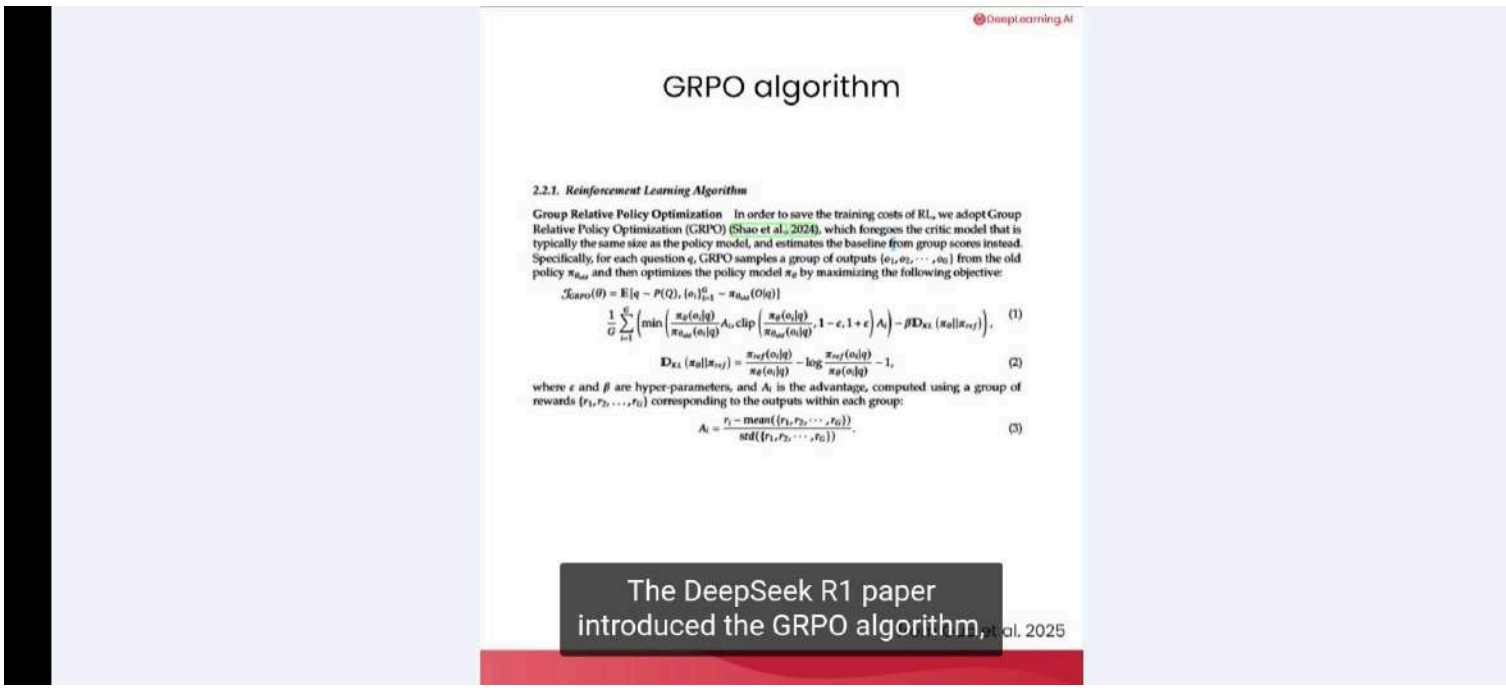

```
advantages = compute_advantages(total_rewards)
min_adv = min(advantages)
max_adv = max(advantages)

plt.figure(figsize=(10,6), facecolor='black')
plt.style.use('dark_background')
scatter = plt.scatter(length_rewards, quiz_rewards, c=advantages, cmap='
plt.colorbar(scatter, label='Advantage')
plt.xlabel('Length Reward')
plt.ylabel('Quiz Reward')
plt.title('Length Reward vs Quiz Reward (colored by advantage)')
plt.grid(True, alpha=0.2)
plt.show()
```

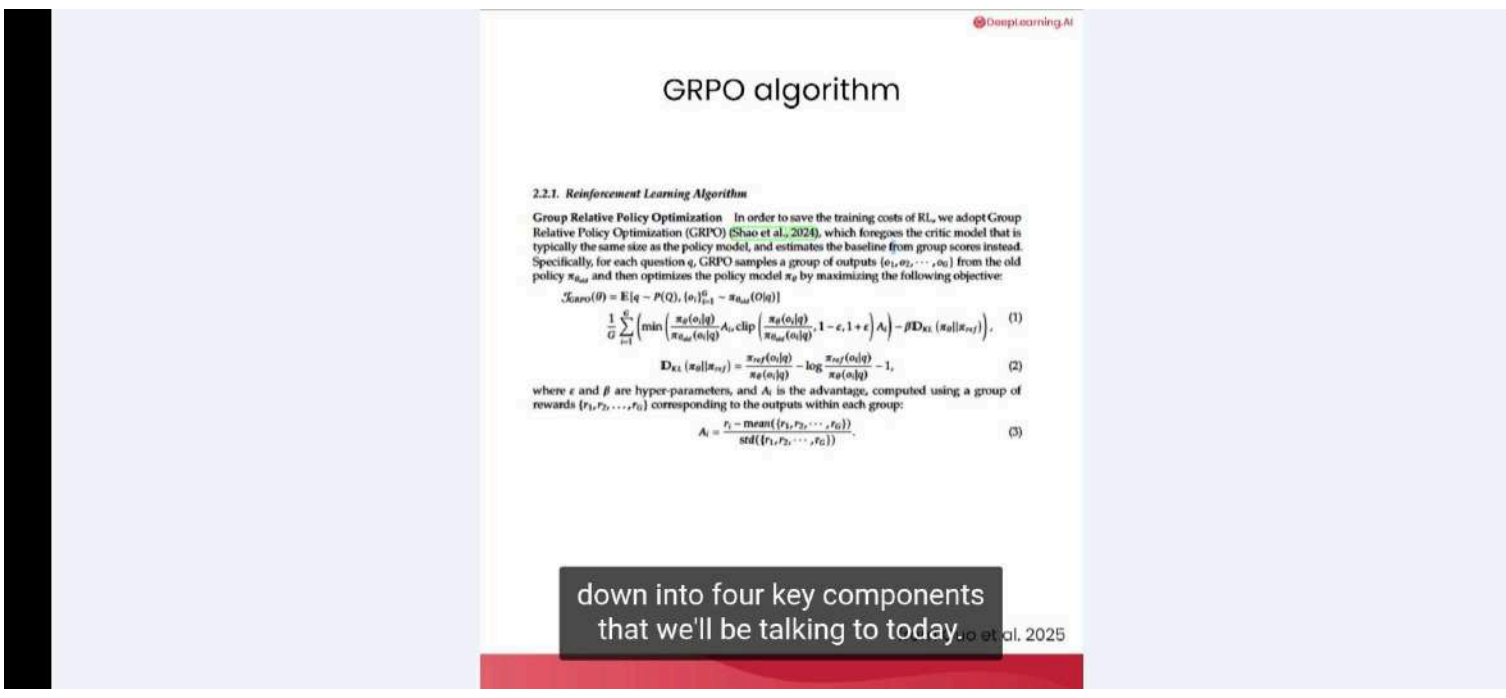



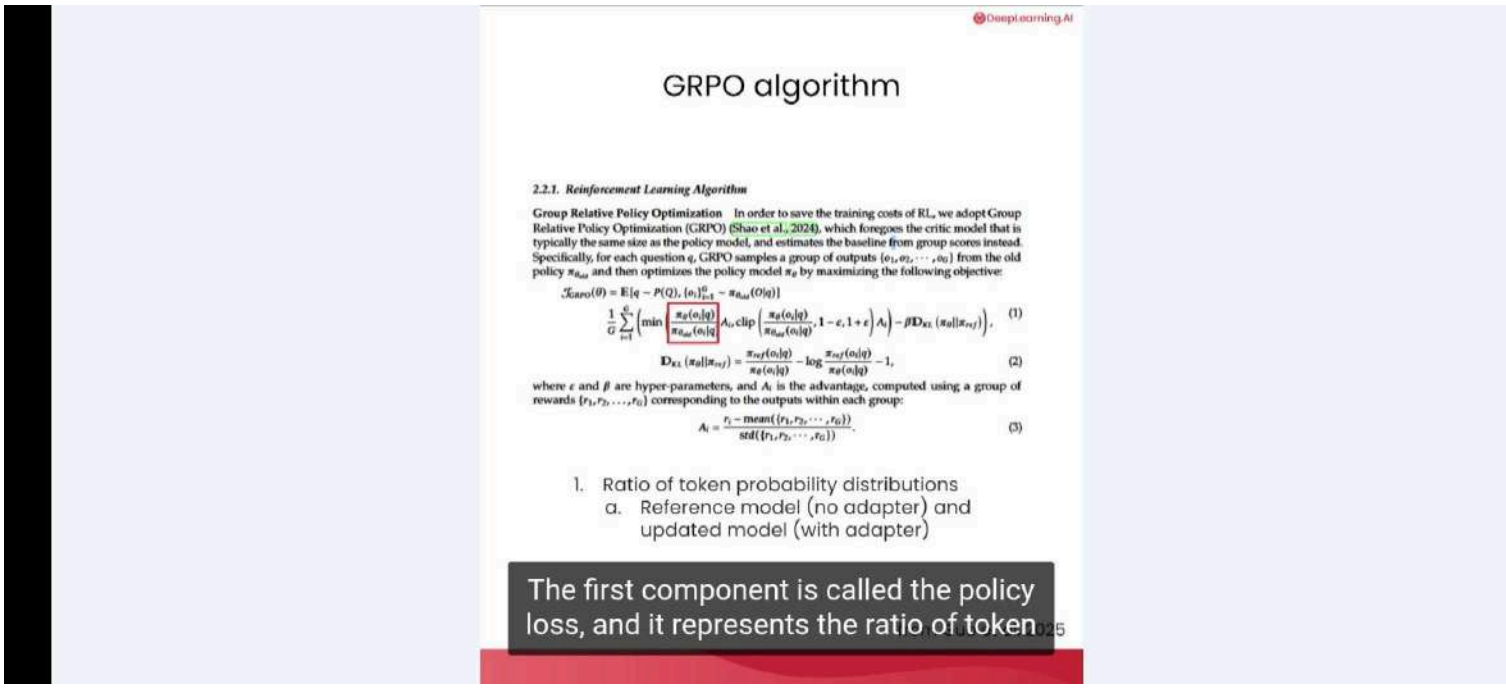
Calculating loss in GRPO

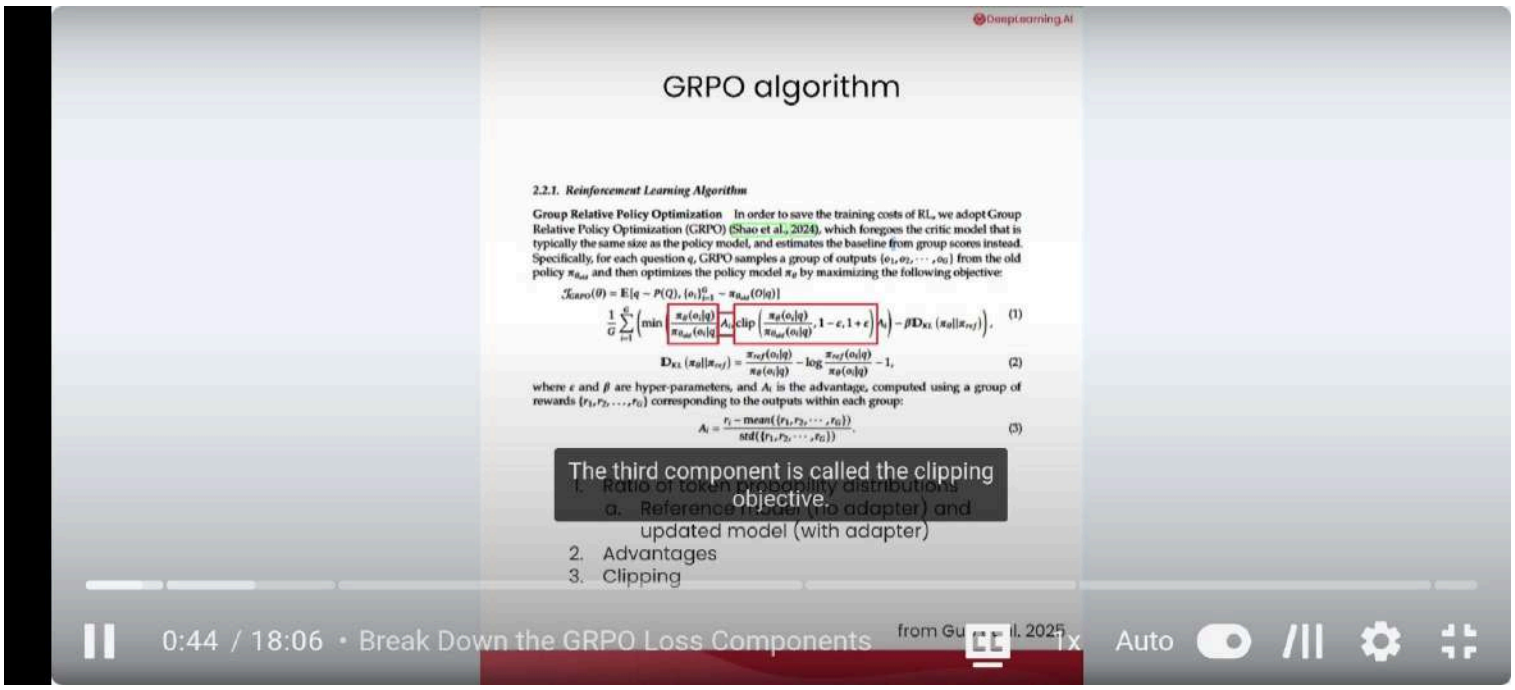


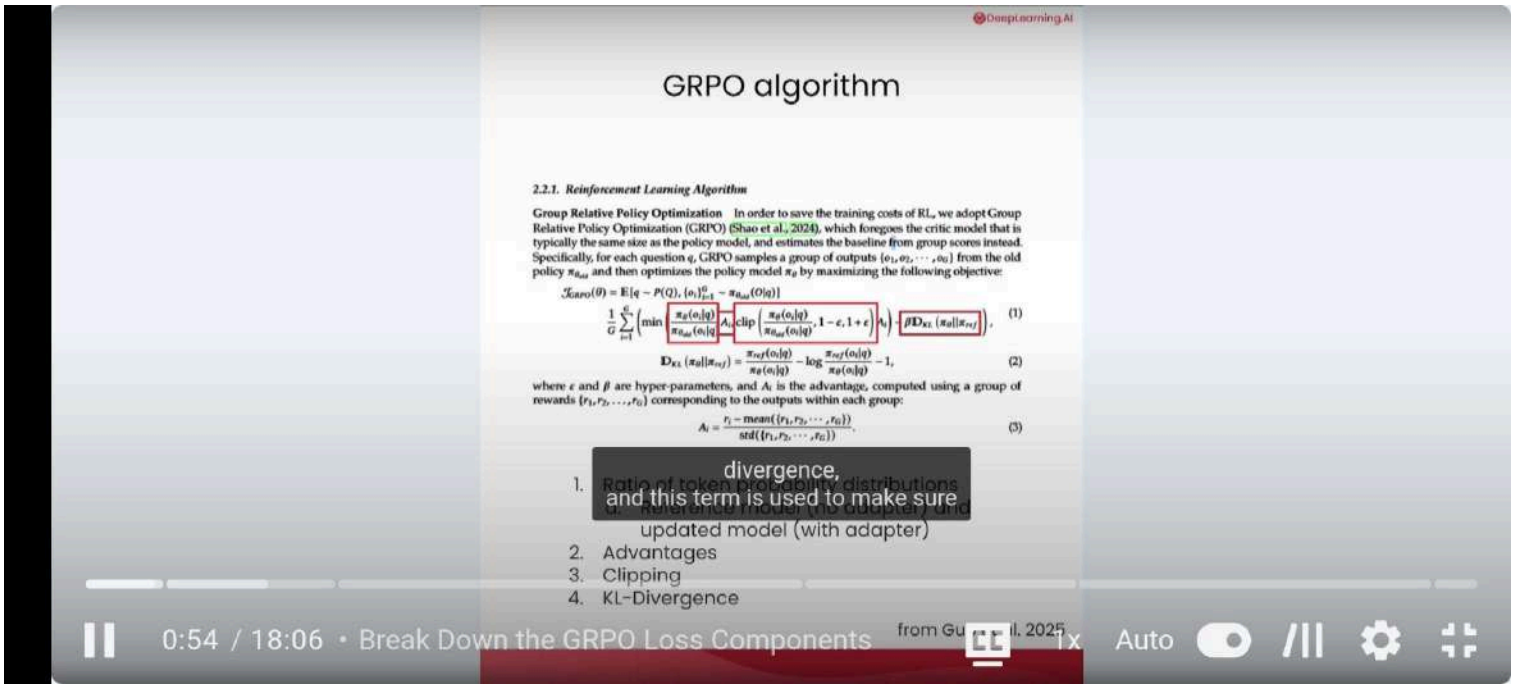
GRPO algorithm

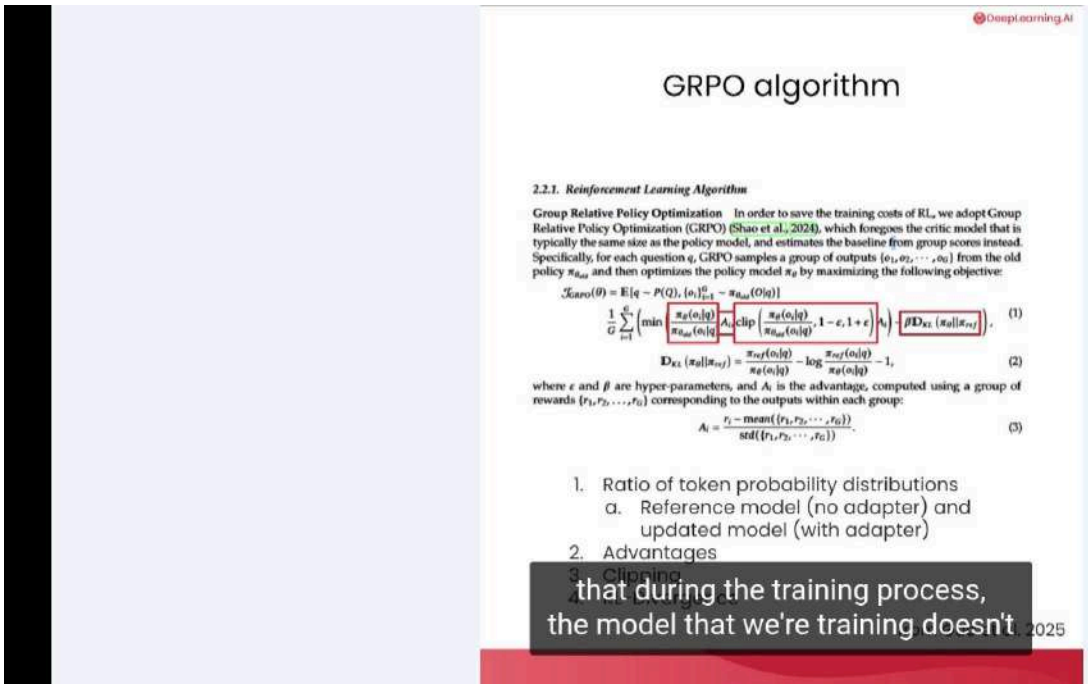


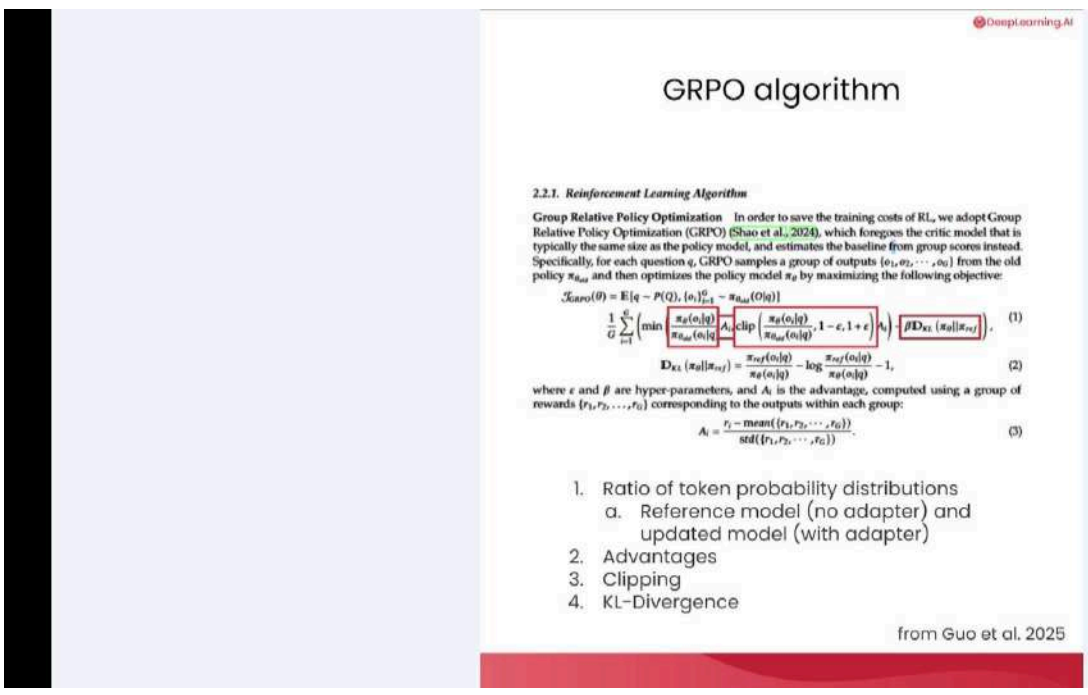




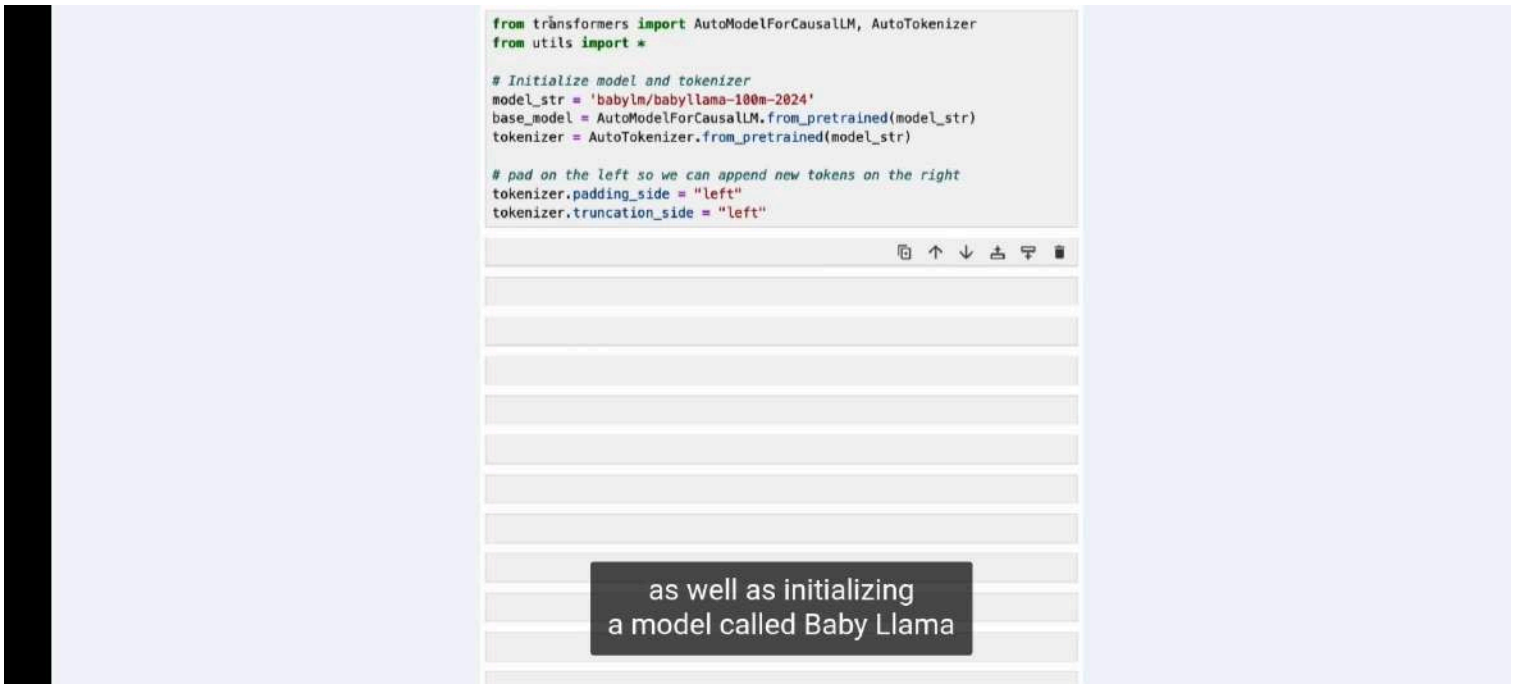

DeepLearning.AI
GRPO algorithm
2.2.1. Reinforcement Learning Algorithm
Group Relative Policy Optimization In order to save the training costs of RL, we adopt Group Relative Policy Optimization (GRPO) (Shao et al., 2024), which foregoes the critic model that is typically the same size as the policy model, and estimates the baseline from group scores instead. Specifically, for each question q, GRPO samples a group of outputs $\{o_1, o_2, \cdots, o_G\}$ from the old policy $\pi_{\theta_{old}}$ and then optimizes the policy model π_θ by maximizing the following objective:
$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^{G} \sim \pi_{\theta_{old}}(O|q)]$
$\frac{1}{G}\sum_{i=1}^{G}\left(\min\left[\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}A_i, \text{clip}\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1-\varepsilon, 1+\varepsilon\right)A_i\right] - \beta\mathbb{D}_{KL}\left(\pi_\theta||\pi_{ref}\right)\right),$ (1)
$\mathbb{D}_{KL}\left(\pi_\theta||\pi_{ref}\right) = \frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - \log\frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - 1,$ (2)
where ε and β are hyper-parameters, and A_i is the advantage, computed using a group of rewards $\{r_1, r_2, \ldots, r_G\}$ corresponding to the outputs within each group:
$A_i = \frac{r_i - \text{mean}(\{r_1, r_2, \cdots, r_G\})}{\text{std}(\{r_1, r_2, \cdots, r_G\})}.$ (3)
The third component is called the clipping objective.
1. Ratio of token probability distributions
a. Reference model (no adapter) and updated model (with adapter)
2. Advantages
3. Clipping
0:44 / 18:06 • Break Down the GRPO Loss Components
from Gu... l. 2025
1x
Auto


DeepLearning.AI
GRPO algorithm
2.2.1. Reinforcement Learning Algorithm
Group Relative Policy Optimization In order to save the training costs of RL, we adopt Group Relative Policy Optimization (GRPO) (Shao et al., 2024), which foregoes the critic model that is typically the same size as the policy model, and estimates the baseline from group scores instead. Specifically, for each question q, GRPO samples a group of outputs $\{o_1, o_2, \cdots, o_G\}$ from the old policy $\pi_{\theta_{old}}$ and then optimizes the policy model π_θ by maximizing the following objective:
$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^{G} \sim \pi_{\theta_{old}}(O|q)]$
$\frac{1}{G}\sum_{i=1}^{G}\left(\min\left[\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}A_i, \text{clip}\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1-\varepsilon, 1+\varepsilon\right)A_i\right] - \beta\mathbb{D}_{KL}\left(\pi_\theta||\pi_{ref}\right)\right),$ (1)
$\mathbb{D}_{KL}\left(\pi_\theta||\pi_{ref}\right) = \frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - \log\frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - 1,$ (2)
where ε and β are hyper-parameters, and A_i is the advantage, computed using a group of rewards $\{r_1, r_2, \ldots, r_G\}$ corresponding to the outputs within each group:
$A_i = \frac{r_i - \text{mean}(\{r_1, r_2, \cdots, r_G\})}{\text{std}(\{r_1, r_2, \cdots, r_G\})}.$ (3)
divergence,
and this term is used to make sure
1. Ratio of token probability distributions
a. Reference model (no adapter) and updated model (with adapter)
2. Advantages
3. Clipping
4. KL-Divergence
0:54 / 18:06 • Break Down the GRPO Loss Components
from Gu... l. 2025
1x
Auto





Load model and tokenizer



Base model

```
print(base_model)
```

```
LlamaForCausalLM(
  (model): LlamaModel(
    (embed_tokens): Embedding(16000, 512, padding_idx=0)
    (layers): ModuleList(
      (0-15): 16 x LlamaDecoderLayer(
        (self_attn): LlamaAttention(
          (q_proj): Linear(in_features=512, out_features=512, bias=Fals
e)
          (k_proj): Linear(in_features=512, out_features=512, bias=Fals
e)
          (v_proj): Linear(in_features=512, out_features=512, bias=Fals
e)
          (o_proj): Linear(in_features=512, out_features=512, bias=Fals
e)
        )
        (mlp): LlamaMLP(
          (gate_proj): Linear(in_features=512, out_features=1024, bias=F
alse)
          (up_proj): Linear(in_features=512, out_features=1024, bias=Fal
se)
          (down_proj): Linear(in_features=1024, out_features=512, bias=F
alse)
          (act_fn): SiLU()
        )
        (input_layernorm): LlamaRMSNorm((512,), eps=1e-06)
        (post_attention_layernorm): LlamaRMSNorm((512,), eps=1e-06)
      )
    )
    (norm): LlamaRMSNorm((512,), eps=1e-06)
    (rotary_emb): LlamaRotaryEmbedding()
  )
  (lm_head): Linear(in_features=512, out_features=16000, bias=False)
)
```

So here you can see that it's a Llama model that has the embedding layer,

```
print(base_model)
```

```
LlamaForCausalLM(
  (model): LlamaModel(
    (embed_tokens): Embedding(16000, 512, padding_idx=0)
    (layers): ModuleList(
      (0-15): 16 x LlamaDecoderLayer(
        (self_attn): LlamaAttention(
          (q_proj): Linear(in_features=512, out_features=512, bias=Fals
e)
          (k_proj): Linear(in_features=512, out_features=512, bias=Fals
e)
          (v_proj): Linear(in_features=512, out_features=512, bias=Fals
e)
          (o_proj): Linear(in_features=512, out_features=512, bias=Fals
e)
        )
        (mlp): LlamaMLP(
          (gate_proj): Linear(in_features=512, out_features=1024, bias=F
alse)
          (up_proj): Linear(in_features=512, out_features=1024, bias=Fal
se)
          (down_proj): Linear(in_features=1024, out_features=512, bias=F
alse)
          (act_fn): SiLU()
        )
        (input_layernorm): LlamaRMSNorm((512,), eps=1e-06)
        (post_attention_layernorm): LlamaRMSNorm((512,), eps=1e-06)
      )
    )
    (norm): LlamaRMSNorm((512,), eps=1e-06)
    (rotary_emb): LlamaRotaryEmbedding()
  )
  (lm_head): Linear(in_features=512, out_features=16000, bias=False)
)
```

to generate some tokens.

```
  (lm_head): Linear(in_features=512, out_features=16000, bias=False)
)
```

```
prompt = "The quick brown fox jumped over the "

# Tokenize the prompt
input_ids = tokenizer(prompt, return_tensors="pt")
print(input_ids)

# Generate next 2 tokens
with torch.no_grad():
    outputs = base_model.generate(
        **input_ids,
        max_new_tokens=2,
        pad_token_id=tokenizer.pad_token_id
    )

# Decode the generated text
generated_text = tokenizer.decode(
    outputs[0], skip_special_tokens=True
)
generated_portion = generated_text[len(prompt):]
print(f"Generated text: {prompt}\033[94m{generated_portion}\033[0m")
```

jumped over it the".

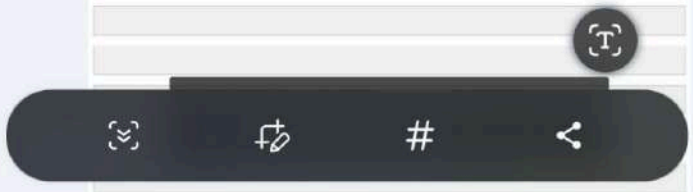
```
  (lm_head): Linear(in_features=512, out_features=16000, bias=False)
)
```

```
prompt = "The quick brown fox jumped over the "

# Tokenize the prompt
input_ids = tokenizer(prompt, return_tensors="pt")
print(input_ids)

# Generate next 2 tokens
with torch.no_grad():
    outputs = base_model.generate(
        **input_ids,
        max_new_tokens=2,
        pad_token_id=tokenizer.pad_token_id
    )

# Decode the generated text
generated_text = tokenizer.decode(
    outputs[0], skip_special_tokens=True
)
generated_portion = generated_text[len(prompt):]
print(f"Generated text: {prompt}\033[94m{generated_portion}\033[0m")
```

Once we have tokens,
we can pass this into the model


```
  (lm_head): Linear(in_features=512, out_features=16000, bias=False)
)
```

```
prompt = "The quick brown fox jumped over the "

# Tokenize the prompt
input_ids = tokenizer(prompt, return_tensors="pt")
print(input_ids)

# Generate next 2 tokens
with torch.no_grad():
    outputs = base_model.generate(
        **input_ids,
        max_new_tokens=2,
        pad_token_id=tokenizer.pad_token_id
    )

# Decode the generated text
generated_text = tokenizer.decode(
    outputs[0], skip_special_tokens=True
)
generated_portion = generated_text[len(prompt):]
print(f"Generated text: {prompt}\033[94m{generated_portion}\033[0m")
```

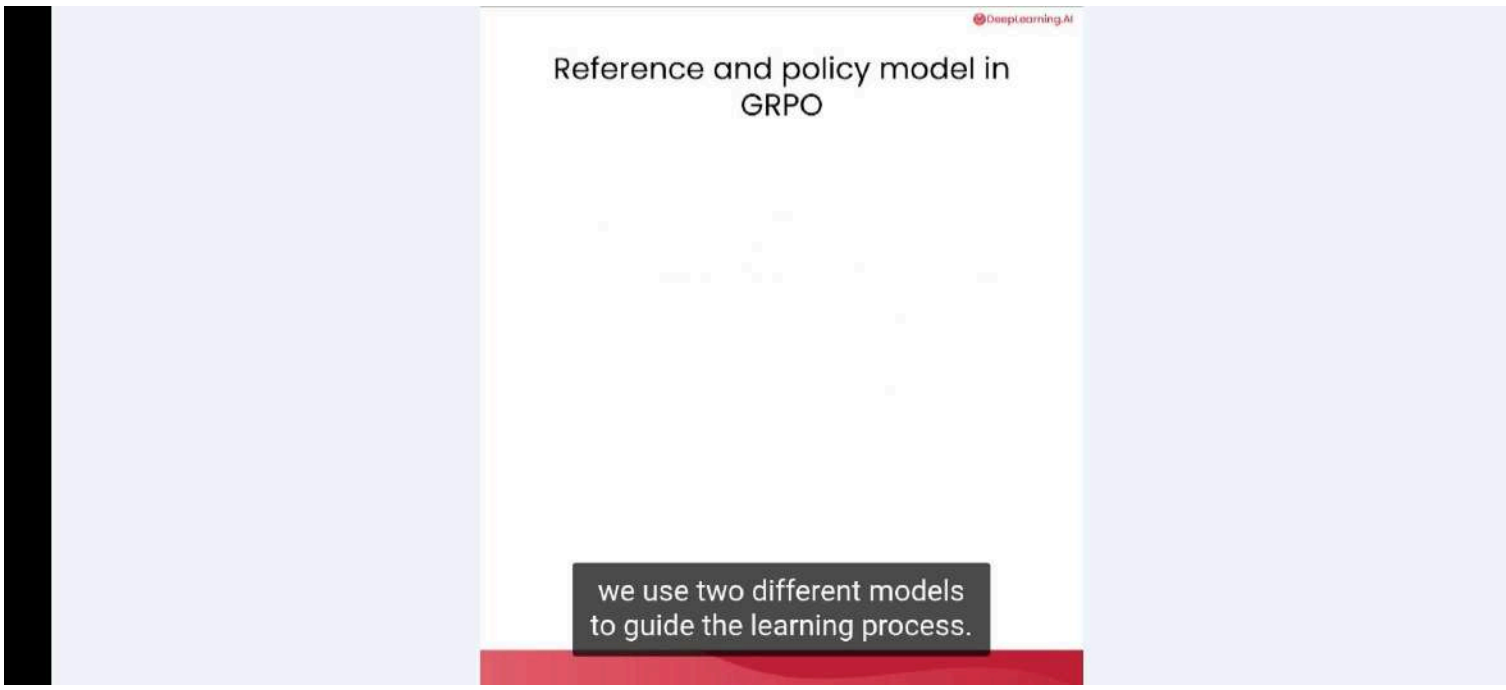

```
)
```

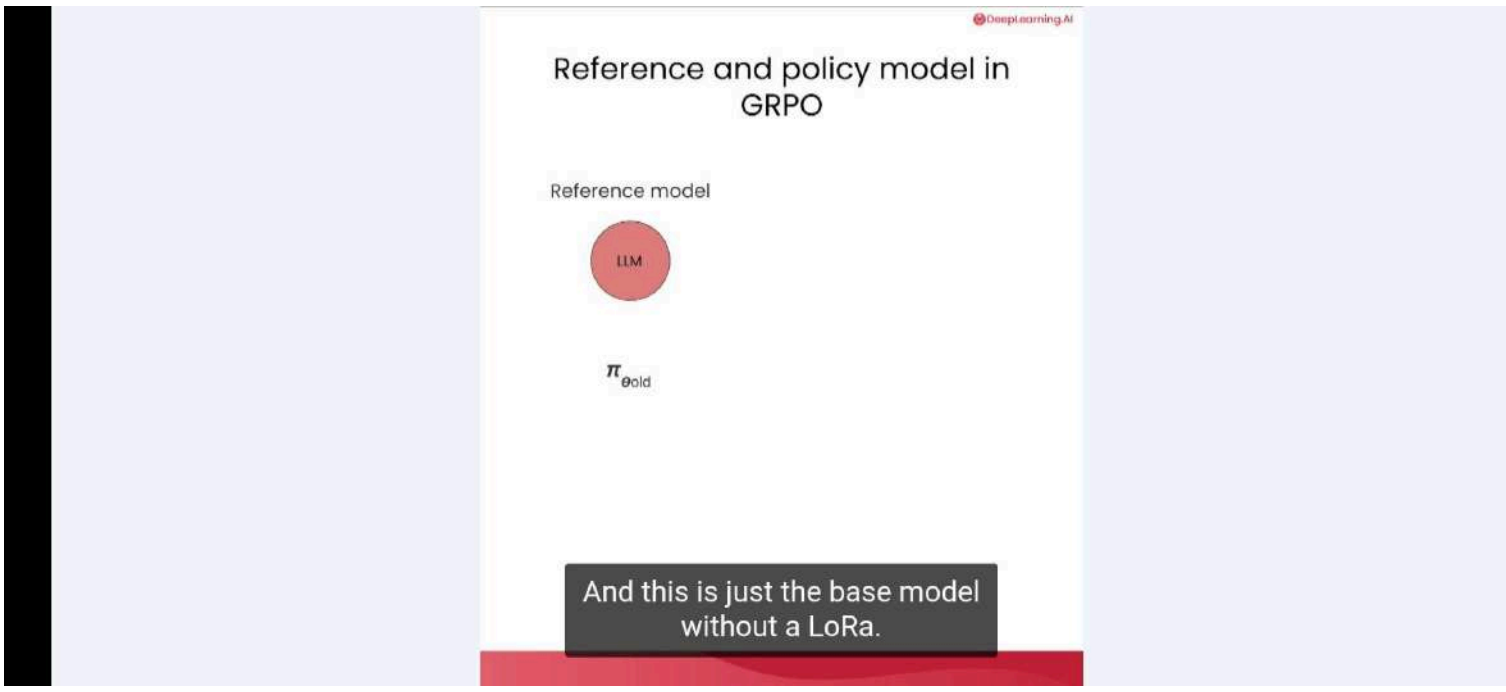
```
prompt = "The quick brown fox jumped over the "

# Tokenize the prompt
input_ids = tokenizer(prompt, return_tensors="pt")
print(input_ids)

# Generate next 2 tokens
with torch.no_grad():
    outputs = base_model.generate(
        **input_ids,
        max_new_tokens=2,
        pad_token_id=tokenizer.pad_token_id
    )

# Decode the generated text
generated_text = tokenizer.decode(
    outputs[0], skip_special_tokens=True
)
generated_portion = generated_text[len(prompt):]
print(f"Generated text: {prompt}\033[94m{generated_portion}\033[0m")
```

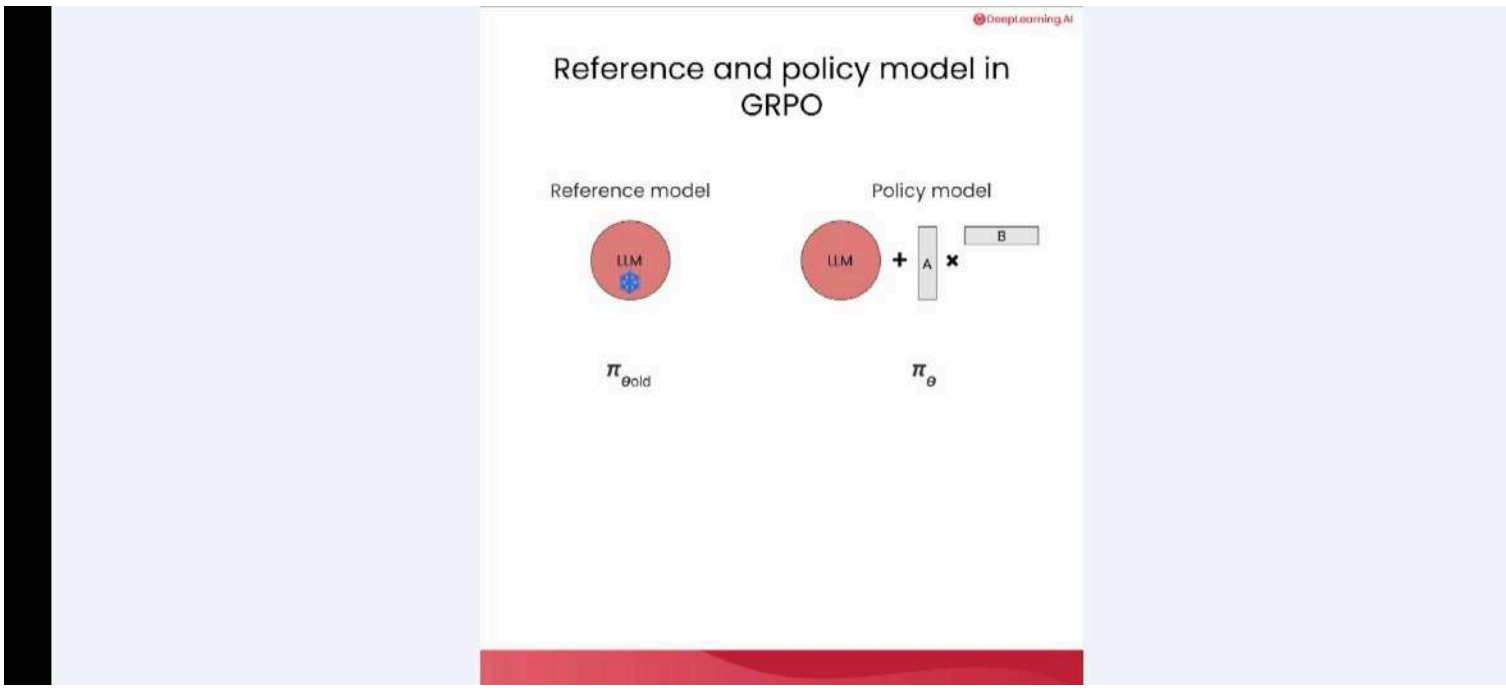

```
{'input_ids': tensor([[1086, 1617, 2837, 6114, 4749,  551,  196,  15
4]]), 'attention_mask': tensor([[1, 1, 1, 1, 1, 1, 1, 1]])}
Generated text: The quick brown fox jumped over the icy ground
```

And what you can see here
is that the model predicted

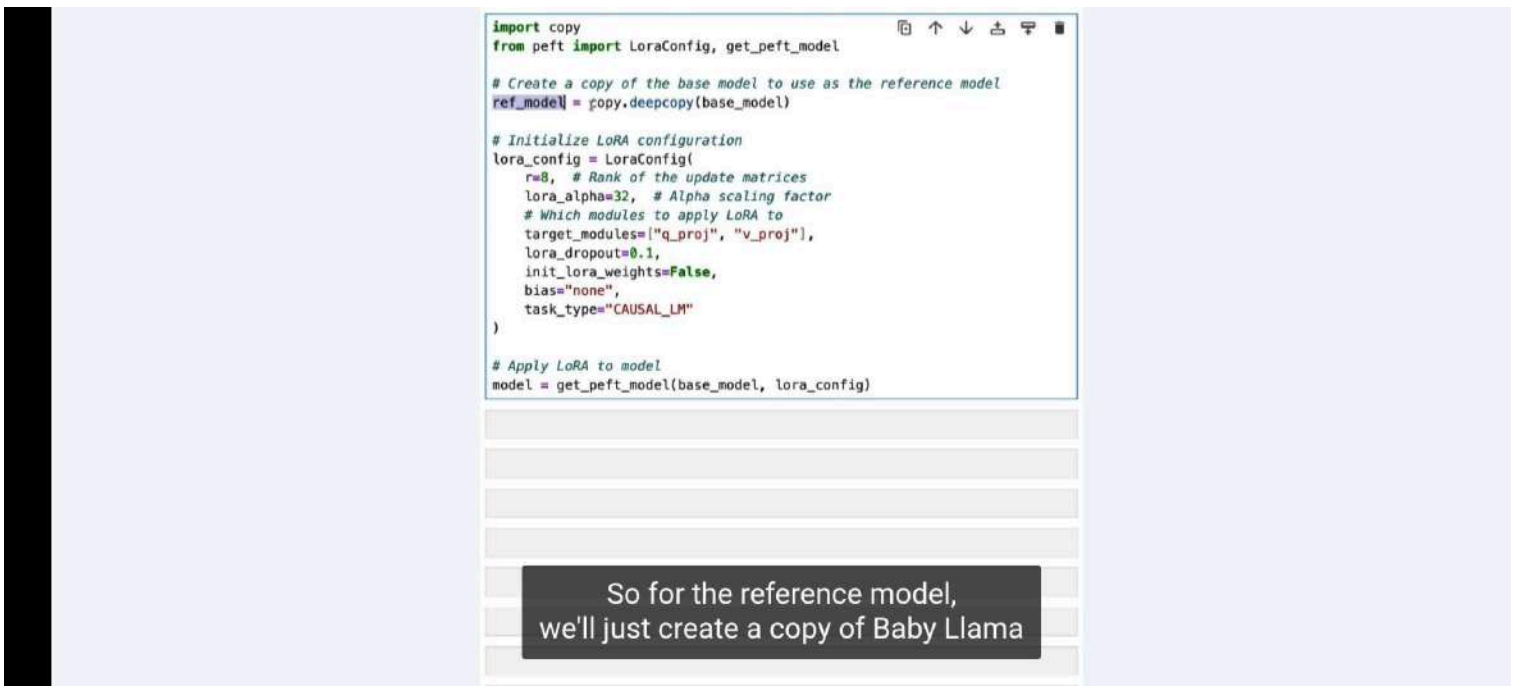
Reference and policy model in GRPO



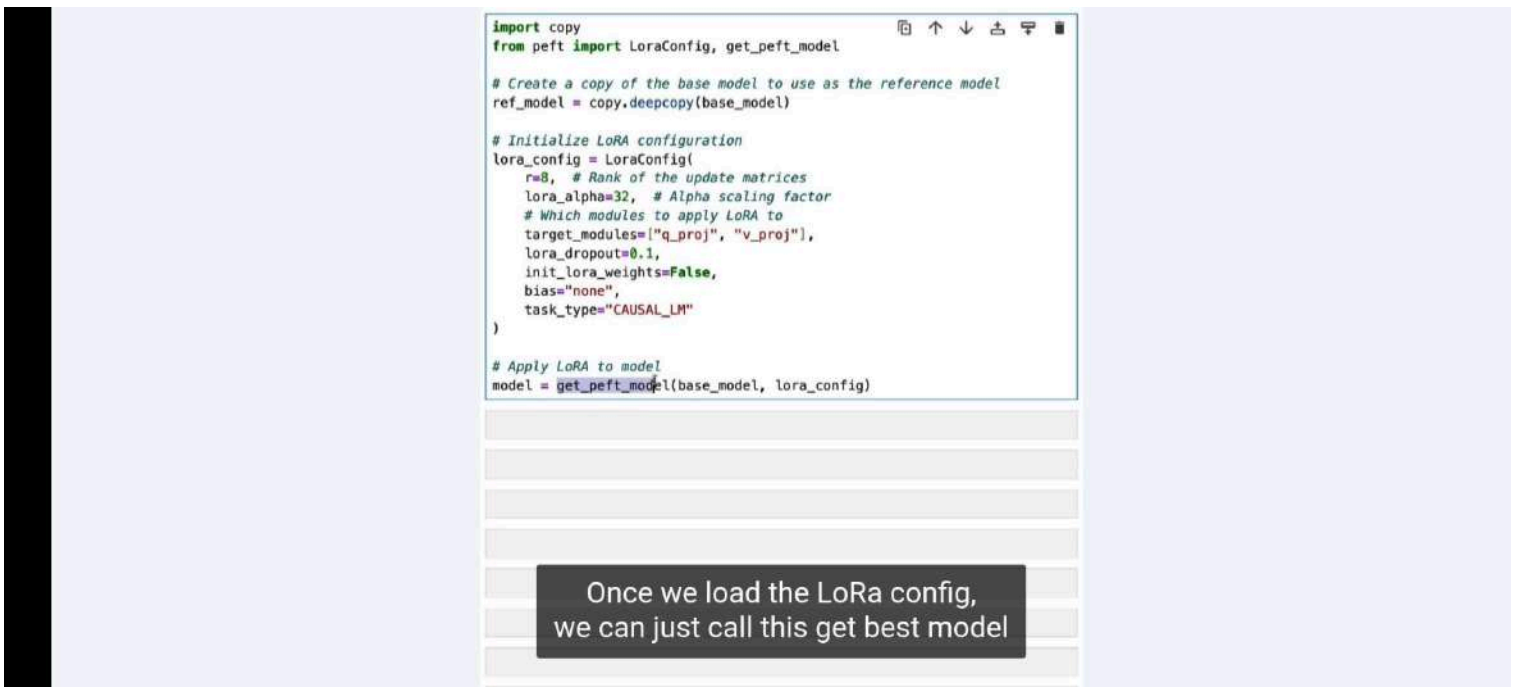




Create reference model



Apply LoRA



Print model

```
    target_modules=["q_proj", "v_proj"],
    lora_dropout=0.1,
    init_lora_weights=False,
    bias="none",
    task_type="CAUSAL_LM"
)

# Apply LoRA to model
model = get_peft_model(base_model, lora_config)
```

```
print(model)
```

```
PeftModelForCausalLM(
  (base_model): LoraModel(
    (model): LlamaForCausalLM(
      (model): LlamaModel(
        (embed_tokens): Embedding(16000, 512, padding_idx=0)
        (layers): ModuleList(
          (0-15): 16 x LlamaDecoderLayer(
            (self_attn): LlamaAttention(
              (q_proj): lora.Linear(
                (base_layer): Linear(in_features=512, out_features=512,
bias=False)
                (lora_dropout): ModuleDict(
                  (default): Dropout(p=0.1, inplace=False)
                )
                (lora_A): ModuleDict(
                  (default): Linear(in_features=512, out_features=8, bia
s=False)
                )
                (lora_B): ModuleDict(
                  (default): Linear(in_features=8, out_features=512, bia
s=False)
                )
                (lora_embedding_A): ParameterDict()
                (lora_embedding_B): ParameterDict()
                (lora_magnitude_vector): ModuleDict()
              )
              (k_proj): Linear(in_features=512, out_features=512, bias=F
alse)
              (v_proj): lora.Linear(
                (base_layer): Linear(in_features=512, out_features=512,
bias=False)
                (lora_dropout): ModuleDict(
                  (default): Dropout(p=0.1, inplace=False)
                )
                (lora_A): ModuleDict(
                  (default): Linear(in_features=512, out_features=8, bia
s=False)
                )
                (lora_B): ModuleDict(
                  (default): Linear(in_features=8, out_features=512, bia
s=False)
                )
                (lora_embedding_A): ParameterDict()
                (lora_embedding_B): ParameterDict()
                (lora_magnitude_vector): ModuleDict()
              )
```

So you can see that now in the Q proj and the V proj layers

learn.deeplearning.ai – To exit full screen, drag from the top and touch the back button

prepare_inputs

```python
def prepare_inputs(prompt, completion):
    # Tokenization
    prompt_tokens = tokenizer(prompt, return_tensors="pt")
    completion_tokens = tokenizer(completion, return_tensors="pt")
```

The first step

```python
def prepare_inputs(prompt, completion):
    # Tokenization
    prompt_tokens = tokenizer(prompt, return_tensors="pt")
    completion_tokens = tokenizer(completion, return_tensors="pt")

    # Combined input
    input_ids = torch.cat(
        [
            prompt_tokens["input_ids"],
            completion_tokens["input_ids"]
        ],
        dim=1
    )
    attention_mask = torch.cat(
        [
            prompt_tokens["attention_mask"],
            completion_tokens["attention_mask"]
        ],
        dim=1
    )
```

tokens into a single tensor,
so that we can pass this to the model.

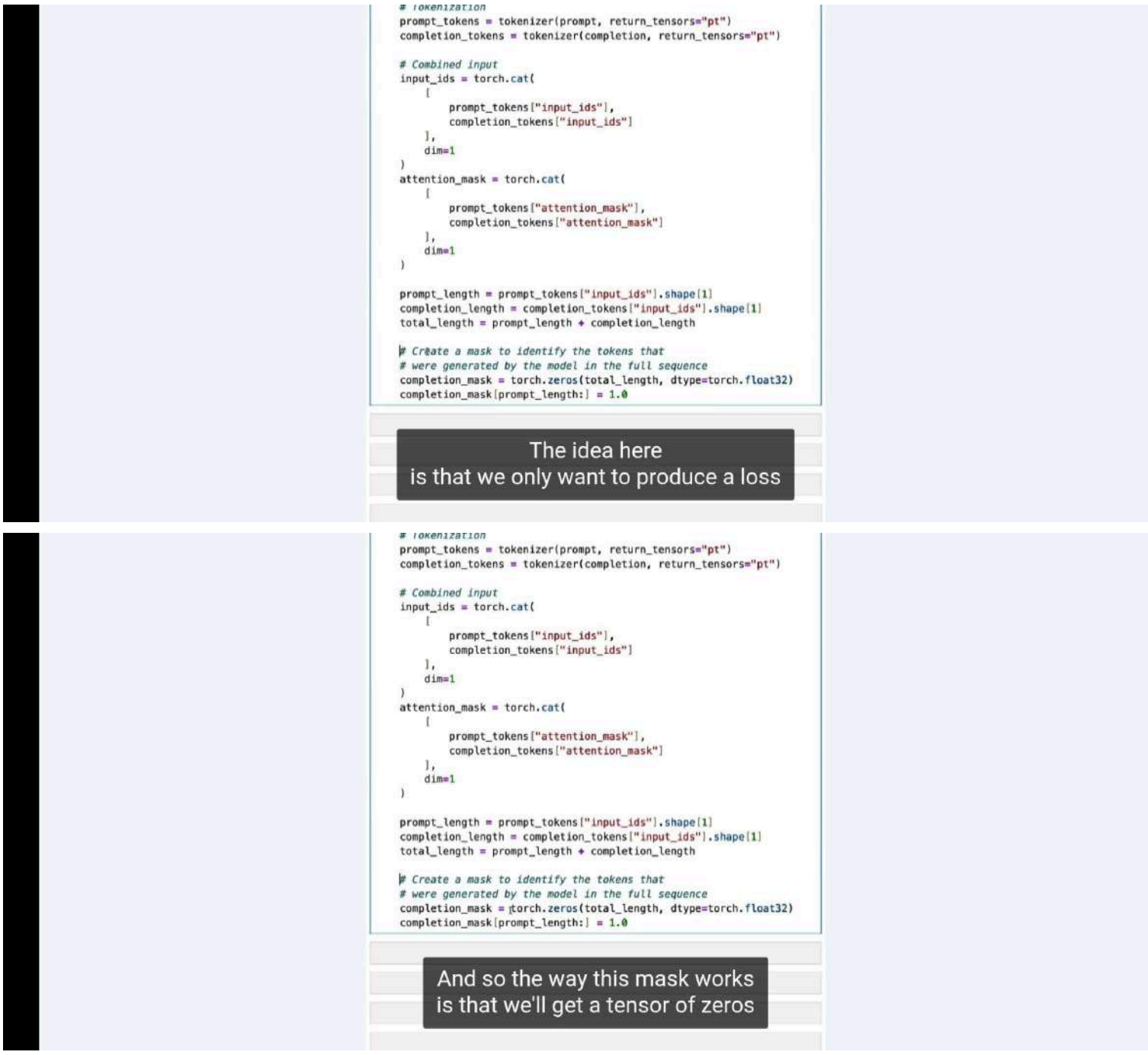
```python
def prepare_inputs(prompt, completion):
    # Tokenization
    prompt_tokens = tokenizer(prompt, return_tensors="pt")
    completion_tokens = tokenizer(completion, return_tensors="pt")

    # Combined input
    input_ids = torch.cat(
        [
            prompt_tokens["input_ids"],
            completion_tokens["input_ids"]
        ],
        dim=1
    )
    attention_mask = torch.cat(
        [
            prompt_tokens["attention_mask"],
            completion_tokens["attention_mask"]
        ],
        dim=1
    )
```

what tokens it should attend to during the forward pass.

```python
    # Tokenization
    prompt_tokens = tokenizer(prompt, return_tensors="pt")
    completion_tokens = tokenizer(completion, return_tensors="pt")

    # Combined input
    input_ids = torch.cat(
        [
            prompt_tokens["input_ids"],
            completion_tokens["input_ids"]
        ],
        dim=1
    )
    attention_mask = torch.cat(
        [
            prompt_tokens["attention_mask"],
            completion_tokens["attention_mask"]
        ],
        dim=1
    )

    prompt_length = prompt_tokens["input_ids"].shape[1]
    completion_length = completion_tokens["input_ids"].shape[1]
    total_length = prompt_length + completion_length
```

Next, we'll capture some metadata the length of the prompt,

attention_mask



```
# Tokenization
prompt_tokens = tokenizer(prompt, return_tensors="pt")
completion_tokens = tokenizer(completion, return_tensors="pt")

# Combined input
input_ids = torch.cat(
    [
        prompt_tokens["input_ids"],
        completion_tokens["input_ids"]
    ],
    dim=1
)
attention_mask = torch.cat(
    [
        prompt_tokens["attention_mask"],
        completion_tokens["attention_mask"]
    ],
    dim=1
)

prompt_length = prompt_tokens["input_ids"].shape[1]
completion_length = completion_tokens["input_ids"].shape[1]
total_length = prompt_length + completion_length

# Create a mask to identify the tokens that
# were generated by the model in the full sequence
completion_mask = torch.zeros(total_length, dtype=torch.float32)
completion_mask[prompt_length:] = 1.0
```

And then we'll set all values to one that followed all the prompt tokens.

```
def compute_log_probs(model, input_ids, attention_mask):
    outputs = model(input_ids, attention_mask=attention_mask)

    # Computing the log-probability of each token in the sequence
    # outputs.logits is the logits for all tokens in the vocabulary for
    log_probs = F.log_softmax(outputs.logits, dim=-1)

    # Extract the log-probability for the actual token that
    # was generated at each position in the sequence.
    return log_probs.gather(
        dim=-1,
        index=input_ids.unsqueeze(-1)
    ).squeeze(-1)
```

Once we've prepared our inputs, we'll define a method called compute log props

Compute_log_probs


```
def compute_log_probs(model, input_ids, attention_mask):
    outputs = model(input_ids, attention_mask=attention_mask)

    # Computing the log-probability of each token in the sequence
    # outputs.logits is the logits for all tokens in the vocabulary for
    log_probs = F.log_softmax(outputs.logits, dim=-1)

    # Extract the log-probability for the actual token that
    # was generated at each position in the sequence.
    return log_probs.gather(
        dim=-1,
        index=input_ids.unsqueeze(-1)
    ).squeeze(-1)
```


We do this by passing in the input IDs and attention mask from the prepare

```python
def compute_log_probs(model, input_ids, attention_mask):
    outputs = model(input_ids, attention_mask=attention_mask)

    # Computing the log-probability of each token in the sequence
    # outputs.logits is the logits for all tokens in the vocabulary for
    log_probs = F.log_softmax(outputs.logits, dim=-1)

    # Extract the log-probability for the actual token that
    # was generated at each position in the sequence.
    return log_probs.gather(
        dim=-1,
        index=input_ids.unsqueeze(-1)
    ).squeeze(-1)
```

We do this by passing in the input IDs and attention mask from the prepare

Grop_loss funtion

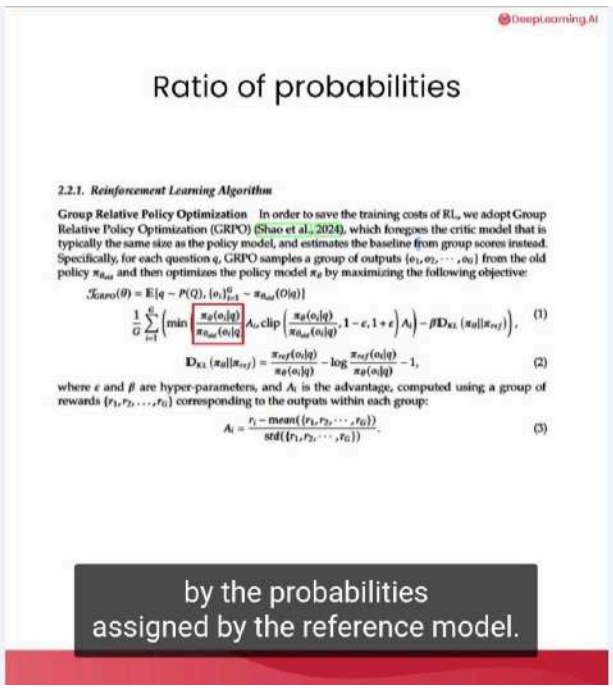
```python
def grpo_loss(model, ref_model, prompt, completion, advantage):
    input_ids, attention_mask, completion_mask = prepare_inputs(
        prompt, completion
    )
```

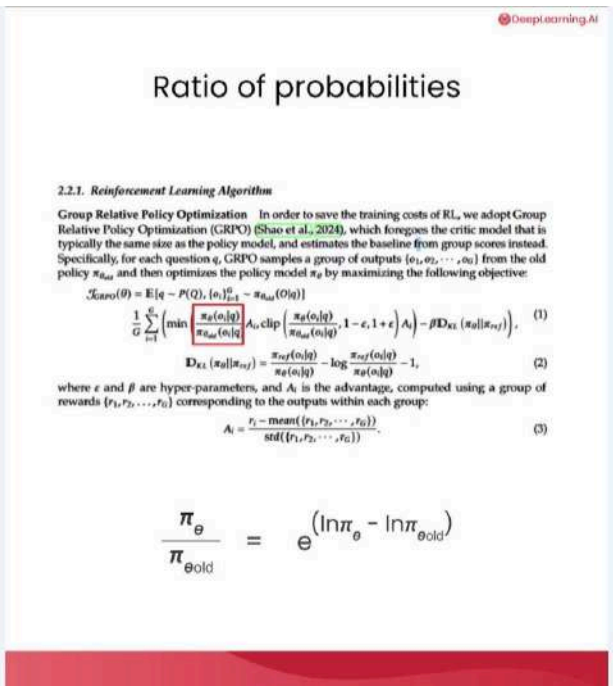
and as you know, the advantage associated with this completion

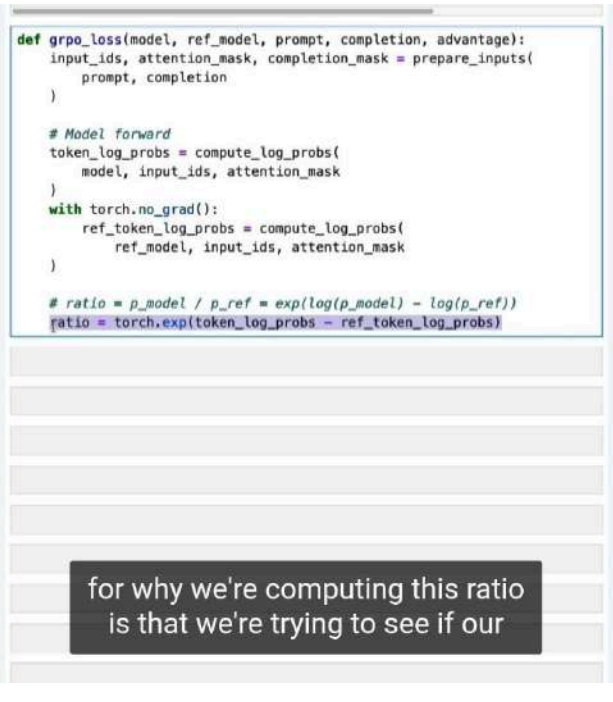
```python
def grpo_loss(model, ref_model, prompt, completion, advantage):
    input_ids, attention_mask, completion_mask = prepare_inputs(
        prompt, completion
    )

    # Model forward
    token_log_probs = compute_log_probs(
        model, input_ids, attention_mask
    )
    with torch.no_grad():
        ref_token_log_probs = compute_log_probs(
            ref_model, input_ids, attention_mask
    )
```

Once we have these two components, we can then focus on the core

Ratio of probabilities







```python
def grpo_loss(model, ref_model, prompt, completion, advantage):
    input_ids, attention_mask, completion_mask = prepare_inputs(
        prompt, completion
    )

    # Model forward
    token_log_probs = compute_log_probs(
        model, input_ids, attention_mask
    )
    with torch.no_grad():
        ref_token_log_probs = compute_log_probs(
            ref_model, input_ids, attention_mask
    )

    # ratio = p_model / p_ref = exp(log(p_model) - log(p_ref))
    ratio = torch.exp(token_log_probs - ref_token_log_probs)

    # Scale the ratio by the advantage function
    policy_loss = ratio * advantage
```

So this tells us that
if the token that the model produced led


```python
def grpo_loss(model, ref_model, prompt, completion, advantage):
    input_ids, attention_mask, completion_mask = prepare_inputs(
        prompt, completion
    )

    # Model forward
    token_log_probs = compute_log_probs(
        model, input_ids, attention_mask
    )
    with torch.no_grad():
        ref_token_log_probs = compute_log_probs(
            ref_model, input_ids, attention_mask
    )

    # ratio = p_model / p_ref = exp(log(p_model) - log(p_ref))
    ratio = torch.exp(token_log_probs - ref_token_log_probs)

    # Scale the ratio by the advantage function
    policy_loss = ratio * advantage
```


to a positive advantage,
we want to boost that as part of the loss.


```python
def grpo_loss(model, ref_model, prompt, completion, advantage):
    input_ids, attention_mask, completion_mask = prepare_inputs(
        prompt, completion
    )

    # Model forward
    token_log_probs = compute_log_probs(
        model, input_ids, attention_mask
    )
    with torch.no_grad():
        ref_token_log_probs = compute_log_probs(
            ref_model, input_ids, attention_mask
    )

    # ratio = p_model / p_ref = exp(log(p_model) - log(p_ref))
    ratio = torch.exp(token_log_probs - ref_token_log_probs)

    # Scale the ratio by the advantage function
    policy_loss = ratio * advantage

    # We want to maximize reward, so we make the loss negative
    # because optimizers minimize loss.
    per_token_loss = -policy_loss
```


the sign of the policy loss
because they're mathematically equivalent.

```python
def grpo_loss(model, ref_model, prompt, completion, advantage):
    input_ids, attention_mask, completion_mask = prepare_inputs(
        prompt, completion
    )

    # Model forward
    token_log_probs = compute_log_probs(
        model, input_ids, attention_mask
    )
    with torch.no_grad():
        ref_token_log_probs = compute_log_probs(
            ref_model, input_ids, attention_mask
    )

    # ratio = p_model / p_ref = exp(log(p_model) - log(p_ref))
    ratio = torch.exp(token_log_probs - ref_token_log_probs)

    # Scale the ratio by the advantage function
    policy_loss = ratio * advantage

    # We want to maximize reward, so we make the loss negative
    # because optimizers minimize loss.
    per_token_loss = -policy_loss
```

loss over just the tokens we care about, which are the output tokens.

```python
def grpo_loss(model, ref_model, prompt, completion, advantage):
    input_ids, attention_mask, completion_mask = prepare_inputs(
        prompt, completion
    )

    # Model forward
    token_log_probs = compute_log_probs(
        model, input_ids, attention_mask
    )
    with torch.no_grad():
        ref_token_log_probs = compute_log_probs(
            ref_model, input_ids, attention_mask
    )

    # ratio = p_model / p_ref = exp(log(p_model) - log(p_ref))
    ratio = torch.exp(token_log_probs - ref_token_log_probs)

    # Scale the ratio by the advantage function
    policy_loss = ratio * advantage

    # We want to maximize reward, so we make the loss negative
    # because optimizers minimize loss.
    per_token_loss = -policy_loss

    # Only compute loss over the output tokens
    loss = (per_token_loss * completion_mask).sum() / completion_mask.su
    return loss
```

If you remember, completion mask is a set of zeros followed by a set of ones.

```python
def grpo_loss(model, ref_model, prompt, completion, advantage):
    input_ids, attention_mask, completion_mask = prepare_inputs(
        prompt, completion
    )

    # Model forward
    token_log_probs = compute_log_probs(
        model, input_ids, attention_mask
    )
    with torch.no_grad():
        ref_token_log_probs = compute_log_probs(
            ref_model, input_ids, attention_mask
    )

    # ratio = p_model / p_ref = exp(log(p_model) - log(p_ref))
    ratio = torch.exp(token_log_probs - ref_token_log_probs)

    # Scale the ratio by the advantage function
    policy_loss = ratio * advantage

    # We want to maximize reward, so we make the loss negative
    # because optimizers minimize loss.
    per_token_loss = -policy_loss

    # Only compute loss over the output tokens
    loss = (per_token_loss * completion_mask).sum() / completion_mask.su
    return loss
```

We sum the last across all the tokens and divided by the length

```
grpo_loss(model, ref_model, prompt, "fence and", advantage=2.0)
```

let's see what happens when we pass in the model reference model.

```
grpo_loss(model, ref_model, prompt, "fence and", advantage=2.0)
```

```
tensor(-1.6791, grad_fn=<DivBackward0>)
```

So you'll see here that if computes a loss of -1.6, which is pretty good,

```
grpo_loss(model, ref_model, prompt, "fence and", advantage=2.0)
```

```
tensor(-1.6791, grad_fn=<DivBackward0>)
```

```
# At step 1, the model and reference model are the same
# So the loss is the advantage function because the ratio of
# per-token log-probabilities is 1
grpo_loss(ref_model, ref_model, prompt, "fence and", advantage=2.0)
```

```
tensor(-2., grad_fn=<DivBackward0>)
```

And it's because in the loss function we multiply the ratio by the advantage.

Calculate grop_loss

```
grpo_loss(model, ref_model, prompt, "fence and", advantage=2.0)
```

```
tensor(-1.6791, grad_fn=<DivBackward0>)
```

```
# At step 1, the model and reference model are the same
# So the loss is the advantage function because the ratio of
# per-token log-probabilities is 1
grpo_loss(ref_model, ref_model, prompt, "fence and", advantage=2.0)
```

```
tensor(-2., grad_fn=<DivBackward0>)
```


```
grpo_loss(model, ref_model, prompt, "fence and", advantage=2.0)
```

```
tensor(-1.6791, grad_fn=<DivBackward0>)
```

```
# At step 1, the model and reference model are the same
# So the loss is the advantage function because the ratio of
# per-token log-probabilities is 1
grpo_loss(ref_model, ref_model, prompt, "fence and", advantage=2.0)
```

```
tensor(-2., grad_fn=<DivBackward0>)
```


If the reward scores are all constant,
we end up with an advantage of zero,

Calculate ratio

```
ratio = torch.exp(token_log_probs - ref_token_log_probs)
print(ratio)
```

```
tensor([[5.2612e+01, 1.3165e+00, 2.7034e-01, 6.1479e-01, 2.1242e+00, 5.3
041e+00,
         2.3053e+01, 1.3070e+03, 2.4560e-01, 6.8247e-01, 3.2403e+00]])
```

One thing you'll notice as you look at this is that some of these values


```
ratio = torch.exp(token_log_probs - ref_token_log_probs)
print(ratio)
```

```
tensor([[5.2612e+01, 1.3165e+00, 2.7034e-01, 6.1479e-01, 2.1242e+00, 5.3
041e+00,
         2.3053e+01, 1.3070e+03, 2.4560e-01, 6.8247e-01, 3.2403e+00]])
```


Clipping

Clipping

2.2.1. Reinforcement Learning Algorithm

Group Relative Policy Optimization In order to save the training costs of RL, we adopt Group Relative Policy Optimization (GRPO) (Shao et al., 2024), which foregoes the critic model that is typically the same size as the policy model, and estimates the baseline from group scores instead. Specifically, for each question q, GRPO samples a group of outputs $\{o_1, o_2, \cdots, o_G\}$ from the old policy $\pi_{\theta_{old}}$ and then optimizes the policy model π_θ by maximizing the following objective:

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^{G} \sim \pi_{\theta_{old}}(O|q)]$$

$$\frac{1}{G}\sum_{i=1}^{G}\left(\min\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}A_i, \text{clip}\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1-\varepsilon, 1+\varepsilon\right)A_i\right) - \beta\mathbb{D}_{KL}\left(\pi_\theta||\pi_{ref}\right)\right), \quad (1)$$

$$\mathbb{D}_{KL}\left(\pi_\theta||\pi_{ref}\right) = \frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - \log\frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - 1, \quad (2)$$

where ε and β are hyper-parameters, and A_i is the advantage, computed using a group of rewards $\{r_1, r_2, \ldots, r_G\}$ corresponding to the outputs within each group:

$$A_i = \frac{r_i - \text{mean}(\{r_1, r_2, \cdots, r_G\})}{\text{std}(\{r_1, r_2, \cdots, r_G\})}. \quad (3)$$

And the mechanism to do this is called ratio clipping,

Grpo_loss with clipped

```python
def grpo_loss(model, ref_model, prompt, completion, advantage, epsilon=0.
    input_ids, attention_mask, completion_mask = prepare_inputs(
        prompt, completion
    )

    # Model forward
    token_log_probs = compute_log_probs(
        model, input_ids, attention_mask
    )
    with torch.no_grad():
        ref_token_log_probs = compute_log_probs(
            ref_model, input_ids, attention_mask
    )

    # ratio = p_model / p_ref = exp(log(p_model) - log(p_ref))
    ratio = torch.exp(token_log_probs - ref_token_log_probs)

    # Scale the ratio by the advantage function
    unclipped = ratio * advantage
    clipped = torch.clamp(ratio, 1 - epsilon, 1 + epsilon) * advantage

    # We want to maximize reward, so we make the loss negative
    # because optimizers minimize loss.
    per_token_loss = -policy_loss

    # Only compute loss over the output tokens
    loss = (per_token_loss * completion_mask).sum() / completion_mask.sur
    return loss
```

these two bounds,
it will clamp them to within the bounds.

Pick the minimum ratio

DeepLearning.AI

Pick the minimum ratio

2.2.1. *Reinforcement Learning Algorithm*

Group Relative Policy Optimization In order to save the training costs of RL, we adopt Group Relative Policy Optimization (GRPO) (Shao et al., 2024), which foregoes the critic model that is typically the same size as the policy model, and estimates the baseline from group scores instead. Specifically, for each question q, GRPO samples a group of outputs $\{o_1, o_2, \cdots, o_G\}$ from the old policy $\pi_{\theta_{old}}$ and then optimizes the policy model π_θ by maximizing the following objective:

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^{G} \sim \pi_{\theta_{old}}(O|q)]$$

$$\frac{1}{G}\sum_{i=1}^{G}\left(\min\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}A_i, \text{clip}\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1-\varepsilon, 1+\varepsilon\right)A_i\right) - \beta\mathbb{D}_{KL}\left(\pi_\theta||\pi_{ref}\right)\right), \quad (1)$$

$$\mathbb{D}_{KL}\left(\pi_\theta||\pi_{ref}\right) = \frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - \log\frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - 1, \quad (2)$$

where ε and β are hyper-parameters, and A_i is the advantage, computed using a group of rewards $\{r_1, r_2, \ldots, r_G\}$ corresponding to the outputs within each group:

$$A_i = \frac{r_i - \text{mean}(\{r_1, r_2, \cdots, r_G\})}{\text{std}(\{r_1, r_2, \cdots, r_G\})}. \quad (3)$$

Once we have the unclip loss and the clip loss we want to pick

Add polcy_loss

```python
def grpo_loss(model, ref_model, prompt, completion, advantage, epsilon=0.
    input_ids, attention_mask, completion_mask = prepare_inputs(
        prompt, completion
    )

    # Model forward
    token_log_probs = compute_log_probs(
        model, input_ids, attention_mask
    )
    with torch.no_grad():
        ref_token_log_probs = compute_log_probs(
            ref_model, input_ids, attention_mask
    )

    # ratio = p_model / p_ref = exp(log(p_model) - log(p_ref))
    ratio = torch.exp(token_log_probs - ref_token_log_probs)

    # Scale the ratio by the advantage function
    unclipped = ratio * advantage
    clipped = torch.clamp(ratio, 1 - epsilon, 1 + epsilon) * advantage

    policy_loss = torch.min(unclipped, clipped)

    # We want to maximize reward, so we make the loss negative
    # because optimizers minimize loss.
    per_token_loss = -policy_loss

    # Only compute loss over the output tokens
    loss = (per_token_loss * completion_mask).sum() / completion_mask.sum
    return loss
```

We invert the sign
so that we can minimize it.

Rename to grop_loss_with_clip

```python
def grpo_loss_with_clip(model, ref_model, prompt, completion, advantage,
    input_ids, attention_mask, completion_mask = prepare_inputs(
        prompt, completion
    )

    # Model forward
    token_log_probs = compute_log_probs(
        model, input_ids, attention_mask
    )
    with torch.no_grad():
        ref_token_log_probs = compute_log_probs(
            ref_model, input_ids, attention_mask
    )

    # ratio = p_model / p_ref = exp(log(p_model) - log(p_ref))
    ratio = torch.exp(token_log_probs - ref_token_log_probs)

    # Scale the ratio by the advantage function
    unclipped = ratio * advantage
    clipped = torch.clamp(ratio, 1 - epsilon, 1 + epsilon) * advantage

    policy_loss = torch.min(unclipped, clipped)

    # We want to maximize reward, so we make the loss negative
    # because optimizers minimize loss.
    per_token_loss = -policy_loss

    # Only compute loss over the output tokens
    loss = (per_token_loss * completion_mask).sum() / completion_mask.sum
    return loss
```

Let's rename this the GRPO loss with clip.

Calculate Grop_loss_with_clip

```
grpo_loss_with_clip(
    model,
    ref_model,
    prompt,
    "fence and",
    advantage=2.0,
    epsilon=0.2
)
```

```
tensor(-1.0822, grad_fn=<DivBackward0>)
```

the loss is much lower than the GRPO loss without the clipping function.

```python
# If we pass the reference model as also the model we're training, the ra
# so your loss will be the advantage.
grpo_loss_with_clip(
    ref_model,
    ref_model,
    prompt,
    "fence and",
    advantage=2.0
)
```

```
tensor(-2., grad_fn=<DivBackward0>)
```

on each token's probability ratio,
and not on the overall loss.

ref_token_log_probs

```
import pandas as pd

completion = "fence and"

input_ids, attention_mask, _ = prepare_inputs(prompt, completion)
with torch.no_grad():
    token_log_probs = compute_log_probs(
        model, input_ids, attention_mask
    )
    ref_token_log_probs = compute_log_probs(
        ref_model, input_ids, attention_mask
    )

with torch.no_grad():
    epsilon = 0.2
    ratio = torch.exp(token_log_probs - ref_token_log_probs)
    ratio_unclipped = ratio
    ratio_clipped = torch.clamp(ratio, 1 - epsilon, 1 + epsilon)
    policy_loss = torch.min(ratio_unclipped, ratio_clipped)

visualize_clipped_ratios(ratio_unclipped[0][9:], ratio_clipped[0][9:], ep
```

probabilities from the reference model and the policy model.

```python
import pandas as pd

completion = "fence and"

input_ids, attention_mask, _ = prepare_inputs(prompt, completion)
with torch.no_grad():
    token_log_probs = compute_log_probs(
        model, input_ids, attention_mask
    )
    ref_token_log_probs = compute_log_probs(
        ref_model, input_ids, attention_mask
    )

with torch.no_grad():
    epsilon = 0.2
    ratio = torch.exp(token_log_probs - ref_token_log_probs)
    ratio_unclipped = ratio
    ratio_clipped = torch.clamp(ratio, 1 - epsilon, 1 + epsilon)
    policy_loss = torch.min(ratio_unclipped, ratio_clipped)

visualize_clipped_ratios(ratio_unclipped[0][9:], ratio_clipped[0][9:], ep
```

	token_position	ratio_unclipped	ratio_clipped	was_clipped
0	0	11.140301	1.2	✓
1	1	0.773877	0.8	✓

And so the clip ratio is 1.2.

```python
import pandas as pd

completion = "fence and"

input_ids, attention_mask, _ = prepare_inputs(prompt, completion)
with torch.no_grad():
    token_log_probs = compute_log_probs(
        model, input_ids, attention_mask
    )
    ref_token_log_probs = compute_log_probs(
        ref_model, input_ids, attention_mask
    )

with torch.no_grad():
    epsilon = 0.2
    ratio = torch.exp(token_log_probs - ref_token_log_probs)
    ratio_unclipped = ratio
    ratio_clipped = torch.clamp(ratio, 1 - epsilon, 1 + epsilon)
    policy_loss = torch.min(ratio_unclipped, ratio_clipped)

visualize_clipped_ratios(ratio_unclipped[0][9:], ratio_clipped[0][9:], ep
```

	token_position	ratio_unclipped	ratio_clipped	was_clipped
0	0	11.140301	1.2	✓
1	1	0.773877	0.8	✓

And this is not necessarily a bad thing, but often we introduce a penalty term

KL Divergence

DeepLearning.AI

KL Divergence

2.2.1. *Reinforcement Learning Algorithm*

Group Relative Policy Optimization In order to save the training costs of RL, we adopt Group Relative Policy Optimization (GRPO) (Shao et al., 2024), which foregoes the critic model that is typically the same size as the policy model, and estimates the baseline from group scores instead. Specifically, for each question q, GRPO samples a group of outputs $\{o_1, o_2, \cdots, o_G\}$ from the old policy $\pi_{\theta_{old}}$ and then optimizes the policy model π_θ by maximizing the following objective:

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^{G} \sim \pi_{\theta_{old}}(O|q)]$$

$$\frac{1}{G}\sum_{i=1}^{G}\left(\min\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}A_i, \text{clip}\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1-\varepsilon, 1+\varepsilon\right)A_i\right) - \beta\mathbb{D}_{KL}\left(\pi_\theta||\pi_{ref}\right)\right), \quad (1)$$

$$\mathbb{D}_{KL}\left(\pi_\theta||\pi_{ref}\right) = \frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - \log\frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - 1, \quad (2)$$

where ε and β are hyper-parameters, and A_i is the advantage, computed using a group of rewards $\{r_1, r_2, \ldots, r_G\}$ corresponding to the outputs within each group:

$$A_i = \frac{r_i - \text{mean}(\{r_1, r_2, \cdots, r_G\})}{\text{std}(\{r_1, r_2, \cdots, r_G\})}. \quad (3)$$

And this is not necessarily a bad thing, but often we introduce a penalty term

grpo_loss_with_kl

```python
def grpo_loss_with_kl(model, ref_model, prompt, completion, advantage, ep
    input_ids, attention_mask, completion_mask = prepare_inputs(
        prompt, completion
    )

    # Model forward
    token_log_probs = compute_log_probs(
        model, input_ids, attention_mask
    )
    with torch.no_grad():
        ref_token_log_probs = compute_log_probs(
            ref_model, input_ids, attention_mask
    )

    # ratio = p_model / p_ref = exp(log(p_model) - log(p_ref))
    ratio = torch.exp(token_log_probs - ref_token_log_probs)

    # Scale the ratio by the advantage function
    unclipped = ratio * advantage
    clipped = torch.clamp(ratio, 1 - epsilon, 1 + epsilon) * advantage

    policy_loss = torch.min(unclipped, clipped)

    # Compute the per-token KL divergence to encourage the model
    # to stay close to the reference model
    delta = token_log_probs - ref_token_log_probs
    per_token_kl = torch.exp(-delta) + delta - 1

    # We want to maximize reward, so we make the loss negative
    # because we want to minimize loss.
    per_token_loss = -policy_loss
    # Only compute loss over the output tokens
    loss = (per_token_loss * completion_mask).sum() / completion_mask.sum
    return loss
```

deviate between the policy model
and the reference model for defining delta

Beta * per_token_kl

```python
def grpo_loss_with_kl(model, ref_model, prompt, completion, advantage, ep
    input_ids, attention_mask, completion_mask = prepare_inputs(
        prompt, completion
    )

    # Model forward
    token_log_probs = compute_log_probs(
        model, input_ids, attention_mask
    )
    with torch.no_grad():
        ref_token_log_probs = compute_log_probs(
            ref_model, input_ids, attention_mask
    )

    # ratio = p_model / p_ref = exp(log(p_model) - log(p_ref))
    ratio = torch.exp(token_log_probs - ref_token_log_probs)

    # Scale the ratio by the advantage function
    unclipped = ratio * advantage
    clipped = torch.clamp(ratio, 1 - epsilon, 1 + epsilon) * advantage

    policy_loss = torch.min(unclipped, clipped)

    # Compute the per-token KL divergence to encourage the model
    # to stay close to the reference model
    delta = token_log_probs - ref_token_log_probs
    per_token_kl = torch.exp(-delta) + delta - 1

    # We want to maximize reward, so we make the loss negative
    # because optimizers minimize loss.
    per_token_loss = -(policy_loss - beta * per_token_kl)

    # Only compute loss over the output tokens
    loss = (per_token_loss * completion_mask).sum() / completion_mask.sum
    return loss
```

```python
def grpo_loss_with_kl(model, ref_model, prompt, completion, advantage, ep
    input_ids, attention_mask, completion_mask = prepare_inputs(
        prompt, completion
    )

    # Model forward
    token_log_probs = compute_log_probs(
        model, input_ids, attention_mask
    )
    with torch.no_grad():
        ref_token_log_probs = compute_log_probs(
            ref_model, input_ids, attention_mask
    )

    # ratio = p_model / p_ref = exp(log(p_model) - log(p_ref))
    ratio = torch.exp(token_log_probs - ref_token_log_probs)

    # Scale the ratio by the advantage function
    unclipped = ratio * advantage
    clipped = torch.clamp(ratio, 1 - epsilon, 1 + epsilon) * advantage

    policy_loss = torch.min(unclipped, clipped)

    # Compute the per-token KL divergence to encourage the model
    # to stay close to the reference model
    delta = token_log_probs - ref_token_log_probs
    per_token_kl = torch.exp(-delta) + delta - 1

    # We want to maximize reward, so we make the loss negative
    # because we minimize loss
    per_token_loss = -(policy_loss - beta * per_token_kl)

    # Only compute loss over the output tokens
    loss = (per_token_loss * completion_mask).sum() / completion_mask.sum
    return loss
```

Now let's start by understanding how the KL divergence term works.

Plot KL_divergence

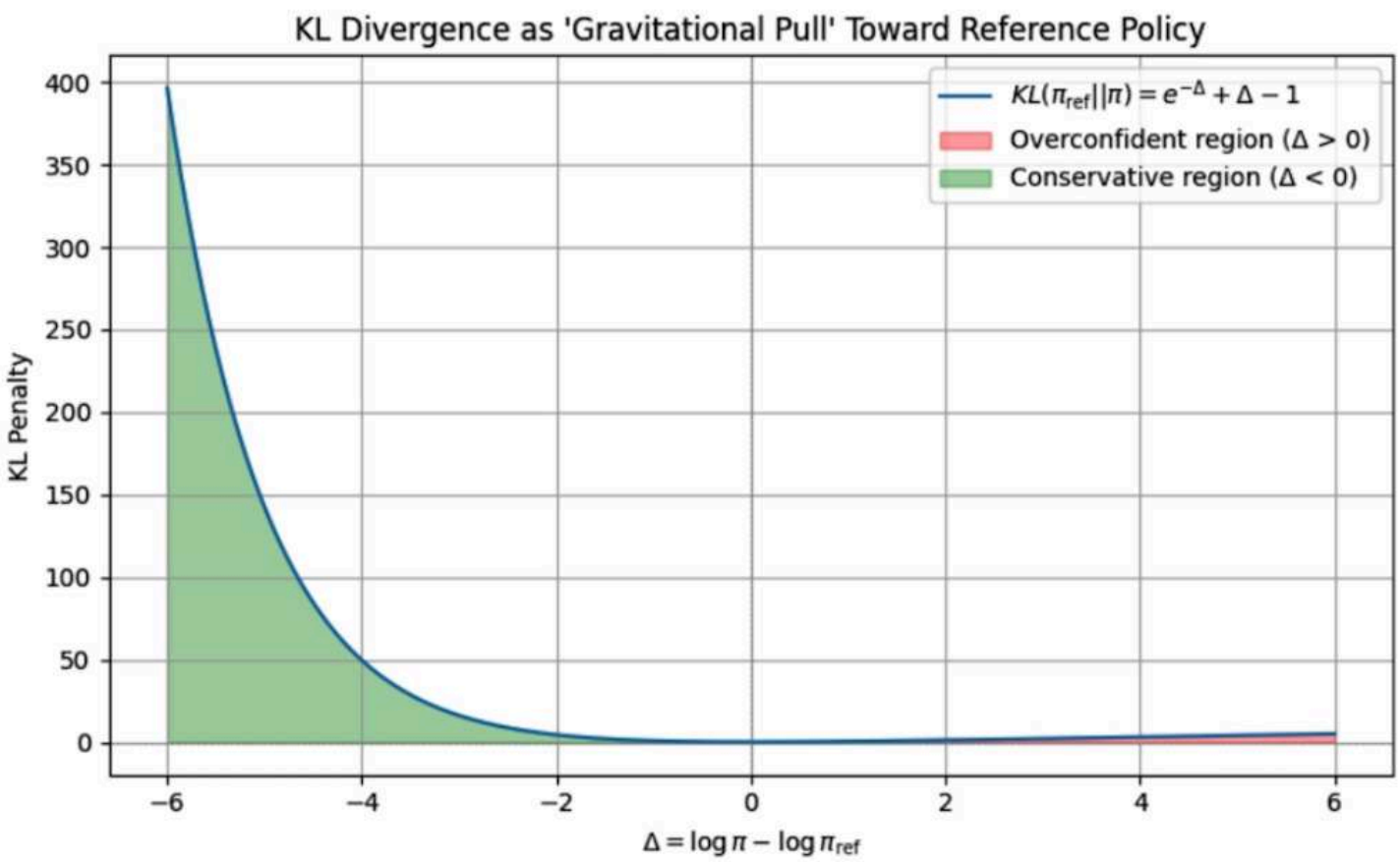
```python
import matplotlib.pyplot as plt

# Define the range of Δ (log-probability difference between
# model and reference)
delta = np.linspace(-6, 6, 500)

# Compute the per-token reverse KL divergence: KL(π_ref || π)
kl_divergence = np.exp(-delta) + delta - 1

# Plot the KL divergence
plt.figure(figsize=(8, 5))
plt.plot(delta, kl_divergence, label=r'$KL(\pi_{\mathrm{ref}} || \pi) = e
plt.axhline(0, color='gray', linestyle='--', linewidth=0.5)
plt.axvline(0, color='gray', linestyle='--', linewidth=0.5)
plt.fill_between(delta, kl_divergence, where=(delta > 0), color='red', a
plt.fill_between(delta, kl_divergence, where=(delta < 0), color='green',
plt.title("KL Divergence as 'Gravitational Pull' Toward Reference Policy'
plt.xlabel(r'$\Delta = \log \pi - \log \pi_{\mathrm{ref}}$')
plt.ylabel('KL Penalty')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

when the delta is between a range of negative six and six. At the point

Deviation

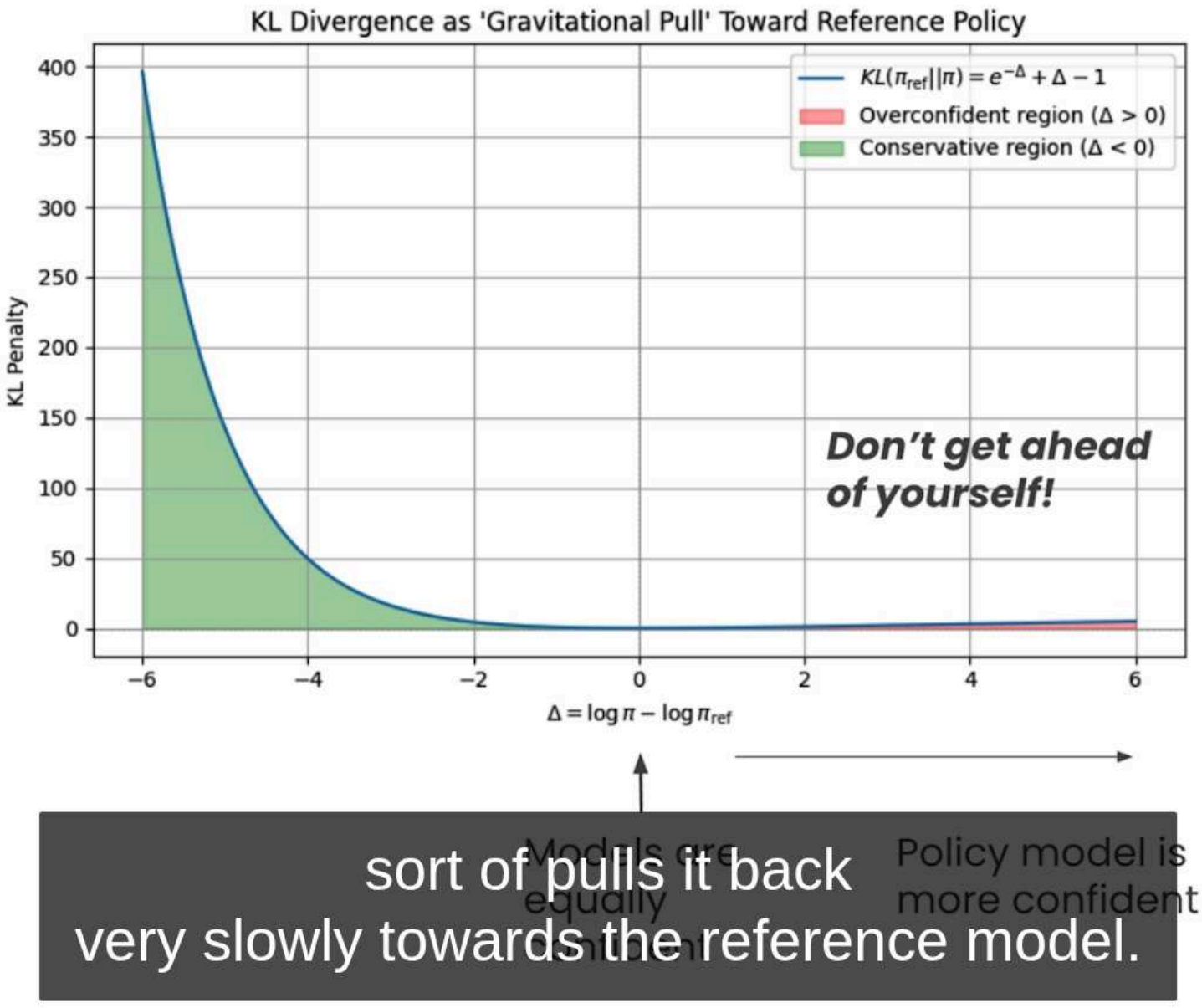
DeepLearning.AI

Course correcting with KL divergence



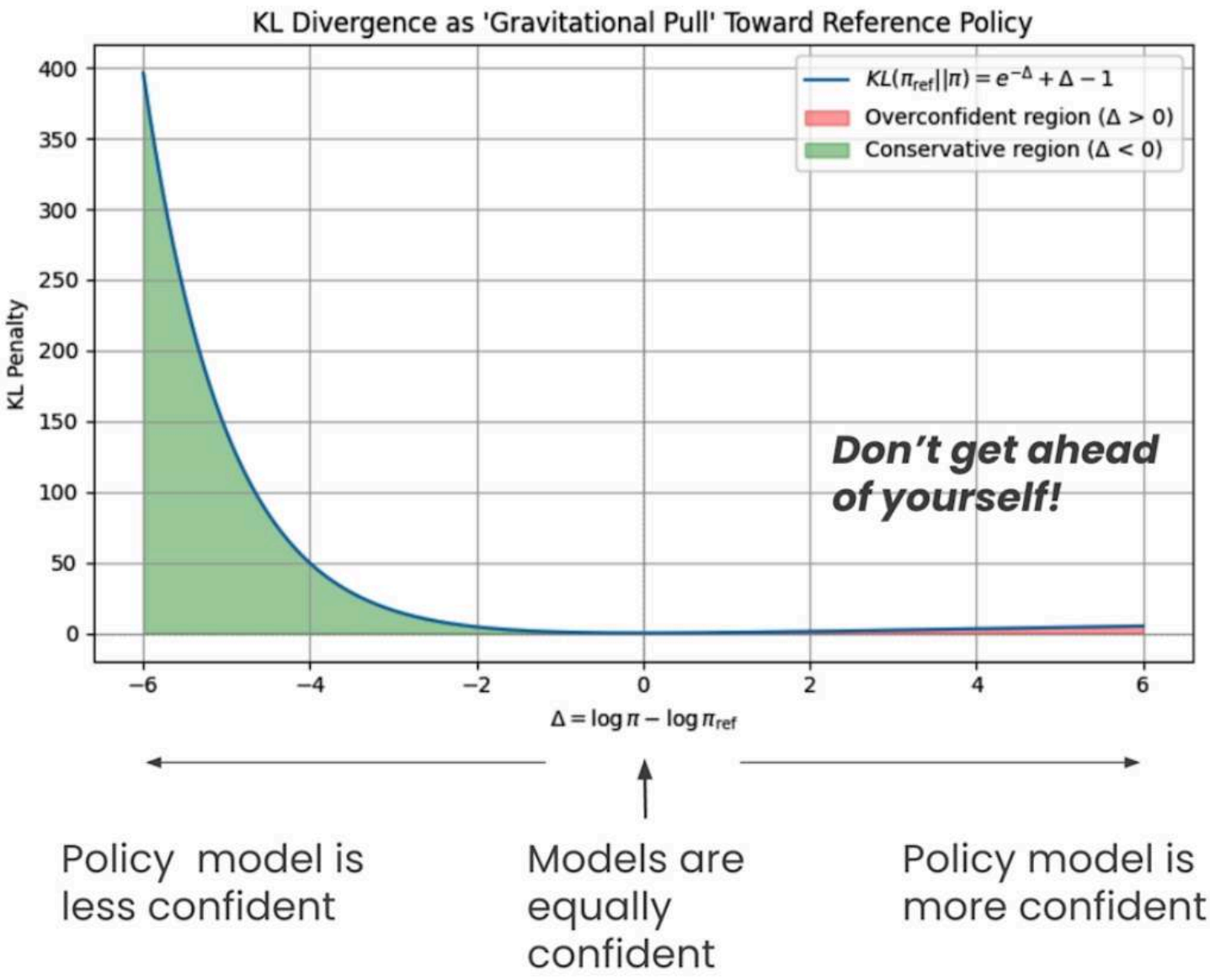
and so the equally confident in the tokens being produced.

Course correcting with KL divergence



sort of pulls it back
very slowly towards the reference model.

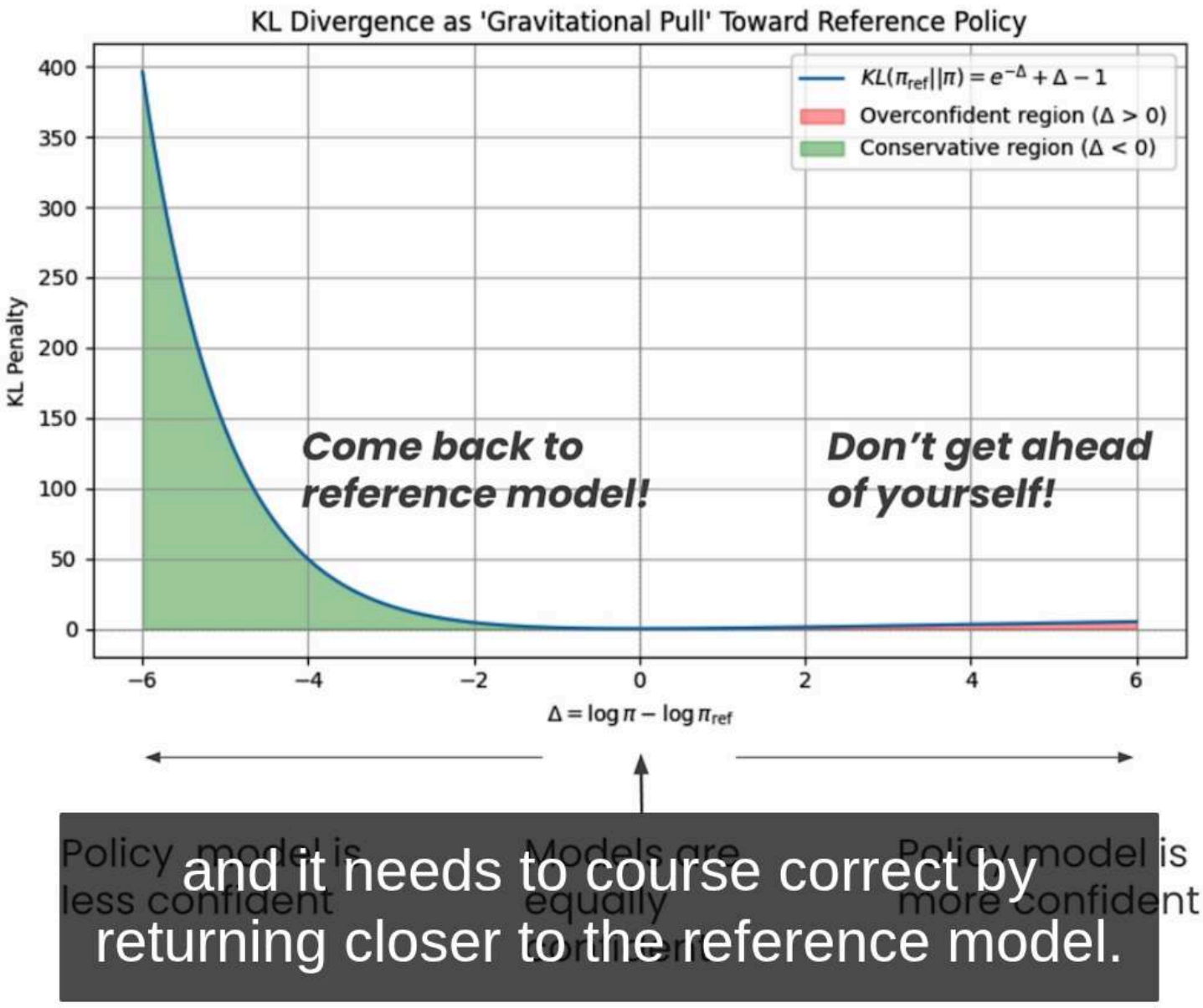
Course correcting with KL divergence



Come back to reference model

DeepLearning.AI

Course correcting with KL divergence



Beta term in KL divergence

Beta term in KL divergence

2.2.1. *Reinforcement Learning Algorithm*

Group Relative Policy Optimization In order to save the training costs of RL, we adopt Group Relative Policy Optimization (GRPO) (Shao et al., 2024), which foregoes the critic model that is typically the same size as the policy model, and estimates the baseline from group scores instead. Specifically, for each question q, GRPO samples a group of outputs $\{o_1, o_2, \cdots, o_G\}$ from the old policy $\pi_{\theta_{old}}$ and then optimizes the policy model π_θ by maximizing the following objective:

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^{G} \sim \pi_{\theta_{old}}(O|q)]$$

$$\frac{1}{G}\sum_{i=1}^{G}\left(\min\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}A_i, \text{clip}\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1-\varepsilon, 1+\varepsilon\right)A_i\right) - \beta\mathbb{D}_{KL}\left(\pi_\theta||\pi_{ref}\right)\right), \quad (1)$$

$$\mathbb{D}_{KL}\left(\pi_\theta||\pi_{ref}\right) = \frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - \log\frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - 1, \quad (2)$$

where ε and β are hyper-parameters, and A_i is the advantage, computed using a group of rewards $\{r_1, r_2, \ldots, r_G\}$ corresponding to the outputs within each group:

$$A_i = \frac{r_i - mean(\{r_1, r_2, \cdots, r_G\})}{std(\{r_1, r_2, \cdots, r_G\})}. \quad (3)$$

how much KL divergence
will we actually add to the loss.

Beta increase → loss decreasing

```
for beta in [0, 0.1, 0.5]:
    loss = grpo_lo
        model,
        ref_model,
        prompt,
        "fence and",
        advantage=2.0,
        epsilon=0.2,
        beta=beta
    )
    print(f"beta={beta}")
    print(f"loss={loss.item():.3f}")
    print()
```



```
beta=0
loss=-2.269

beta=0.1
loss=-2.062

beta=0.5
loss=-0.472
```

you can see that the loss value actually starts to decrease.

GRPO algorithm

2.2.1. *Reinforcement Learning Algorithm*

Group Relative Policy Optimization In order to save the training costs of RL, we adopt Group Relative Policy Optimization (GRPO) (Shao et al., 2024), which foregoes the critic model that is typically the same size as the policy model, and estimates the baseline from group scores instead. Specifically, for each question q, GRPO samples a group of outputs $\{o_1, o_2, \cdots, o_G\}$ from the old policy $\pi_{\theta_{old}}$ and then optimizes the policy model π_θ by maximizing the following objective:

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^{G} \sim \pi_{\theta_{old}}(O|q)]$$
$$\frac{1}{G}\sum_{i=1}^{G}\left(\min\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}A_i, \text{clip}\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1-\varepsilon, 1+\varepsilon\right)A_i\right) - \beta\mathbb{D}_{KL}\left(\pi_\theta||\pi_{ref}\right)\right), \quad (1)$$

$$\mathbb{D}_{KL}\left(\pi_\theta||\pi_{ref}\right) = \frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - \log\frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - 1, \quad (2)$$

where ε and β are hyper-parameters, and A_i is the advantage, computed using a group of rewards $\{r_1, r_2, \ldots, r_G\}$ corresponding to the outputs within each group:

$$A_i = \frac{r_i - mean(\{r_1, r_2, \cdots, r_G\})}{std(\{r_1, r_2, \cdots, r_G\})}. \quad (3)$$

is that each text sample is weighted by how much reward it receives.

from Guo et al. 2025

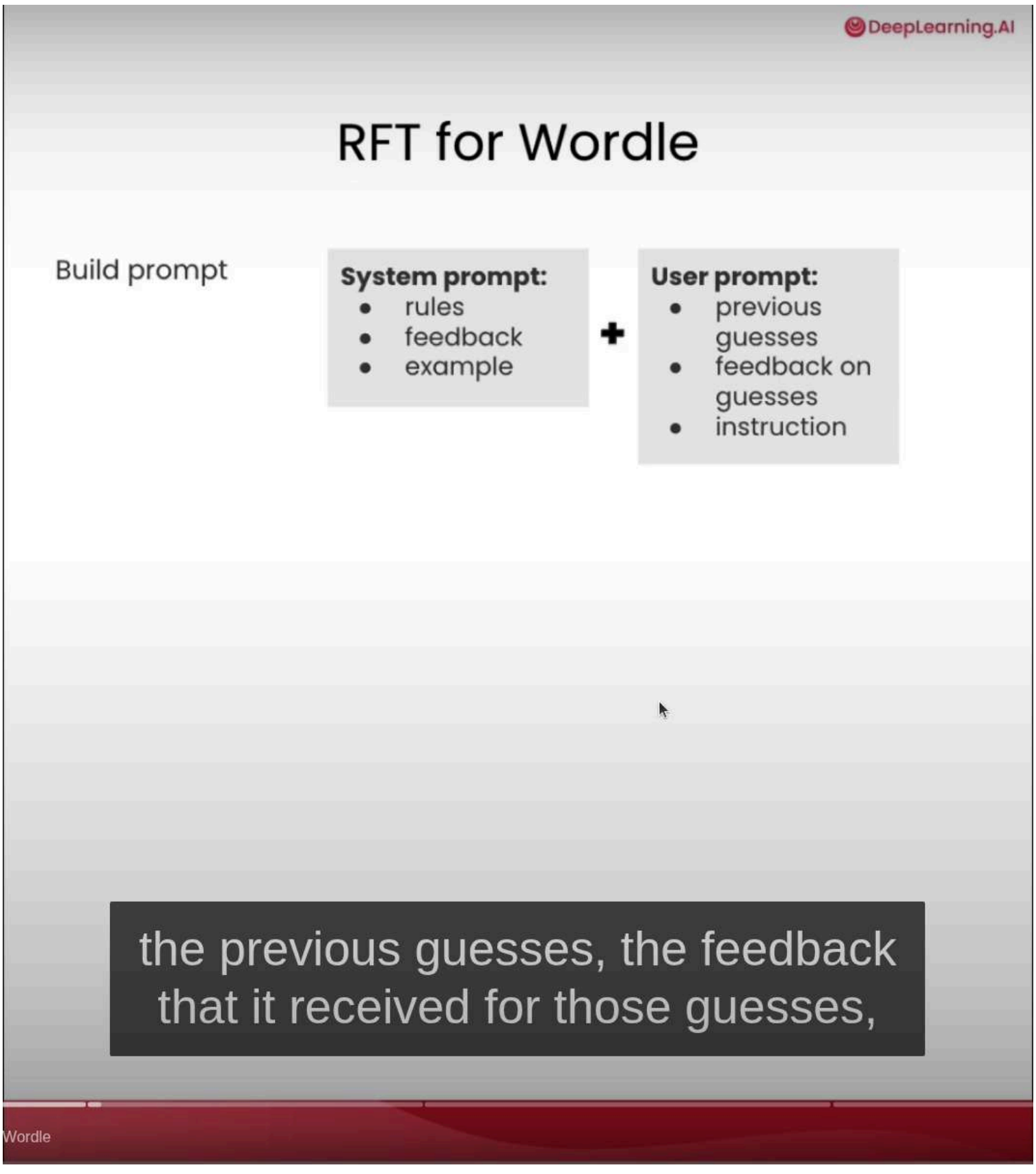
Putting it all together: Training wordle

Predibase

DeepLearning.AI

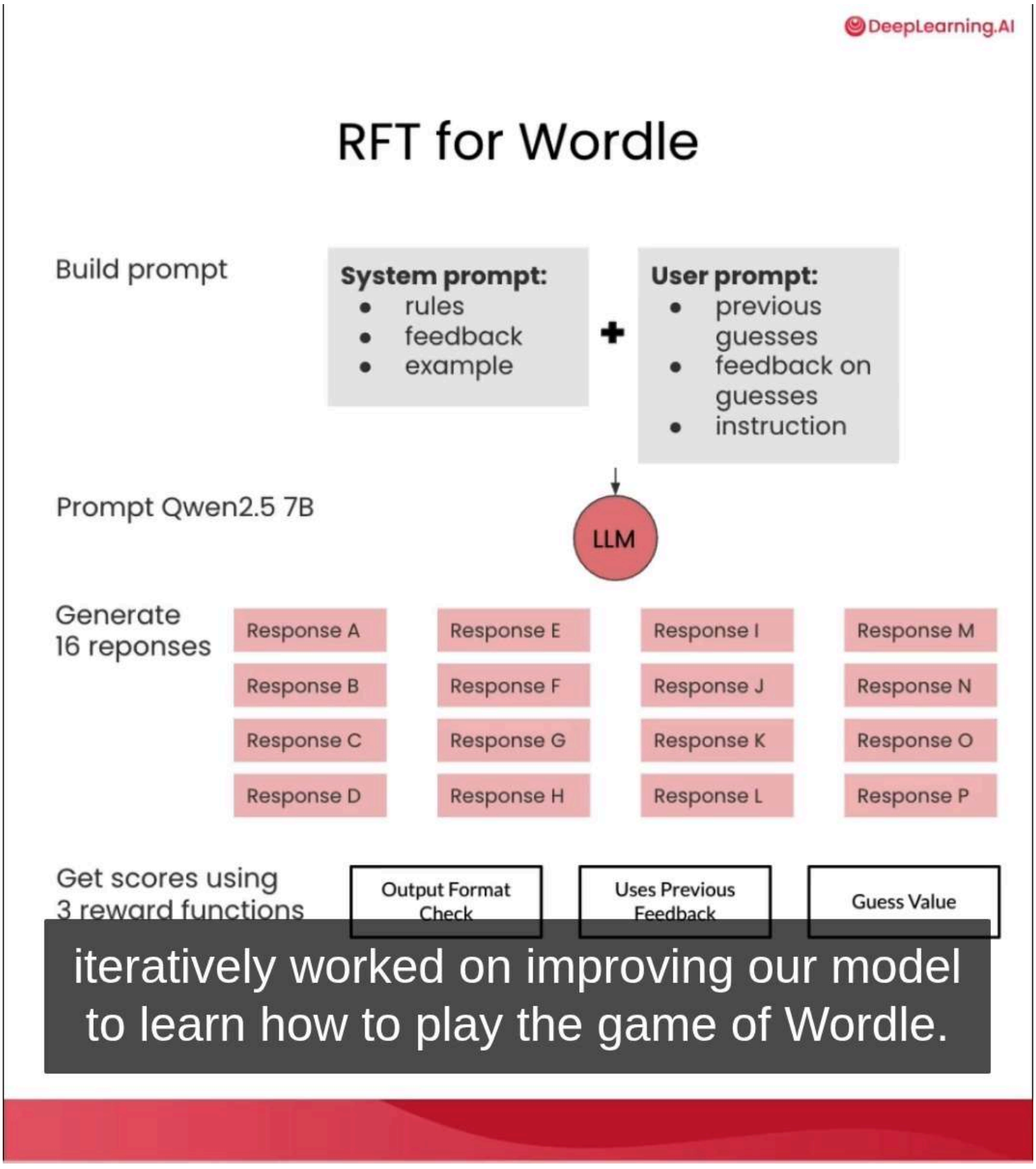
RFT for Wordle

Build prompt



Get 16 response

Get score using 3 rewarding function



** Reward function for wordle

1 Output format check

Predibase DeepLearning.AI

Reward functions for wordle

1. Output format check
   - Check for correct use of **<think></think>** and **<guess></guess>** tags
   - Check for valid 5-letter word

that the model's response includes the correct think and guess tags,

Predibase DeepLearning.AI

Reward functions for wordle

1. Output format check
   - Check for correct use of **‹think›‹/think›** and **‹guess›‹/guess›** tags
   - Check for valid 5-letter word

2. Uses previous feedback
   - Score assigned based on how well prior feedback is used in the new guess

3 Guess value

Predibase DeepLearning.AI

Reward functions for wordle

1. Output format check
   - Check for correct use of **<think></think>** and **<guess></guess>** tags
   - Check for valid 5-letter word
2. Uses previous feedback
   - Score assigned based on how well prior feedback is used in the new guess
3. Guess value
   - Score assigned based on how useful the guess is at eliminating possible solutions

function scores, how effective it guesses, and then eliminating possibilities.

Full code in reward_function.py

Predibase DeepLearning.AI

Reward functions for wordle

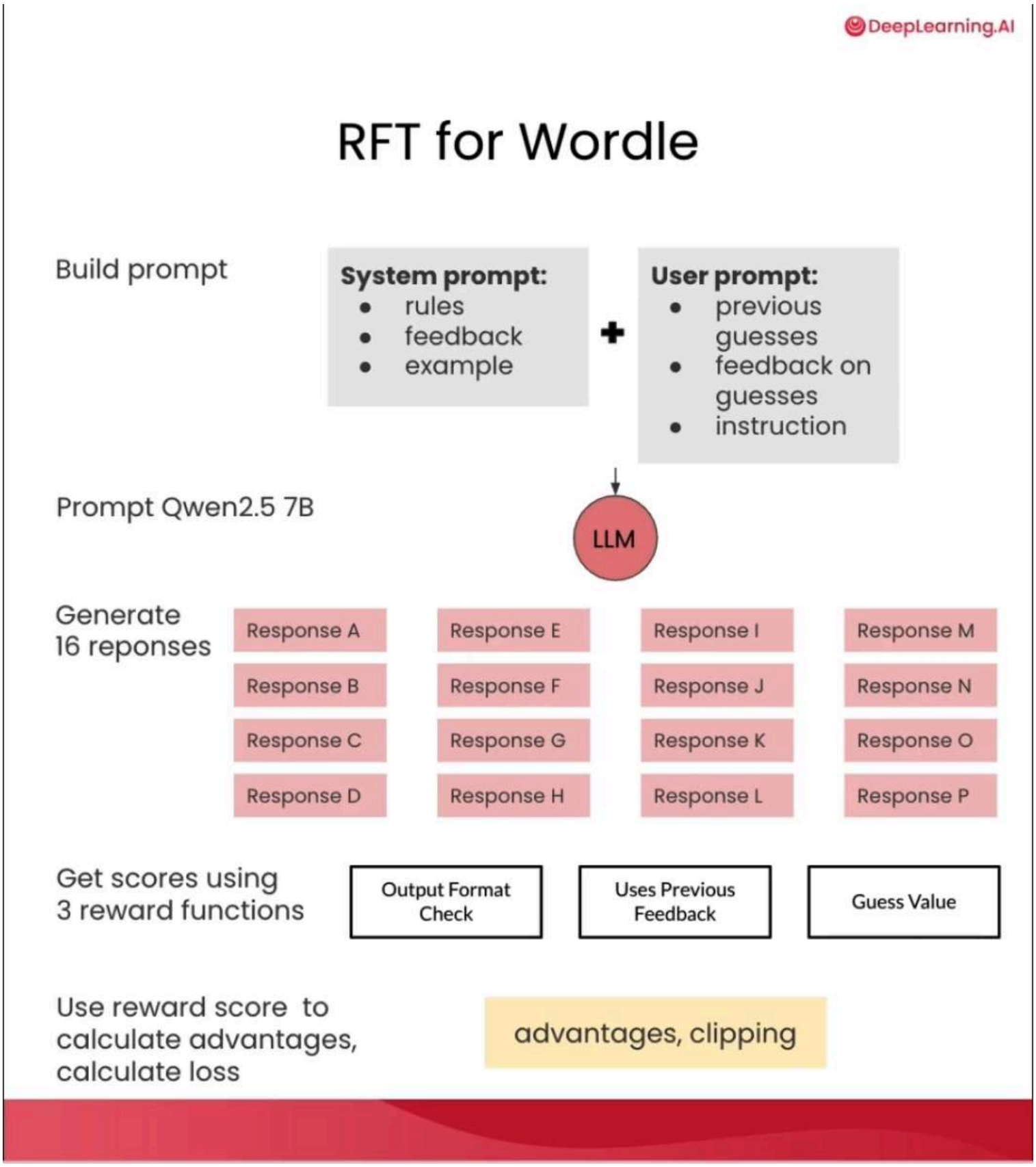
1. Output format check
   - Check for correct use of **<think></think>** and **<guess></guess>** tags
   - Check for valid 5-letter word

2. Uses previous feedback
   - Score assigned based on how well prior feedback is used in the new guess

3. Guess value
   - Score assigned based on how useful the guess is at eliminating possible solutions

To see the full code for these functions, please check out the **reward_functions.py** file for this lesson, accessible via the **File -> Open** menu item in the Jupyter notebook.

Use reward score → calculate advantage , calculate loss



Project inital

```
import os

from predibase import (
    Predibase,
    GRPOConfig,
    RewardFunctionsConfig,
    RewardFunctionsRuntimeConfig,
    SFTConfig,
    SamplingParamsConfig,
)
from datasets import load_dataset
from dotenv import load_dotenv
```

```
load_dotenv("../.env")
pb = Predibase(api_token=os.environ["PREDIBASE_API_TOKEN"])
```

and you can sign into the Predibase SDK by providing your API token as so.

```python
import os

from predibase import (
    Predibase,
    GRPOConfig,
    RewardFunctionsConfig,
    RewardFunctionsRuntimeConfig,
    SFTConfig,
    SamplingParamsConfig,
)
from datasets import load_dataset
from dotenv import load_dotenv
```

```python
load_dotenv("../.env")
pb = Predibase(api_token=os.environ["PREDIBASE_API_TOKEN"])
```

```
WARN: Currently installed SDK is outdated. This can lead to bugs o
latest version. Installed: 2025.3.2 Latest: 2025.4.1.
```

```
WARN: Currently installed SDK is outdated. This can lead to bugs o
latest version. Installed: 2025.3.2 Latest: 2025.4.1.
```

```
Connected to Predibase as User(id=67b2c694-adb7-4297-b49f-1aefd1a6
username=arnav-dogfooding-prod@predibase.com)
```

Load dataset

```python
# Load dataset from HuggingFace
dataset = load_dataset("predibase/wordle-grpo", split="train")
dataset = dataset.to_pandas()

# Upload dataset in Predibase
try:
    dataset = pb.datasets.from_pandas_dataframe(
        dataset,
        name="wordle_grpo_data"
    )
except Exception:
    pass
```

Create new repository in Predibase

```
# Create repository in Predibase
repo = pb.repos.create(name="wordle", exists_ok=True)
```

A repository is just like a GitHub repository, except that you can use it

Import reward function

```
# Import reward functions
from reward_functions import (
    guess_value,
    output_format_check,
    uses_previous_feedback,
)
```

the output from our check,
and the user's previous feedback.

Create fine tuning job

```python
# Create GRPO training run in Predibase by specifying the config,
# dataset, repository and reward functions
pb.finetuning.jobs.create(
    config=GRPOConfig(
        base_model="qwen2-5-7b-instruct",
        reward_fns=RewardFunctionsConfig(
            runtime=RewardFunctionsRuntimeConfig(
                packages=["pandas"]
            ),
            functions={
                "output_format_check": output_format_check,
                "uses_previous_feedback": uses_previous_feedback,
                "guess_value": guess_value,
            }
        ),
        sampling_params=SamplingParamsConfig(max_tokens=4096),
        num_generations=8
    ),
    dataset=dataset,
    repo="wordle",
    description="Wordle GRPO"
)
```

a config, to define what we want to train with, reward

RPO Training Config

Fine tuning

```
        base_model="qwen2-5-7b-instruct",
        reward_fns=RewardFunctionsConfig(
            runtime=RewardFunctionsRuntimeConfig(
                packages=["pandas"]
            ),
            functions={
                "output_format_check": output_format_check,
                "uses_previous_feedback": uses_previous_feedback,
                "guess_value": guess_value,
            }
        ),
        sampling_params=SamplingParamsConfig(max_tokens=4096),
        num_generations=16
    ),
    dataset=dataset,
    repo="wordle",
    description="Wordle GRPO"
)
```

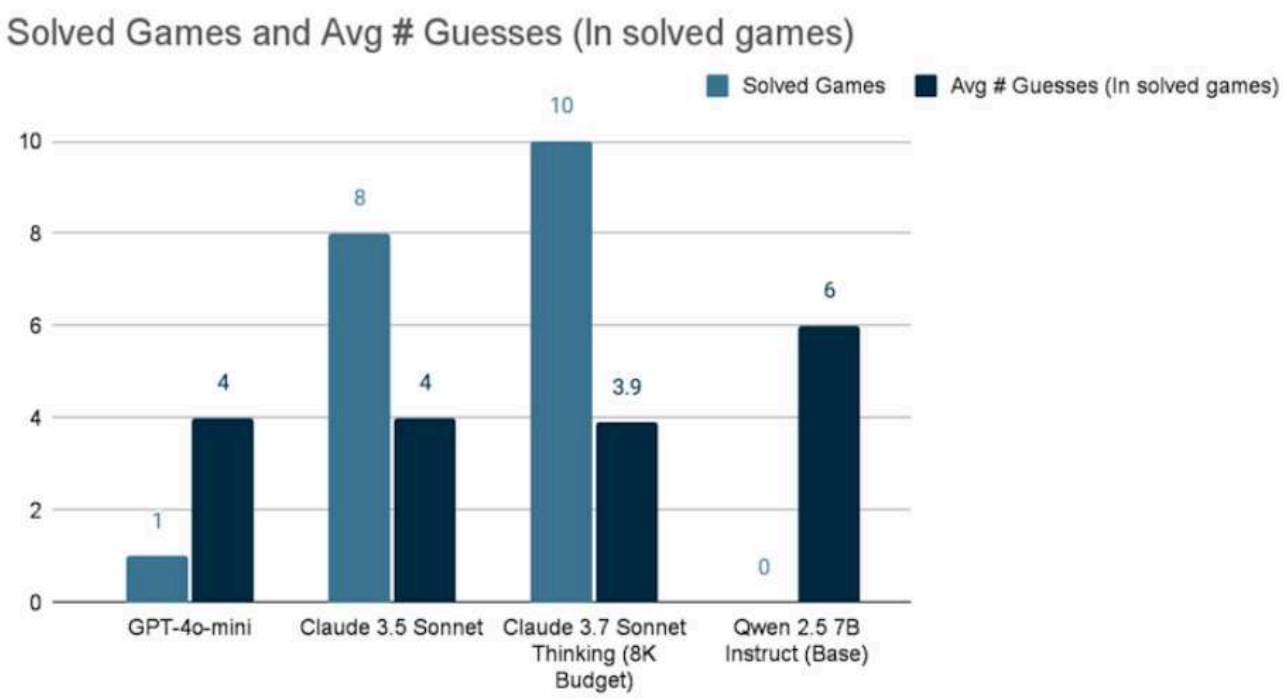
learn.deeplearning.ai – To exit full screen, press Esc

```
Successfully requested finetuning of qwen2-5-7b-instruct from base as `w
ordle/15`. (Job UUID: 4b3db722-a124-4257-9063-9854ae3d3438).

FinetuningJob(uuid='4b3db722-a124-4257-9063-9854ae3d3438', base_model='q
wen2-5-7b-instruct', dataset='wordle_grpo_data', description='Wordle GRP
O', target_repo='wordle', target_version_tag=15, accelerator='', status
='pending', params=GRPOConfig(base_model='qwen2-5-7b-instruct', adapter
='lora', task='grpo', epochs=None, learning_rate=None, rank=None, target
_modules=None, enable_early_stopping=None, apply_chat_template=None, lr_
scheduler=None, optimizer=None, lora_alpha=None, lora_dropout=None, warm
up_ratio=None, effective_batch_size=None, beta=None, num_generations=16,
sampling_params=SamplingParamsConfig(temperature=None, top_p=None, top_k
=None, max_tokens=4096), reward_fns=RewardFunctionsConfig(runtime=Reward
FunctionsRuntimeConfig(packages=['pandas']), functions={'guess_value': R
ewardFunction(encoded_fn='ZGVmIGd1ZXNzX3ZhbHVlKHByb21wdD...'), 'output_f
ormat_check': RewardFunction(encoded_fn='ZGVmIG91dHB1dF9mb3JtYXRfY2hlY
2...'), 'uses_previous_feedback': RewardFunction(encoded_fn='ZGVmIHVzZXN
fcHJldmlvdXNfZmVlZG...')})))
```

If you want to try this yourself,

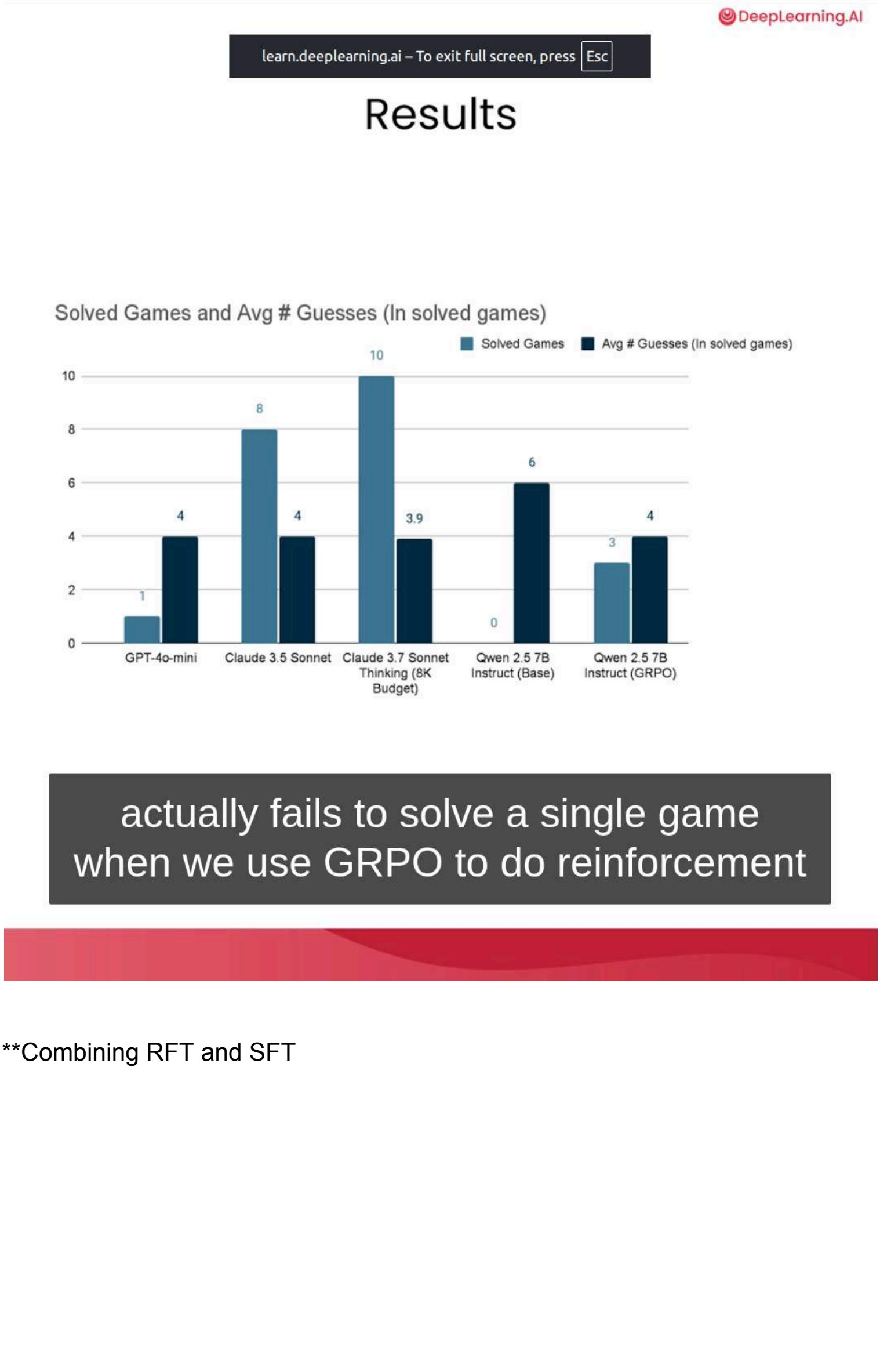
DeepLearning.AI

Results



And specifically we measured two metrics, the number of games

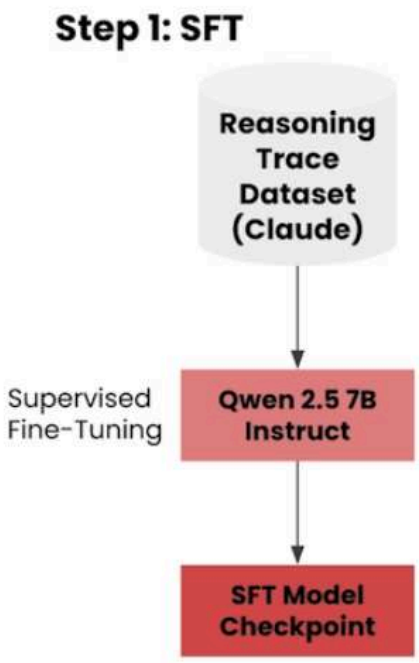
After GPRO result



**Combining RFT and SFT

DeepLearning.AI

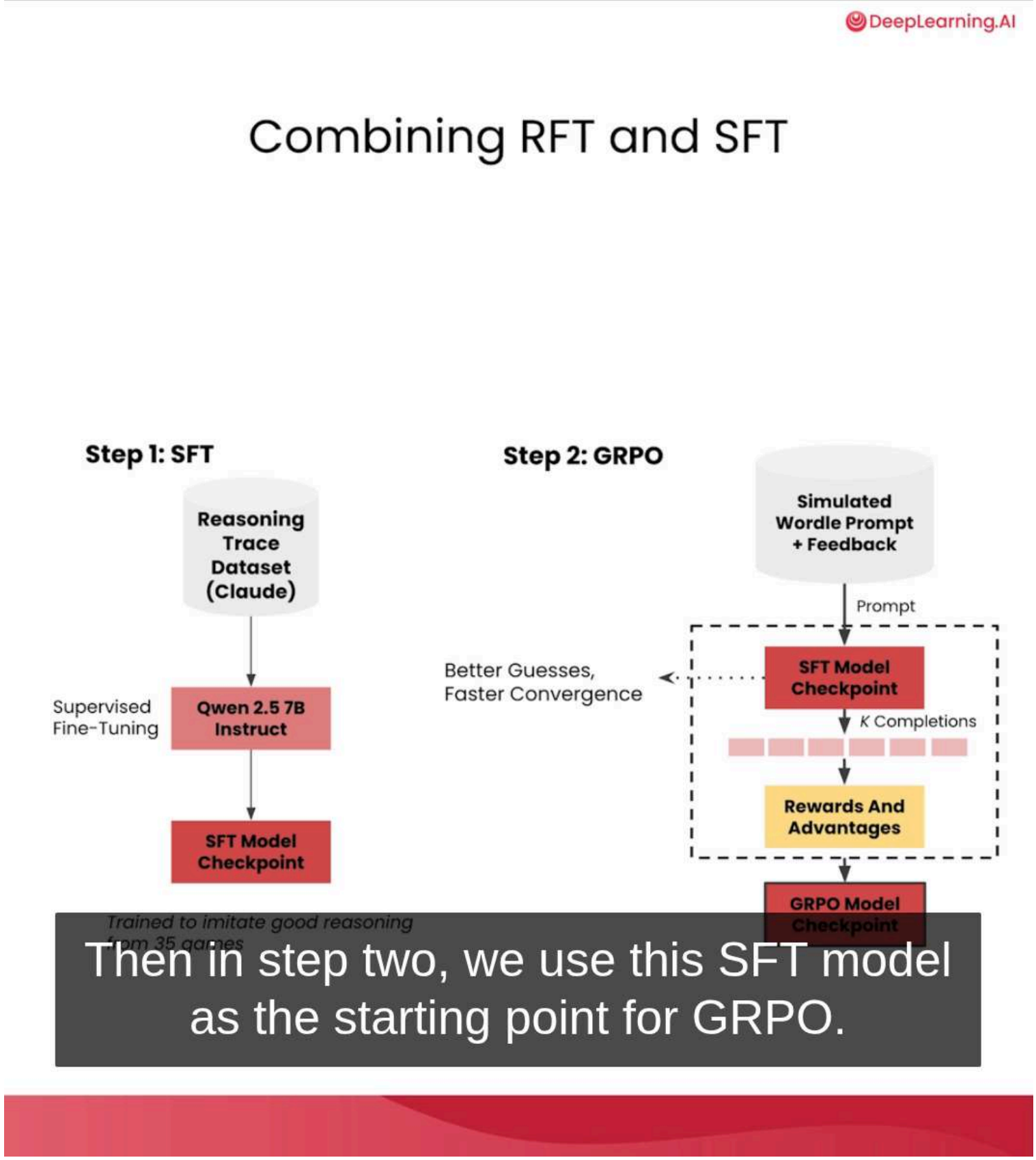
Combining RFT and SFT



for further optimization, essentially, a model that mimics good reasoning.

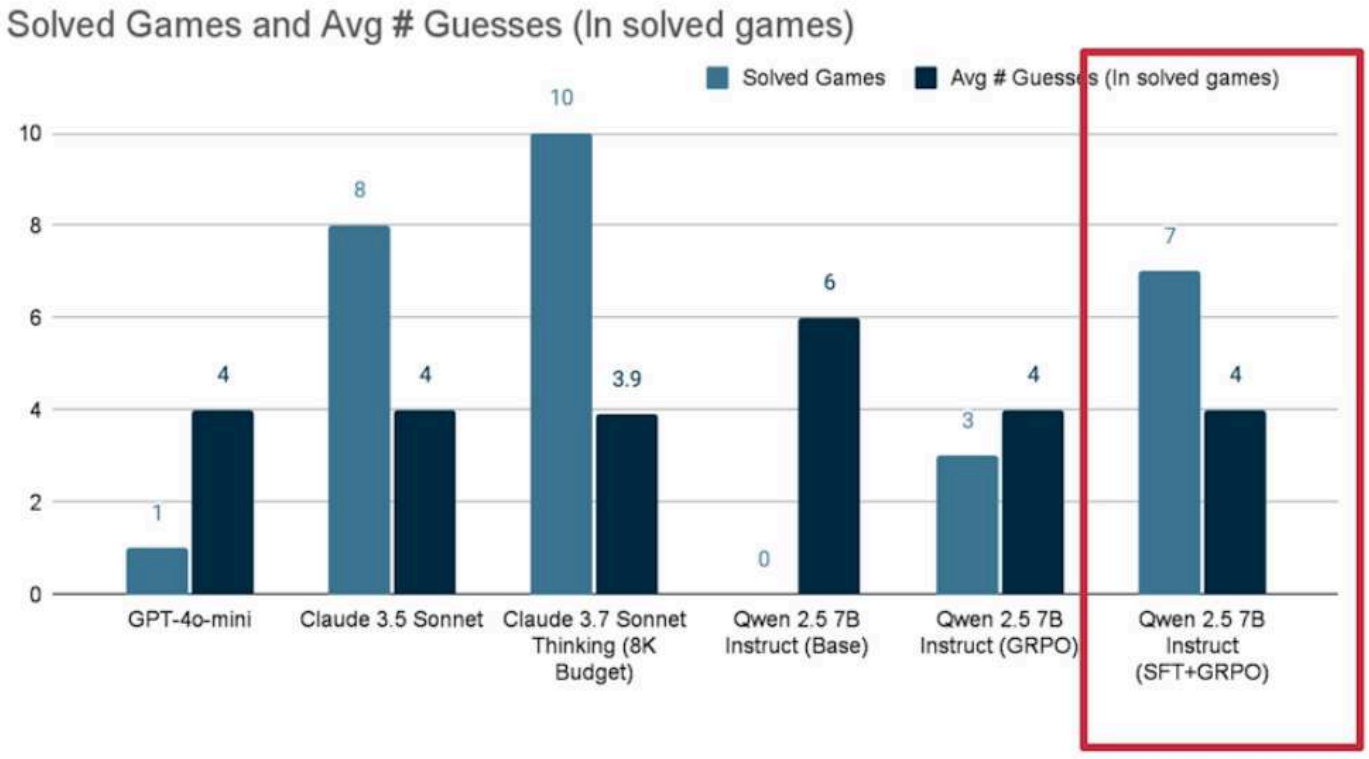
Step 2 GRPO

Use SFT model → GRPO (guess faster convergence)



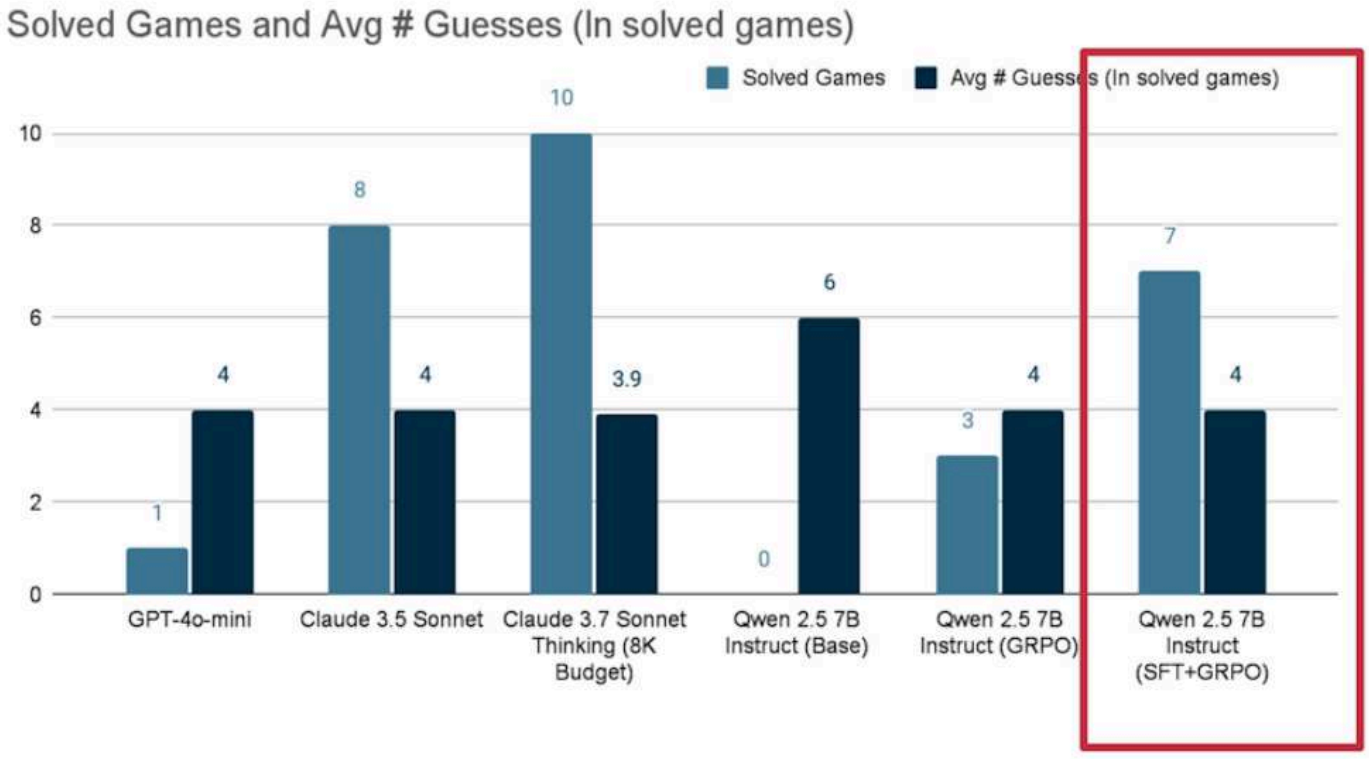
Combine SFT+ GRPO result

Results



Our Qwen 2.5 model was not able to solve seven out of ten games

Results



Check out the code at the end of the notebook to see how to set up SFT and SFT+GRPO training runs!